

Table of Contents

1. Action types • Overview • Palantir
2. Action types • Overview • Palantir
3. Action types • Getting started • Palantir
4. Action types • Use actions in the platform • Palantir
5. Action types • Rules • Palantir
6. Action types • Parameters • Overview • Palantir
7. Action types • Parameters • Set parameter default value • Palantir
8. Action types • Parameters • Filter results of a parameter dropdown • Palantir
9. Action types • Parameters • Object dropdown security considerations • Palantir
10. Action types • Parameters • Override parameter configurations • Palantir
11. Action types • Parameters • Performance considerations • Palantir
12. Action types • Submission criteria • Palantir
13. Action types • Actions on interfaces • Palantir
14. Action types • Actions on structs • Palantir
15. Action types • Function-backed actions • Overview • Palantir
16. Action types • Function-backed actions • Getting started • Palantir
17. Action types • Function-backed actions • Batched execution • Palantir
18. Action types • Side effects • Overview • Palantir
19. Action types • Side effects • Notifications • Palantir
20. Action types • Side effects • Set up a notification • Palantir
21. Action types • Side effects • Webhooks • Palantir
22. Action types • Side effects • Set up a webhook • Palantir
23. Action types • Trigger schedule build • Palantir
24. Action types • Configure sections • Palantir
25. Action types • Upload media • Palantir
26. Action types • Upload attachments • Palantir
27. Action types • Scale and property limits • Palantir
28. Action types • Inline edits • Palantir
29. Action types • Permissions • Palantir
30. Action types • Read and write authorizations • Palantir
31. Action types • Monitoring • Palantir
32. Action types • Undo or revert Actions • Palantir
33. Action types • Branching action types • Palantir
34. Action types • Action metrics • Palantir
35. Action types • Action log • Palantir
36. Action types • Add action types to a Marketplace product • Palantir

Action types

In the Ontology, users can make changes to objects, properties, and links by applying actions. An action is a single transaction that changes the properties of one or more objects, based on a user-defined logic. Actions enable users to handle and manage data while thinking about overall objectives instead of specific property edits.

An **action type** is the definition of a set of changes or edits to objects, property values, and links that a user can take at once. It also includes the side effect behaviors that occur with action submission.

Example:

You may create an `Assign Employee` action type that defines how users can change the `role` property value for a given `Employee` object. This action type could require a parameter definition enabling users to input the new role in a standardized form and can include rules for how to automatically create a link between the `Employee` object and that of a new `Manager`.

The action could also:

- Include a notification side effect that will notify the old and new manager of the change.
- Validate that authorized employees such as those working in human resources can perform the action.

With these parameters set, an HR employee can then take an action to switch "Melissa Chang" to a "Product Manager" `role`, for example.

applications. The data asset grows in richness and value as user decisions and insights are captured in the form of edits to the Ontology.

Any changes made to objects, property values, and links will be committed to the Ontology when the user takes the action and will be reflected in all user applications. Likewise, the same action logic and validations can be made available across all user-facing applications, ensuring consistent edits to the Ontology. The most up-to-date version of object data with user edits incorporated will be captured in an object type's writeback dataset.

Get started by learning how to [create an action type](#), or learn about [rules](#), [parameters](#), and [submission criteria](#).

PREVIOUS

← [Object and link types / Add Ontology types to a Marketplace product](#)

NEXT

[Getting started](#) →

© 2026 Palantir Technologies Inc. All rights reserved.

[Cookies Statement](#) ↗

[Privacy Statement](#) ↗

[Terms of Use](#) ↗

— [Cookies Settings](#)

[API Reference](#) ↗

[Send feedback](#)

Action types

In the Ontology, users can make changes to objects, properties, and links by applying actions. An action is a single transaction that changes the properties of one or more objects, based on a user-defined logic. Actions enable users to handle and manage data while thinking about overall objectives instead of specific property edits.

An **action type** is the definition of a set of changes or edits to objects, property values, and links that a user can take at once. It also includes the side effect behaviors that occur with action submission.

Example:

You may create an `Assign Employee` action type that defines how users can change the `role` property value for a given `Employee` object. This action type could require a parameter definition enabling users to input the new role in a standardized form and can include rules for how to automatically create a link between the `Employee` object and that of a new `Manager`.

The action could also:

- Include a notification side effect that will notify the old and new manager of the change.
- Validate that authorized employees such as those working in human resources can perform the action.

With these parameters set, an HR employee can then take an action to switch "Melissa Chang" to a "Product Manager" `role`, for example.

API Reference ↗

By abstracting data into an abstract data model, the Foundry Ontology maps each ontological concept to an organization's actual data, enabling this data asset to p

Send feedback

applications. The data asset grows in richness and value as user decisions and insights are captured in the form of edits to the Ontology.

Any changes made to objects, property values, and links will be committed to the Ontology when the user takes the action and will be reflected in all user applications. Likewise, the same action logic and validations can be made available across all user-facing applications, ensuring consistent edits to the Ontology. The most up-to-date version of object data with user edits incorporated will be captured in an object type's writeback dataset.

Get started by learning how to [create an action type](#), or learn about [rules](#), [parameters](#), and [submission criteria](#).

PREVIOUS

← [Object and link types / Add Ontology types to a Marketplace product](#)

NEXT

[Getting started](#) →

© 2026 Palantir Technologies Inc. All rights reserved.

[Cookies Statement](#) ↗

[Privacy Statement](#) ↗

[Terms of Use](#) ↗

— [Cookies Settings](#)

[API Reference](#) ↗

[Send feedback](#)

Getting started

In this guide, we will create a simple action type for changing the priority on a ticket.

We will configure submission criteria to make sure that the priority is `P0`, `P1` or `P2`, and that the ticket status is `Open`.

Prerequisites

For this guide, we will use a `Demo Ticket` object type, which has four properties:

- `Ticket ID`
- `Title`
- `Priority`
- `Status`

We also have two demo objects available:

Ticket ID	Title	Status	Priority
PDS-123	Demo Ticket One	Open	P2
PDS-124	Demo Ticket Two	Closed	P1

You can recreate these in your Ontology if desired, but it is not essential.

Note that for a user to be able to take an action defined in an action type configuration, [API Reference > Action is required](#). If running Object Storage v2, the user must enable edits with a toggle. If running Object Storage v1 (Phonograph), a writeback dataset must be created. Note that [Object Storage v1](#) is in a [planned deprecation](#) phase. [migrate to Object Storage v2](#).

API Reference >



Send feedback

Create a new action type

We start by creating a new action type for changing the ticket's priority. In the Ontology Manager, select **Action type** on the left sidebar, then choose **New Action type** at the top right of the view.

Create a new action type

1 Action type

2 Mapping

3 Metadata

4 Submission criteria

5 Save location

STEP 1

Select an action type you want to configure

Enable users to make changes to the ontology by configuring actions they can execute.

Object

Link

Function

Webhook

Interface

Notification

Object type

Choose an object type

Object actions

☒ Create object

Configure an action type that adds a new object instance

☐ Modify object(s)

Configure an action type that edits existing object instances.

☐ Modify or create object

Modify an existing instance, otherwise create a new instance.

☐ Delete object(s)

Skip

Next

The creation wizard allows you to configure the most important features of an action type. Enter a **Display name** for your action type. Next, select the **Change object(s)** option and set it to **Modify**. From the following dropdown, select the `Demo Ticket` object type and add the `Priority` property by selecting **Add property**. Finally, select **Create** in the bottom right.

You can now see the full detailed view of your action type. You can make additional adjustments, like adding a **Description** in the **Overview** tab or adding additional properties to modify in the **Rules** tab.

API Reference ↗

Edit parameters

Send feedback

Select the **Forms** tab to get an overview of the parameters. The **Ticket** and **Priority** parameter have already been created based by the **Rule**.

The screenshot shows the 'Change Ticket Priority' form configuration interface. The left sidebar contains a navigation menu with options: 'Back home', 'Change Ticket Priority' (selected), 'Overview', 'Rules', 'Form', 'Legacy Log', 'Log', and 'Security & Submission Criteria'. The main area is split into two panels. The left panel, titled 'FORM CONTENT', shows a list of parameters: 'Lava Ticket' (Used in Rules, New) and 'Priority' (Used in Rules, New). Below this list are buttons for '+ Add new parameter' and 'Add section'. The right panel, titled 'Form Preview', shows a preview of the form. It has a title 'Change Ticket Priority', a dropdown menu for 'Lava Ticket' with the text 'Select an option...', and a text input field for 'Priority'. At the bottom of the preview, there is a red error message 'Issue' and 'Cancel' and 'Submit' buttons. The top right of the interface shows '19 edits', a warning icon, and 'Discard' and 'Save' buttons.

Select the **Priority** parameter to limit the values it can take on. Change the constraints from **User input** to **Multiple choice**. This will allow you to pick what values can be chosen for this parameter. Add **P0**, **P1** and **P2** as options. If you applied your action to an object now, you could change the priority of a ticket to **P0**, **P1**, or **P2**. You will now add submission criteria that will restrict you to only changing the priority for open tickets.

[API Reference ↗](#)

[Send feedback](#)

Constraints

Allow multiple values
Users will be able to select more than one string

 User input

 Multiple choice



 User

 Group

☒ Define options manually

☐ Get options from an object set







 Add option



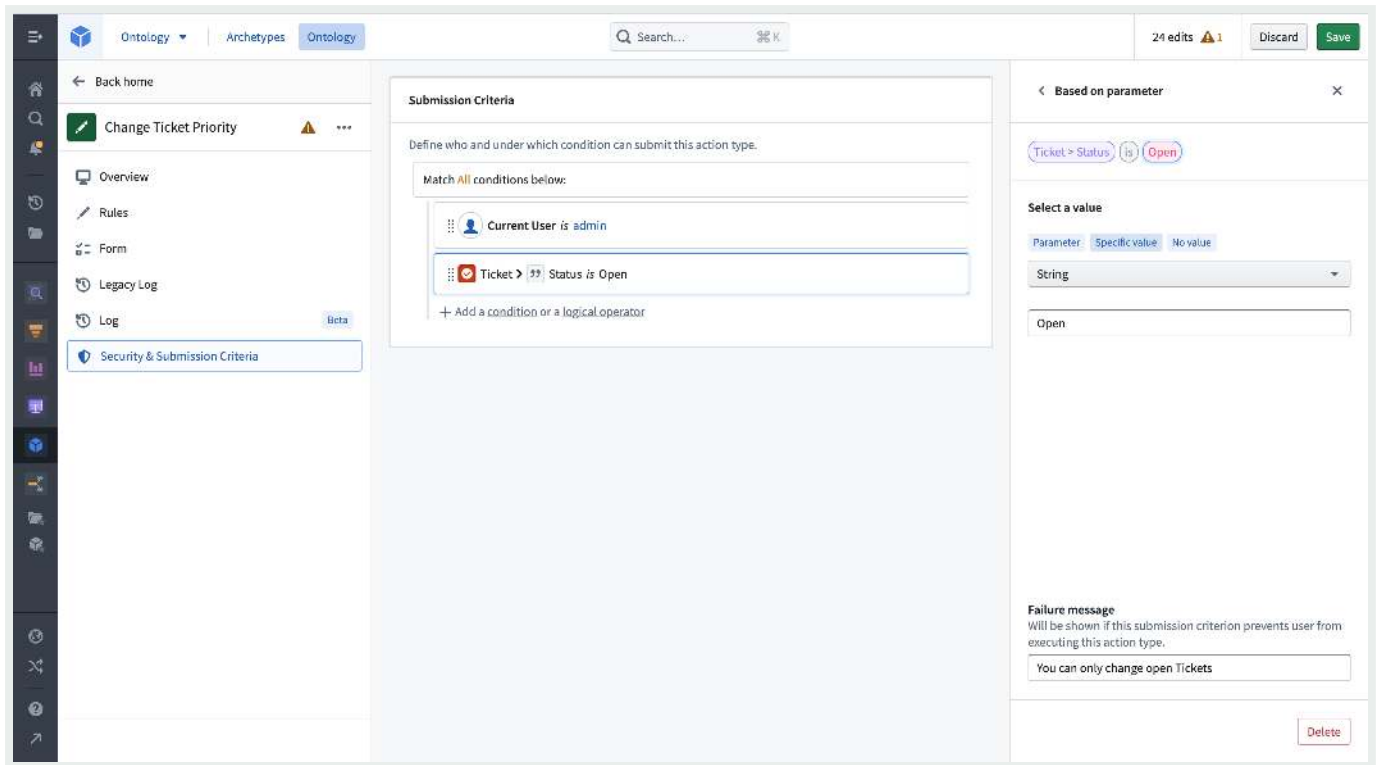
Allow "Other" value
Users will be able to fill **Priority** with their own value

Add submission criteria

Open the submission criteria section in the **Security & Submission Criteria** tab from the sidebar. Create a new condition by selecting **Condition** in the **Execution** section. Using the **Parameter** condition template, set a condition on the `Ticket` object parameter's `Status` property. Using the `is` operator, you can then do an exact string comparison between the ticket status and the specific value `Open`.

[API Reference ↗](#)

[Send feedback](#)



Add a failure message so users can see why an action has failed. Your action definition is now complete, and you can configure it to show up next to the Object View in Object Explorer.

Add the action to an Object View

Go to **Demo Ticket One** and edit its Object View. Add a new widget to the top, and choose the **Actions** widget. In the sidebar, select **Add Item**. Copy and paste the action RID from the Ontology Manager and paste it into the Action RID field. Name the label "Change Ticket Priority".

[API Reference ↗](#)

[Send feedback](#)

← Editing widget: Actions

Settings Format

ACTIONS (required)

+ Add Item

Item ^ v x

Action RID (required)

ri.actions.main.action-type

▼ BUTTON CONFIGURATION

Label (required)

Change Ticket Priority

By default, the action form will show every parameter as a field in the action form, including the `Ticket` parameter. Additionally, an action does not know that it should fill the current object in for the `Ticket` parameter. We will configure the action form to hide the ticket field (so the user cannot change the status of a different ticket), and set its value to the current object. Under **Default value**, select **Add Item**. Type the parameter ID for the `Ticket` parameter—in this tutorial, we set it to `ticket`. Change the value type to **Environment variable** and select **Current object**. Finally, change the display option to **Hidden**.

[API Reference ↗](#)

[Send feedback](#)

Default values

⊕ Add Item

Item

^ v ✕

Parameter ID (required)

ticket

Value type

Static value

Property value

Environment variable

Current object

▼

Display options

Visible

Disabled

Hidden

You will now see the action button on the preview page:

Demo Ticket One

Demo Ticket

↺

💬

Open in ▼

Actions ▼

Overview

Properties

⊕ Add tab

+ Add widget

⋮

⚙ Edit

🗑 Delete

Change Ticket Priority

⋮

☰ Properties

⚙ Edit

🗑 Delete

Ticket ID

PDS-123

Ticket Priority

P2

Ticket Status

Open

Ticket Title

Demo Ticket One

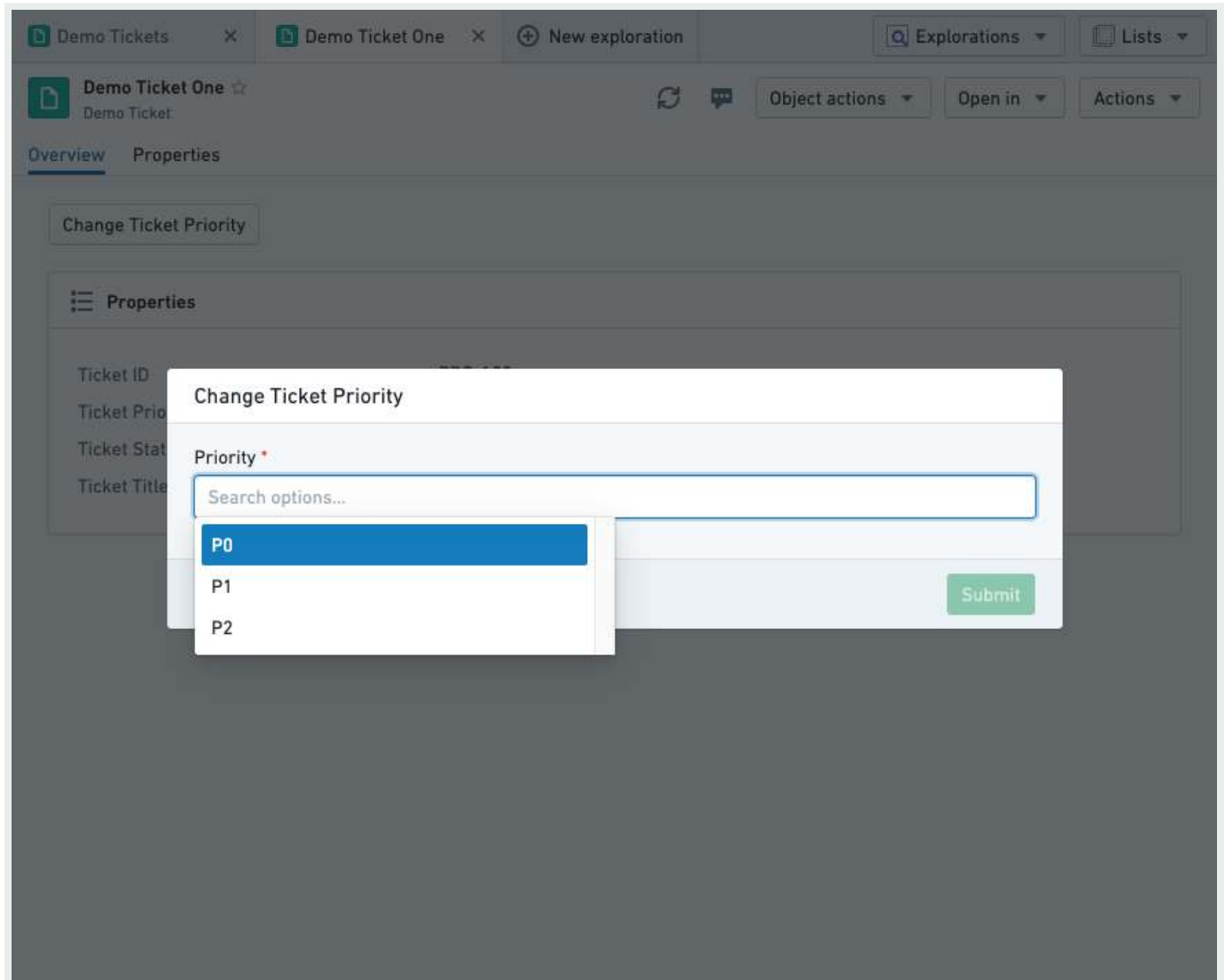
API Reference ↗

You can now save and publish the Object View.

Send feedback

Apply the action

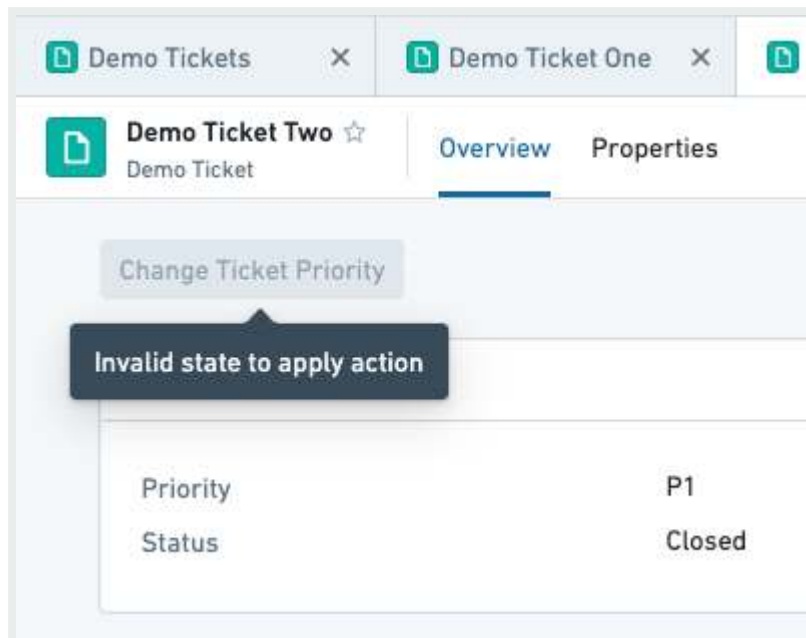
Visit an open ticket and select the **Change Ticket Priority** button we configured. You should see the action form appear over the view. Clicking into the **Priority** field will show the single selected submission criterion we configured on the parameter:



Pick a priority and select submit. The form will disappear and the object view will update with the new priority. Our submission criteria said that it should not be possible to run this action on a closed ticket—if we open Demo Ticket Two, which is closed, we see the following:

[API Reference ↗](#)

[Send feedback](#)



Resolve conflicting user edits (actions) and datasource updates

Object instances in the Foundry Ontology can be created and modified by both input datasources and user edits/actions. When a single object instance (that is, a row or object with a specific primary key value) receives data from both the input datasource and user edits, these received values must be transparently resolved with a conflict resolution strategy.

There are two strategies for resolving conflicts:

- Strategy 1: Apply user edits (default)
- Strategy 2: Apply most recent value (may not be available on your enrollment)

[Learn more about how to resolve conflicting user edits and datasource updates.](#)

Next steps

API Reference ↗

[action permissions.](#)
[-backed action.](#)

- [Use an action elsewhere in the platform.](#)
- [Resolve conflicting user edits \(actions\) and datasource updates](#)

Send feedback

© 2026 Palantir Technologies Inc. All rights reserved.

[Cookies Statement ↗](#)

[Privacy Statement ↗](#)

[Terms of Use ↗](#)

— [Cookies Settings](#)

[API Reference ↗](#)

[Send feedback](#)

Ontology building / Action types / Use actions in the platform

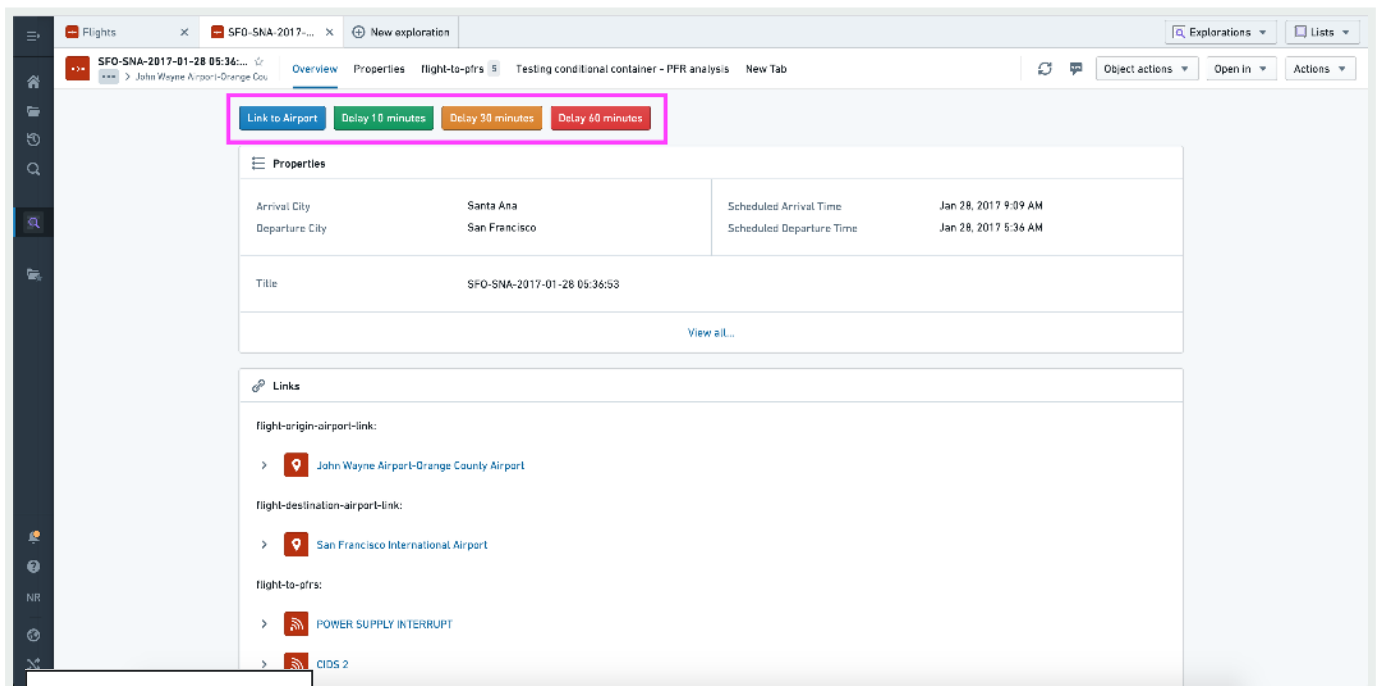
Use actions in the platform

Action types can be seamlessly integrated across applications in Foundry. Read on to learn how to configure and apply an action from [Object Explorer](#) and [Workshop](#).

In the examples below, we use the term **single action type** to refer to an action type using an object reference parameter, and **bulk action type** to refer to an action using an object reference list parameter.

Object views

Actions can be added to an [Object View](#) using the **Actions section**.



API Reference ↗

When configuring the **Actions section** you have the option to:

Send feedback

- Add any action as a button in the section.

- Give each button its own label and color.
- Change the default on-click behavior from opening the form to applying the action immediately using the default values (if valid).
- Specify whether the button should be hidden or disabled if a non-visible parameter is invalid (the idea being that visible parameters could be corrected upon opening the form).
- Provide a default value for each parameter; this can be a property value of the current object or a "local" value (current user, current timestamp, current object, or a manually entered value).
- Override the visibility of each parameter.

As shown above, you can therefore use this section to offer multiple structured versions of the same generic action ("Delay 10 minutes", "Delay 30 minutes", etc.).

Object Explorer

Actions will automatically be shown in three places across [Object Explorer](#):

1. From the **Actions** dropdown in the Exploration View (top right).

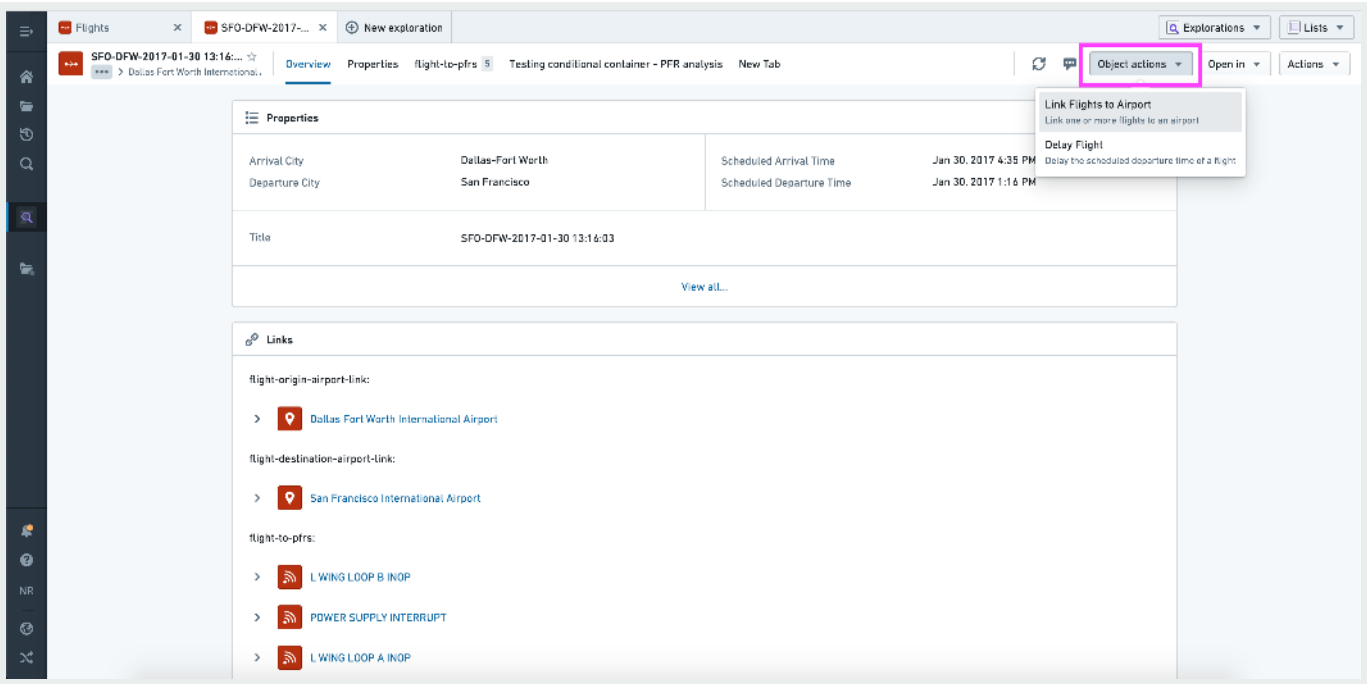
The screenshot shows the Object Explorer interface for a 'Flights' dataset. The top navigation bar includes 'New exploration' and search filters. The main table displays flight data with columns: TITLE, DEPARTURE AIRPORT CODE, ARRIVAL CITY, DEPARTURE CITY, SCHEDULED ARRIVAL TIME, and SCHEDULED DEPARTURE TIME. A dropdown menu labeled 'Actions' is open, showing options like 'Link Flights to Airport' and 'Add to list'. The table contains 346 results.

TITLE	DEPARTURE AIRPORT CODE	ARRIVAL CITY	DEPARTURE CITY	SCHEDULED ARRIVAL TIME	SCHEDULED DEPARTURE TIME
SFO-SNA-2017-01-28 0...	SFO	Santa Ana	San Francisco	Jan 28, 2017 9:09 AM	Jan 28, 2017 9:09 AM
SFO-HNL-2017-01-26 0...	SFO	Honolulu	San Francisco	Jan 26, 2017 11:20 AM	Jan 26, 2017 11:20 AM
SFO-DGG-2017-01-15 0...	SFO	Kahului	San Francisco	Jan 15, 2017 4:14 AM	Jan 15, 2017 1:17 AM
SFO-RDU-2017-01-05 1...	SFO	Raleigh-durham	San Francisco	Jan 5, 2017 2:18 PM	Jan 5, 2017 11:27 AM
SFO-BUF-2017-01-30 1...	SFO	Buffalo	San Francisco	Jan 30, 2017 3:40 PM	Jan 30, 2017 1:23 PM
SFO-TPA-2017-01-07 1...	SFO	Tampa	San Francisco	Jan 7, 2017 2:39 PM	Jan 7, 2017 1:37 PM
SFO-TPA-2017-01-21 1...	SFO	Tampa	San Francisco	Jan 21, 2017 7:19 PM	Jan 21, 2017 4:16 PM
SFO-MSP-2017-01-20 0...	SFO	Minneapolis	San Francisco	Jan 20, 2017 4:57 AM	Jan 20, 2017 1:27 AM
SFO-MCI-2017-01-30 0...	SFO	Kansas City	San Francisco	Jan 30, 2017 10:15 AM	Jan 30, 2017 7:41 AM
SFO-OAK-2017-01-27 1...	SFO	Oakland	San Francisco	Jan 27, 2017 5:02 PM	Jan 27, 2017 3:30 PM
SFO-BUF-2017-01-30 0...	SFO	Buffalo	San Francisco	Jan 30, 2017 1:18 PM	Jan 30, 2017 9:44 AM
SFO-JFK-2017-01-23 0...	SFO	New York	San Francisco	Jan 23, 2017 10:20 AM	Jan 23, 2017 6:28 AM
SFO-EWR-2017-01-21 0...	SFO	Newark	San Francisco	Jan 21, 2017 8:31 AM	Jan 21, 2017 6:44 AM
SFO-PBI-2017-01-22 09...	SFO	West Palm Beach	San Francisco	Jan 22, 2017 10:57 AM	Jan 22, 2017 9:22 AM
SFO-IAD-2017-01-11 15...	SFO	Washington	San Francisco	Jan 11, 2017 4:47 PM	Jan 11, 2017 3:23 PM
SFO-HOU-2017-01-10 0...	SFO	Houston	San Francisco	Jan 10, 2017 9:57 AM	Jan 10, 2017 6:39 AM
SFO-DCA-2017-01-25 0...	SFO	Washington	San Francisco	Jan 25, 2017 7:49 AM	Jan 25, 2017 6:49 AM

of objects, this dropdown is automatically populated with applicable bulk actions.

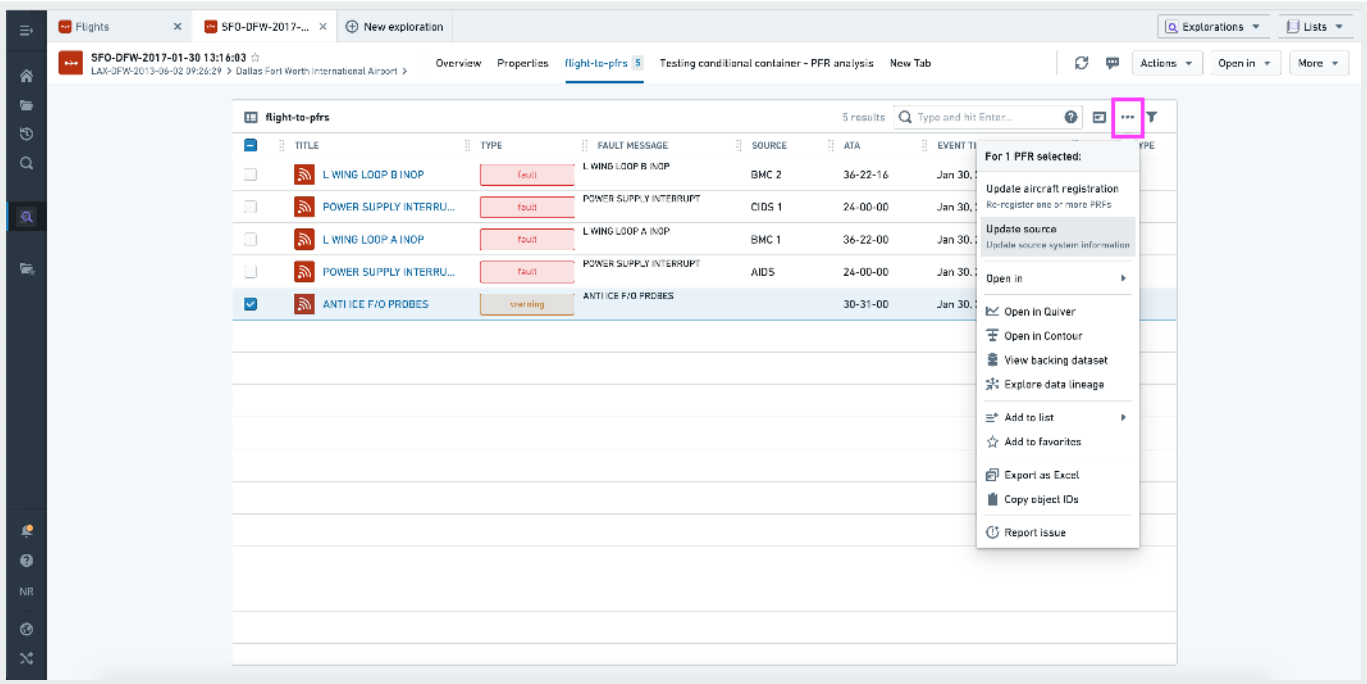
Send feedback

2. From the **Object Actions** dropdown menu in the Object View (top right).



Using the current object, this dropdown menu is automatically populated with applicable single and bulk action types.

3. From the **Linked objects view section** in the Object View (top).



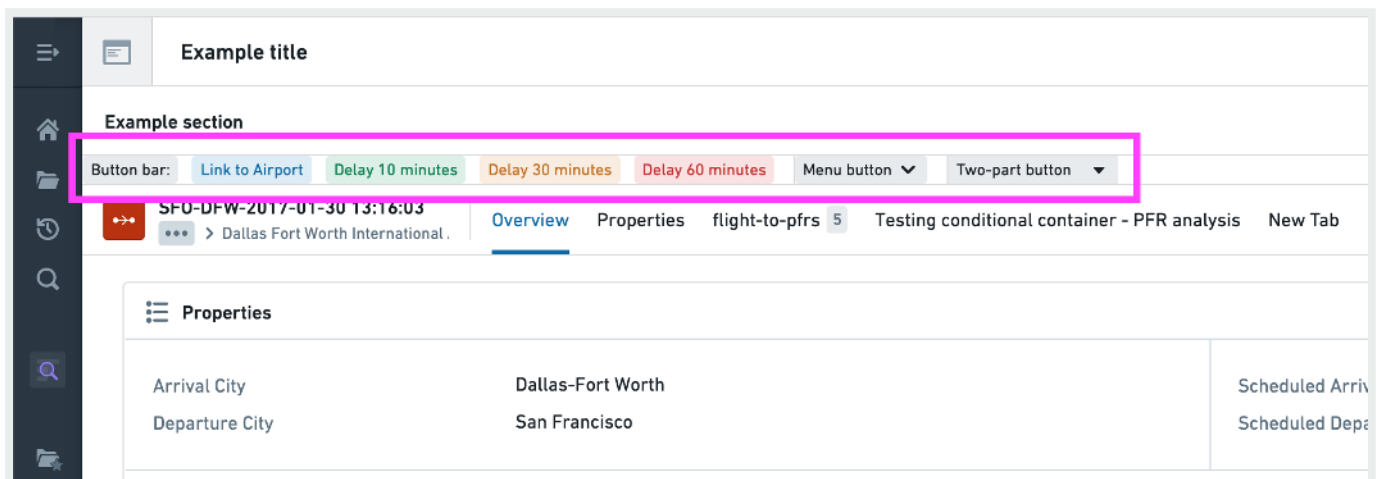
API Reference ↗ object(s), this dropdown is automatically populated with applicable single and/or bulk action types.

Send feedback

i In "bulk" contexts (where multiple objects are shown in a list view), only actions that accept object list parameters of the correct type will be shown.

Workshop

In [Workshop](#), Actions can be configured and applied using the [Button group widget](#).



This widget has the same configuration options as the [Actions section](#) in an Object View, with a few notable extensions:

- There are three possible layouts, all of which are shown above.
- The buttons have additional display options, including left/right icons, minimal styles, and tag styles.
- In addition to an Action, an individual button can trigger a Workshop event, URL, or object set export.

And one difference:

- A default value can be a [variable](#), the current user, or the current timestamp

Read more about [Actions in Workshop](#), or read the full reference for the [Button Group widget](#) to learn about all available configuration options.

[API Reference ↗](#)

PREVIOUS

← [Getting started](#)

[Send feedback](#)

[Rules](#) →

© 2026 Palantir Technologies Inc. All rights reserved.

[Cookies Statement ↗](#)

[Privacy Statement ↗](#)

[Terms of Use ↗](#)

— [Cookies Settings](#)

[API Reference ↗](#)

[Send feedback](#)

Rules

Rules define the logic of the action type that transform the parameters into Ontology edits or other effects. There are two main types of rules: ones that edit the Ontology, and ones that trigger another effect in Foundry.

Ontology rules

An Ontology rule changes specific elements of the Ontology. They can create, modify, or delete objects and links of existing types. To create or delete one-to-many or one-to-one links, object rules need to be used and the foreign key property on the object needs to be modified.

1. **Create object:** Can be used to create an object of a predefined type. The primary key of the object type is a required property which has to be filled. Additional properties can be added optionally.
2. **Modify object(s):** Can be used to modify an existing object whose primary key is derived from object reference parameters. This cannot reference an object created as a part of the current action.
3. **Create or modify object(s):** Can be used to modify an existing object based on an object reference parameter. If an object is not selected, a new object will be created with either an automatically generated unique ID, or with a user submitted primary key.
4. **Delete object(s):** Can be used to delete an existing object whose primary key is derived from object reference parameters. This cannot reference an object created as a part of the current action.
5. **Create link(s):** Can be used to create a many-to-many link between objects that are reference parameters. For foreign key links, one has to use **Modify** to explicitly modify the foreign key property.

API Reference ↗



Send feedback

6. **Delete link:** Can be used to delete a many-to-many link between objects that are passed via object reference parameters. For foreign key links, one has to use **Modify object** rule to explicitly modify the foreign key property.
7. **Function rule:** Can be used to reference an Ontology edit function whose inputs are derived from parameters of the action. When this rule is present, no other rule may be configured since function code alone is capable of handling everything that other rules can do. Read more about [function action types](#).
8. **Create object(s) of interface:** Can be used to create objects of any type that implements a particular interface. Read more about [actions on interfaces](#).
9. **Modify object(s) of interface:** Can be used to modify objects of types that implement a particular interface. Read more about [actions on interfaces](#).
10. **Delete object(s) of interface:** Can be used to delete objects of types that implement a particular interface. Read more about [actions on interfaces](#).
11. **Create link(s) on object(s) of interface:** Can be used to create links between objects of types that implement a particular interface. Read more about [actions on interfaces](#).
12. **Delete link(s) on object(s) of interface:** Can be used to delete links between objects of types that implement a particular interface. Read more about [actions on interfaces](#).

Values and parameters

When creating or modifying links and objects, the Rules require additional values for their operation. When modifying an object, the rules also define which properties are modified. Each property in return is mapped to a value provided by one of multiple options (Rules on links can only take object reference parameters):

- **From parameter:** An existing parameter of the same type as the property. By default, every new property being added to the rule will automatically create a parameter with the same name and will be mapped to take the value of this parameter.
- **Object parameter property:** A property of an existing object reference parameter. The property type of the object parameter needs to match the property type it is mapped to.
- **Static value:** A static value that only exists in the Rules part of the action type. This value is cannot be changed when interacting with the action in Workshop, Slate, or Object Views.

API Reference ↗

Contextual values: String and timestamp properties can also take on contextual values like the actions current user or the time of submission. Just like the **Static value**, these values cannot be interacted with when submitting the action. They are only used in other parts of the action type.

Send feedback

Creating an object & many-to-many link

You can also create objects and linked many-to-many at the same time. While just creating a many-to-many link requires objects on both sides of the link to exist prior, you can create both entities via one action type. Start by configuring a **Create object** rule with an object type that has a many-to-many link. Then click the **Add link** button below **Add property** to select the link type and configure the links.

In order to create a one-to-many or one-to-one link type, simply edit the foreign key on the object.

Invalid combinations

Action types can include combinations of Ontology rules. When multiple rules are defined, the actions backend compiles rules to generate a single edit per object (e.g., **Add object**, **Modify object(s)**, or **Delete object(s)**). For example, if the result of one rule updates a property to "A", but another rule in the same action type updates the same object's property to "B", the resulting edit would just update the property to "B". The order of rules affects the final object edit. As a result, the following combinations of object edits are not supported:

- Objects cannot be deleted before they are added or modified.
- Objects cannot be modified before they are added.
- Objects cannot be created twice in one form submission.

Other rules

There are two types of rules that trigger a [side effect](#):

- [Notification](#) rules can be used to send a notification about the action. Parameters may be used to customize the content of the notification and the recipients. End users may adjust their preferences to receive notifications via in-platform push notification, email, or both. Notifications are sent once all action edits have been applied, however the notification will be generated based on the state of the Ontology before

API Reference ↗

- [Webhooks](#) enable an action to make a request to an external system when an action is applied. Action parameters can be passed into the webhook, which can in turn pass

Send feedback

parameters through to the external request. Webhooks may be configured to run before or after edits are applied.

There is an additional advanced rule that triggers a [build](#):

- [Schedule](#) rules can be used to trigger the build of a schedule. Action parameters can be passed into the schedule, which can in turn pass parameters to the underlying [parameterized transforms](#) inside the build. Foundry applies Ontology edits after the build begins.

PREVIOUS

← [Use actions in the platform](#)

NEXT

[Parameters / Overview](#) →

© 2026 Palantir Technologies Inc. All rights reserved.

[Cookies Statement](#) ↗

[Privacy Statement](#) ↗

[Terms of Use](#) ↗

— [Cookies Settings](#)

[API Reference](#) ↗

[Send feedback](#)

Parameters

Parameters are the inputs of an action type. They are the interface between the **Rules** and other Foundry applications, such as [Workshop](#), [Slate](#), and [Object Views](#). Parameters are treated like variables that contain external values. Each parameter is defined by a type, which dictates what kind of values it can take. Beyond its type, parameters have a variety of other potential configurations. Each parameter can be individually configured as to whether they are exposed in the form or not, or whether they can be changed by the user or not.

Parameters transport values across the action type and can be referenced in rules to pass the value back on an object, link, or side effect, in submission criteria, to check if an action can be submitted, to access the current value of an object property before it is changed by the action or in overrides to change the configuration of a following parameter.

Example

A parameter can take the form of a `Ticket` object type in an action type which allows users to modify the status of a selected ticket. A `Status` parameter is defined as a string. When submitting the action, the object type parameter will take the value of a selected `Ticket` object and the `Status` parameter contains the future status. The action type then passes both parameter values to the rules and executes them to edit the object.

Example

As a variable in `Workshop`, `previous_status` can take the current value of the property of the selected `Ticket` object. This can be passed to a hidden parameter in the action, `Previous Status`, and the `Status` parameter can contain the updated status. Upon submitting the action, the action type

API Reference ↗



Send feedback

the `Previous Stats` and the `Status` values to the rules and executes them to edit the object.

← PREVIOUS
Rules

Set parameter default value →
NEXT

© 2026 Palantir Technologies Inc. All rights reserved.

[Cookies Statement ↗](#)

[Privacy Statement ↗](#)

[Terms of Use ↗](#)

—[Cookies Settings](#)

[API Reference ↗](#)

[Send feedback](#)

Set parameter default value

Default values for action type parameters are used to prefill parameters in the action form. Default values are configured at the parameter level and are supported in Workshop, Object Explorer, Object Views, Quiver, and Slate. They can be deployed to standardize action logic across multiple consuming applications, eliminating the need to add default values in each application individually.

Parameters can be set to default values to display either a fixed value or a property of the selected object.

Default value interaction with local variables

Local default values (for example, Workshop variables) always take precedence over global default values. When passing any Workshop variable to an action with a default value, the action form will prefill with the values from the Workshop variables. The same pattern applies with environment variables from Object Views and defaults from Slate. Defaults provided in each instance of an action take precedence. Any migration to default values will therefore require removing local overrides.

Configuring default values

Selecting any parameter opens the parameter configuration view for that parameter. Select whether the parameter should default to a fixed value or with a value from the property of an

object parameter

API Reference ↗

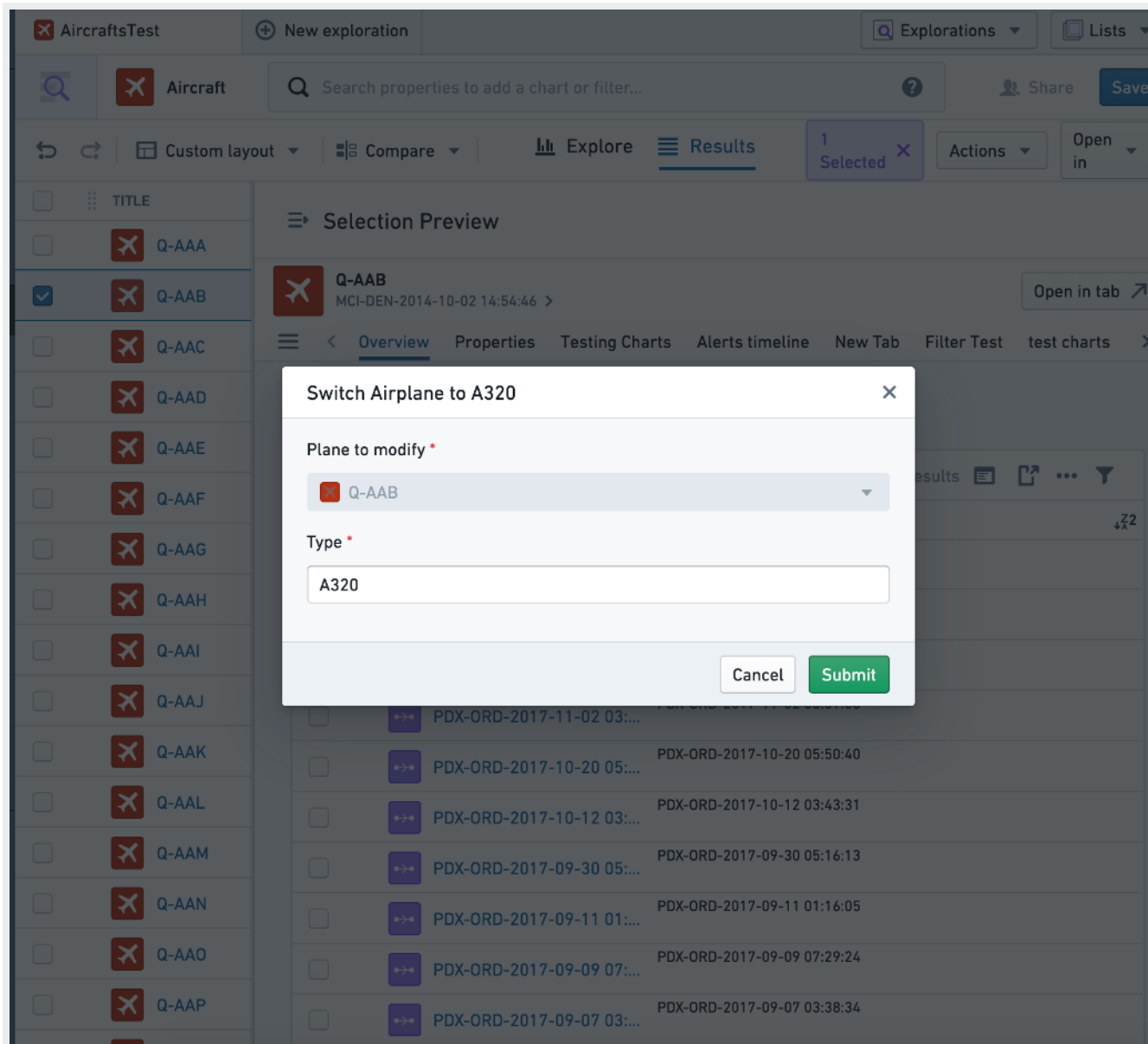
Imagine an example action type that modifies the `Type` property of a selected `Aircraft` object to become `A320`. To configure, click into the `Type` parameter and add a static default value.

The screenshot shows the 'Ontology' configuration interface. On the left, a sidebar contains navigation links: 'Back home', 'Switch Airplane to A320', 'Overview', 'Rules', 'Form' (selected), 'Legacy Log', 'Log', and 'Security & Validations'. The 'Form' section is expanded, showing a list of parameters with 'Type' selected. The main panel displays the configuration for the 'Type' parameter. It includes a 'General' section with 'Visible' (checked), 'Disabled' (unchecked), and 'Required' (checked) options. The 'Constraints' section has 'Allow multiple values' (unchecked) and 'User input' (checked) selected. The 'Default value' section is active, showing 'Object parameter property' and 'As static value' options. The 'As static value' option is selected, and the text 'A320' is entered in the input field. The bottom of the sidebar shows a 'History' section with 'Last edited 1 minute ago'.

To achieve a similar user experience without default values, input values would need to be configured in each application that uses the parameter. Updating this behavior (for instance, to `A380`) would require manually modifying the behavior, possibly across multiple applications.

[API Reference ↗](#)

[Send feedback](#)



Object property default values

To set an object property as the default value for a parameter, begin by selecting an object parameter to configure. Let's assume a more generic action type called `Change Airplane Details` where, for example, users need to know the current value of a property before making edits. This can be achieved by configuring the value of each parameter to be prefilled from the currently selected object (in our case, the `Plane` object to modify). Only object reference parameters that are placed above the parameter in the input list are available to be used as a default value.

[API Reference ↗](#)

[Send feedback](#)

Ontology Archetypes Ontology

Search...

Discard Save

Back home

Switch Airplane to A320

Overview

Rules

Form

Type

Legacy Log

Log

Security & Validations

History
Last edited 4 minutes ago by

Visible
Add override

Disabled
Add override

Required
Add override

Constraints

Allow multiple values
Users will be able to select more than one string

User input Multiple choice User Group

With minimum length

With maximum length

Matching regex

Default value
Set a default value for the input, it can be overridden by users.

Object parameter property As static value

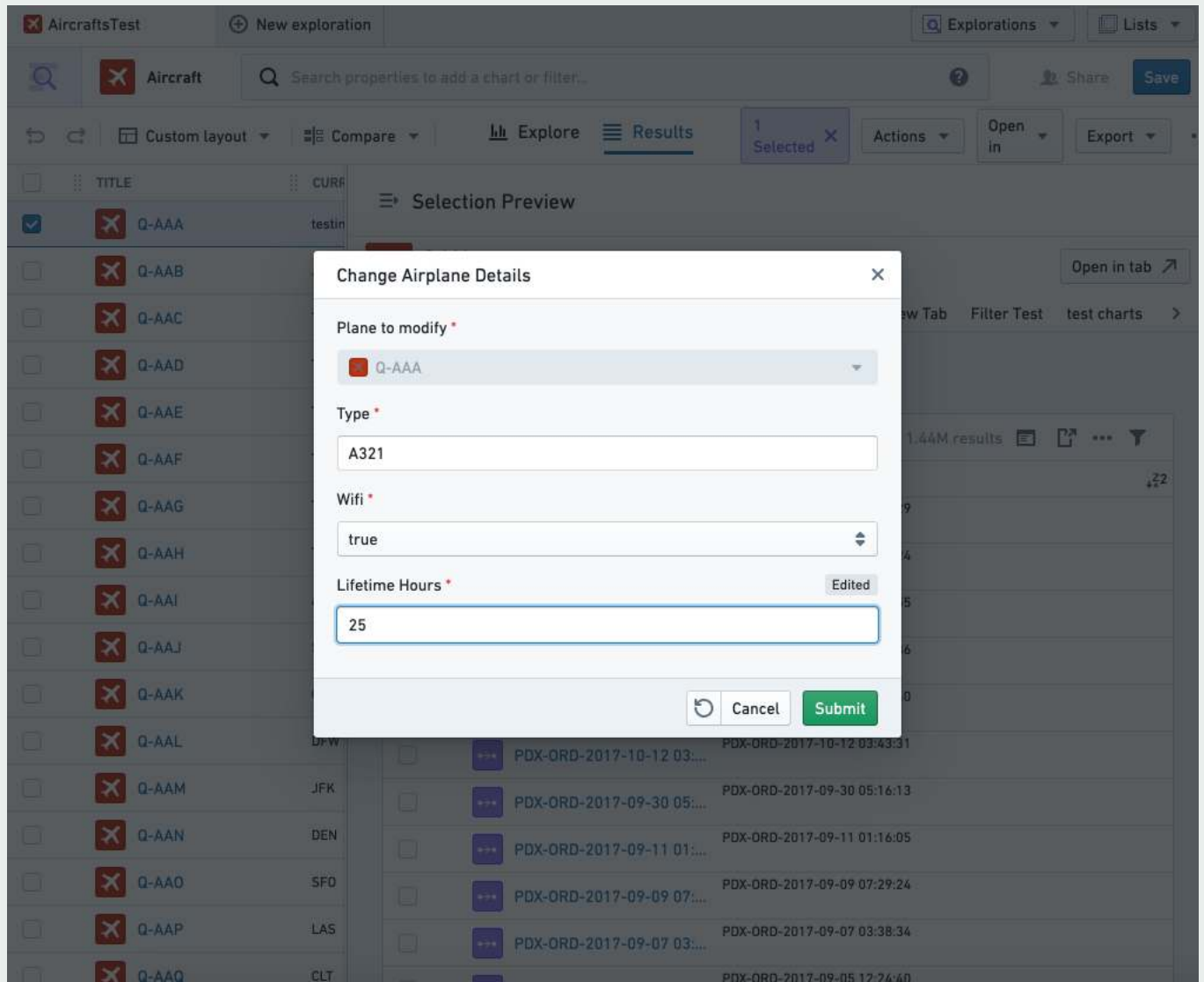
Aircraft

Type Active

In Object Explorer, the **Change Airplane Details** action will be prefilled with current values. In this case, users could choose to modify just one property and keep the rest the same. This same default logic will be present anywhere the action is submitted. Note that the **Lifetime Hours** value shows as edited once this default value is updated by the action user.

[API Reference ↗](#)

[Send feedback](#)



Type class prefills

Action parameters can be prefilled with special values (such as automatically-generated UUIDs or the current user's ID) by annotating them with type classes. The Ontology documentation has [a complete list of the available type classes](#).



In most cases, you should set the parameter visibility to `hidden`, so that you don't have to manually change these special prefilled values.

Send feedback

© 2026 Palantir Technologies Inc. All rights reserved.

[Cookies Statement ↗](#)

[Privacy Statement ↗](#)

[Terms of Use ↗](#)

— [Cookies Settings](#)

[API Reference ↗](#)

[Send feedback](#)

Ontology building / Action types / Parameters / Filter results of a parameter dropdown

Filter results of a parameter dropdown

Adding filters to non-object reference multiple choice or single object reference parameters will determine the allowed values that are selectable in the parameter's dropdown.

Multiple choice parameter dropdowns

When configuring multiple choice parameter dropdown menus, action editors can reduce allowed values to just those that are properties of an object set. This can be leveraged to display or prefill values based on properties of a linked object. To accomplish this, ensure the parameter is set to display multiple choices, select **Get options from an object set**, configure the desired object set, and select the property that includes all allowed values for the parameter dropdown. If only one linked object is available in the resulting object set and the parameter is required, the parameter dropdown will automatically prefill with the corresponding property value. The resulting multiple choice options will be derived from the set of objects that the user has permission to view. In other words, when deriving multiple choice options from an object set, users will not see properties of objects to which they do not have access.

[API Reference ↗](#)[Send feedback](#)

Constraints

Allow multiple values
Users will be able to select more than one string

User input

Multiple choice ✓

User

Group

☐ Define options manually

☒ Get options from an object set

The terms of this filter are exposed to any user who views this action. [See more](#)

Starting object set

Flight

+ Filter on a property

Search around

Destination

Experimental

Object dropdowns

Within the parameter configuration view, action editors can specify filters and Search Arounds to limit the objects that show up in the dropdown across all action interfaces. After configuring the filters, the action form will render a dropdown with only objects that match the filter. The value selected is also validated before the action is executed.

For example, an object dropdown configured to only show **Stock Series** where the **Name** is equal to the value in the `Name` parameter.

[API Reference ↗](#)

[Send feedback](#)

Starting object set

 Stock Series

Name is ▾

 Name X

+ Filter on property

 Search around

[Reset filters](#)

The image below shows the possible values for the `Name` parameter:

Name *

Apple Inc.

Stock series *

Select an option...

 Search...

 Apple Inc. close

 Apple Inc. dividend

 Apple Inc. open

 Apple Inc. vol

Data privacy implications

When using the new validation on an object parameter, it's possible for data to be viewed by everyone who can view the action type. If there are sensitive static values in the parameter filters, users will be able to view those values even if they cannot view the underlying objects that are being filtered. [Learn more about the data privacy implications.](#)

[API Reference ↗](#)

Supported operations

[Send feedback](#)

Filtering on a property

The object dropdown only shows objects where the specified property matches any of the provided values.

 The terms of this filter are exposed to any user who views this action. [See more](#)

Starting object set

 Flight

Origin is

EWR

JFK

LGA

+ Filter on a property

Search around

Origin Airport

 Flight

Destination

Experimental

The value can be statically defined by the user, inferred from another parameter, or a property of an `Object Reference` parameter. If more than one value is provided to compare against, the result will be an **OR** operation.

Changing the starting object set

The **starting set** for the query is set to all objects of the object type by default, but this can be changed to any other type. The starting set could also be set to an `ObjectReference` list parameter.

[API Reference ↗](#)

[Send feedback](#)

The terms of this filter are exposed to any user who views this action. [See more](#)

Starting object set

Airport

+ Filter on a p

Search around

airpo

ALL OBJECTS OF TYPE

Airport

EXISTING OBJECT SET VARIABLE

Flight

maintenance_iss

Default value

Set a default value

Search Arounds

A Search Around would create a new set by traversing a link on every object in the current set. For example, `Github Issue of Current Employee` would take the `Employees` in the current set and create a resulting set of `Github Issues` linked to those `Employees`.

[API Reference ↗](#)

[Send feedback](#)

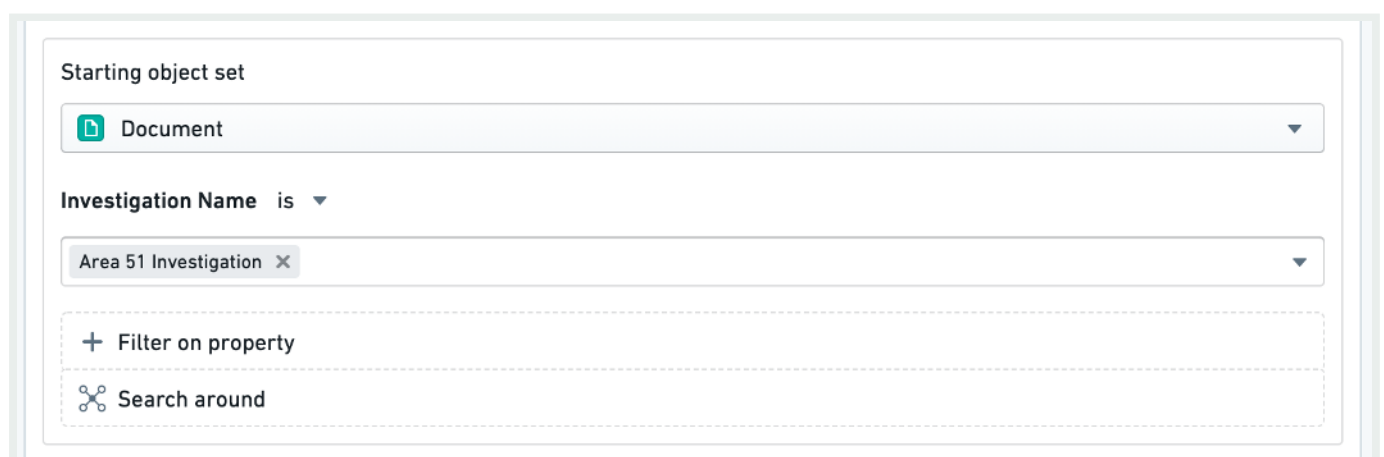
Ontology building / Action types / Parameters / Object dropdown security considerations

Object dropdown security considerations

Static value filters in object dropdown validations are exposed to all users who can view the action type. Use of these filters risks exposing property value combinations to users without permissions to view the filtered objects. This risk is mitigated by relying on object properties or parameters to filter the object set. The values are not directly visible in the interface.

Example: Data privacy issue

As an example, imagine we have a **Document** object with an **Investigation Name** property. In our action type, we add a filter on the object reference parameter to only show **Documents** where **Investigation Name** is **Area 51 Investigation**.



Starting object set

Document

Investigation Name is ▾

Area 51 Investigation ✕

+ Filter on property

🔍 Search around

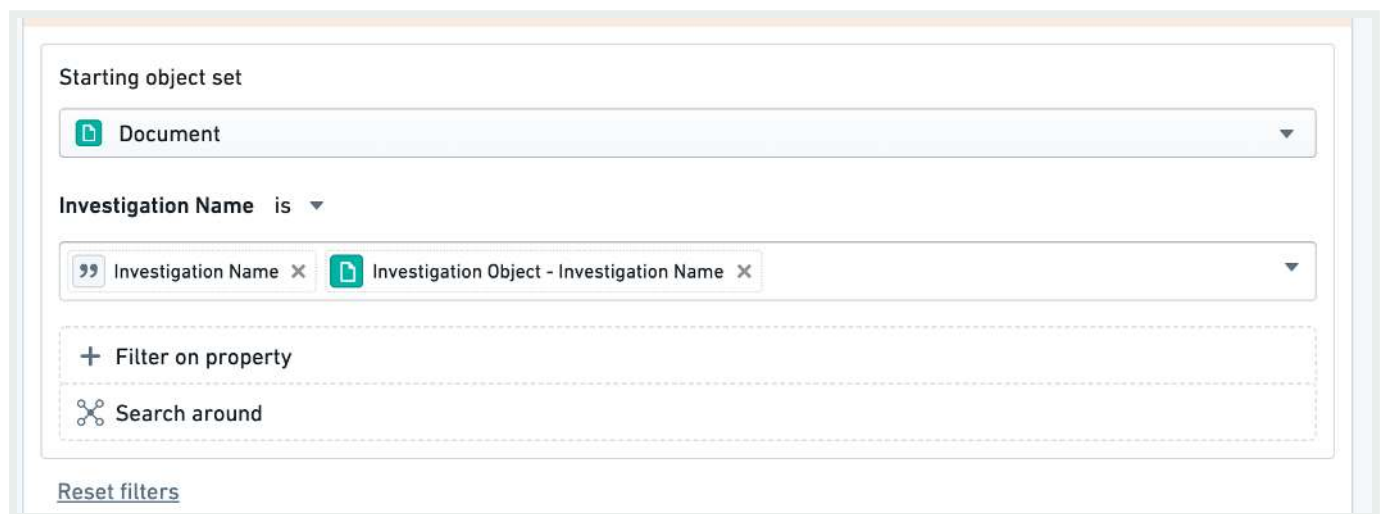
API Reference ▸ Initially be revealing that **Area 51 Investigation** is a property value of **Document** objects to users who cannot view those documents.

Send feedback

This only applies to **static value filters**. There is no reference to the `Area 51 Investigation` when filtering the `Investigation Name` property by a parameter or by the property of another object because:

- The `Investigation Name` parameter is user-provided. No information about the underlying data is exposed to the action type viewer.
- The `Investigation Object` parameter will respect existing restrictions on object visibility for this user.

Therefore, neither of these search queries represents a data privacy concern.



The screenshot shows a search interface with the following components:

- Starting object set:** A dropdown menu with 'Document' selected.
- Investigation Name is:** A dropdown menu with 'Investigation Name' selected.
- Filters:** A list of filters including 'Investigation Name' and 'Investigation Object - Investigation Name'.
- Filter on property:** A button with a plus sign and the text 'Filter on property'.
- Search around:** A button with a magnifying glass icon and the text 'Search around'.
- Reset filters:** A link at the bottom left.

Technical details

In most cases, the actions backend redacts sensitive information in the action type definition to avoid exposing sensitive property values. For example, action submission criteria are hidden from users who cannot edit action types. Similarly, a user will not be able to see the new object dropdown filters in the action type definition in the interface or while inspecting the response in the backend.

However, when viewing the action form, the object dropdown validation is converted into an object set. This means that users could review the network request containing this object set. In the example above, the user would receive an object set RID containing the `Investigation Name = 'Area 51 Investigation'` filter, revealing the existence of that p API Reference ↗ if they could not view any of its corresponding objects.

This means that these values will **not be visible in the interface** for a [Send feedback](#) a greater concern than security, this warning can be ignored.

© 2026 Palantir Technologies Inc. All rights reserved.

[Cookies Statement ↗](#)

[Privacy Statement ↗](#)

[Terms of Use ↗](#)

— [Cookies Settings](#)

[API Reference ↗](#)

[Send feedback](#)

Ontology building / Action types / Parameters / Override parameter configurations

Overrides

Overrides are used to change a parameter's behavior and configuration under specific circumstances. Using overrides, parameters and forms can become more flexible, removing the need to configure separate action types with only minor variations. Appropriate use of overrides can improve the user experience by guiding users through an action submission.

For example, let's assume that you have an action type which changes the status of a support ticket object and you want to restrict action submission to managers and assignees. While assignees can change the status, managers will have to provide a justification. Using overrides, the `Justification reason` parameter can be made required and visible for managers, while it is hidden and optional for the assignee.

Add and edit overrides

You can add and edit overrides from different places on the parameter view. The easiest way to add a new override is directly from the **Value** tab in the **General** section. By clicking **Add override** on one of the three options, you can easily create an override via the pop-up, which now automatically configured the override based on the selected option. The **General** section also shows when and how many overrides have already been configured for one of the options. To edit existing overrides, select the override button.

[API Reference ↗](#)[Send feedback](#)

Add override

Condition on New parameter overrides visibility

Delete

If

Match All conditions below:

Edit condition in the sidebar...

+ Add a condition or a logical operator

Then

Is Hidden

+ Add override

Select condition template

Based on current user

Define who can execute this action type.

Current User

Based on parameter

Define what the value of parameters and environment variables must be in order to execute this action type.

Parameter

Delete

Cancel

Done

You can also add an override manually via the **Overrides** tab. The overrides tab shows an overview of all overrides configured for the parameter. You can add override blocks from here or add new conditions or overrides to existing blocks.

Justification reason

No description

Value

Display

Overrides

Details

Type

String

Override this parameter when conditions are met.

"Justification reason is hidden when current user is admin"

+ Add conditional override

An override block presents the basis for overrides. It defines both the conditions (shown in the "if" part) and the overrides (shown in the "then" part). Each block's header shows a summary of the logic. Every parameter can contain multiple override blocks, however, if more than one is true, only the first one will be executed.

The screenshot shows a configuration interface for an override block. The header is "Condition on Ticket overrides requiredness" with a "Delete" button and a dropdown arrow. The "If" section contains a "Match All conditions below:" label and a list of conditions. One condition is "Current User groups is Managers". Below the list is a link to "Add a condition or a logical operator". The "Then" section contains a "Is Required" toggle switch. At the bottom is a button to "Add override".

"If" and conditions

Each block can contain one or multiple conditions. To read more about conditions and how to configure them, see the [submission criteria documentation on conditions](#). The only difference between override conditions and submission criteria conditions is that only parameters which appear above the current parameter in the form hierarchy can be referenced in override conditions.

"Then" and overrides

The **Then** section defines the overrides which will be applied when the conditions of the block are met. Each block can contain multiple overrides in its **Then** section, which are all be applied together. An override can change the configuration of the parameter's constraints, visibility, requiredness, and default values. If an override is configured to take on the default already set on the parameter, a warning will be shown on the override itself.

API Reference ↗

Send feedback

Multiple override blocks

You can add multiple override blocks to a single parameter. If more than one block is true, only the first override is executed.

PREVIOUS

← [Object dropdown security considerations](#)

NEXT

[Performance considerations](#) →

© 2026 Palantir Technologies Inc. All rights reserved.

[Cookies Statement ↗](#)

[Privacy Statement ↗](#)

[Terms of Use ↗](#)

— [Cookies Settings](#)

[API Reference ↗](#)

[Send feedback](#)

Performance considerations for parameter configuration

Dependencies between parameters, such as in the definitions of [default values](#) and [multiple-choice options](#), can impact the time that it takes for an action form to load. For example, consider the following action parameter configuration:

1. The first parameter is an `object reference` with a `from single result of object set` default value.
2. The second parameter is a string with a default value that is an `object parameter property` referencing the first parameter.
3. The third parameter is also a string, with no default value, but configured as a `multiple choice dropdown` using `get options from an object set`. The object set definition references the second parameter.

When a user loads an action form for this action, multiple operations need to be performed iteratively.

1. First, the default value for the first parameter needs to be retrieved.
2. Then, the second parameter's default value needs to be derived from the first parameter value.
3. Finally, the options for the third parameter need to be derived from the second parameter value.

When configuring action parameters, it is recommended to keep the dependency hierarchy as flat as possible. In the context of the action described above, referencing the first

parameter in the second parameter in the third parameter's object set definition would require the necessary information for the second and third parameters to be retrieved.

API Reference ↗

Send feedback

parallel, reducing the total latency between opening the form and the form being fully interactive.

PREVIOUS
← [Override parameter configurations](#)

NEXT
[Submission criteria](#) →

© 2026 Palantir Technologies Inc. All rights reserved.

[Cookies Statement ↗](#)

[Privacy Statement ↗](#)

[Terms of Use ↗](#)

— [Cookies Settings](#)

[API Reference ↗](#)

[Send feedback](#)

Submission criteria

Submission criteria (formerly known as validations) are the conditions that determine whether an action can be submitted. Submission criteria support encoding business logic into data editing permissions, ensuring Ontology data quality and editing governance.

Submission criteria are created by combining conditions based on the context (like a user or a parameter) and static information to create a logical statement. Submission criteria can incorporate object, relation, and even user information into a logical statement to determine whether an action can be submitted.

Example

For example, an airline might want to change the airplanes listed for a specific flight. The configured action allows users to change the **Aircraft** object linked to a **Flight** object. However, the airline only wants selected users (like a flight controller) to be able to use this action, in order to ensure that only aircraft which are still in operation are used. Using submission criteria, builders can ensure that an action which changes the airplane on a flight can only be submitted when the criteria are met by combining the user's membership to a group with the airplane's status at the moment that the action is submitted.

[API Reference ↗](#)[Send feedback](#)

Execution

Match **All** conditions below:

Current User groups is part of Administrators

Match **Any** conditions below:

Support Ticket > **Request Type** is Customer Request

Support Ticket > **Request Type** is Mechanic Request

+ Add a [condition](#) or a [logical operator](#)

Support Ticket > **Requested At** is 2022-03-01T00:00:00+01:00

+ Add a [condition](#) or a [logical operator](#)

Submission criteria consist of conditions and operators. Conditions are single statements governing the values of parameters or user properties. Operators are used to combine and nest different conditions.

Using the different types of operators, we can create more sophisticated statements, mirroring their business processes and requirements. Actions can only be submitted if all the submission criteria are met. This is independent from the permissions that govern whether a user can edit the action type itself. While an object type can have several action types adding, modifying, and removing objects, each action type has independent submission criteria.

Conditions

A condition is a single comparison check between two values. Each condition either passes or fails based on its parameter or user inputs. A condition can be configured using one of two condition templates: “based on current user” or “based on parameter”. These templates provide a framework for the rest of the condition. Every condition is a simple comparison between two values using an operator in the middle.

[API Reference ↗](#)

[Send feedback](#)

Select condition template

Based on current user

Define who can execute this action type.

 Current User

Based on parameter

Define what the value of parameters and environment variables must be in order to execute this action type.

 Parameter

Example

Continuing with our example, the flight controller requirement can be set using the `Current User` template, as it requires the context of the person submitting the action. To know whether an aircraft is still in operation, the `Aircraft` object needs to be used via the `Parameter` template.

Current user

The `Current User` template defines permissions based on the user submitting the action. The `Current User` input can be used to check a user's ID, group memberships via group IDs, or any other multipass attribute available (such as the user's Organization). Foundry evaluates user IDs as strings, which can be compared against either a statically defined list of user IDs or any string parameter that stores a user ID.

The group IDs option allows you to create conditions using the groups for which the action's user is a member (whether direct or inherited membership). The groups can be compared against a static selection of groups or the group ID provided by other parameters.

Multipass attributes are treated as string lists and can only be compared against other strings. [API Reference ↗](#) A user will have a list of proposed multipass attributes that the user has access to. Using the `other user attribute` field, conditions can be

Send feedback

attributes the user does not have access to. If a user does not have access to an attribute, they will fail the condition.

 Avoid using **NOT** conditions with group, marking, or organization memberships. Using a **NOT** condition in these circumstances is a misconfiguration. The platform supports scoped tokens, which carry only a subset of a user's permissions. These tokens may lack the attribute the **NOT** condition checks against, causing the condition to pass and grant more access than intended.

Parameter

Submission criteria can also use parameters defined in the parameter section. Parameters are passed into an action type from other apps or users themselves. Using conditions on parameters allows builders to embed business logic into the action type and prevent users from submitting actions on data that do not conform with business requirements.

Example

In our example, the operating status is given via the **Aircraft** object and can change with every aircraft. The condition needs to be built on top of the **Aircraft** object type parameter.

Submission criteria do not support attachment and object set parameters. These parameter types are removed from the selection panel.

Select a value

After selecting the condition template, choose what value to compare. Some parameters (like lists or object parameters) require a more granular selection of what value should be used in the comparison. We can also choose to compare the length of a list instead of its

c [API Reference ↗](#)

[Send feedback](#)

Based on parameter

Support Ticket > Request Type

is

Customer Request

Select a value

ParameterUser property

Support Ticket

Only showing compatible options

Assignee

Description

Export Format

Last Updated By

Request ID

Request Type

Requestor

Status

i Example

In the aircraft example, the operating status of an aircraft is stored in the property on the `Aircraft` object.

Operators

Operators define the comparison between the two values. To simplify the configuration workflow, operators are pre-filtered to only show a selection of operators valid for the parameter. When a parameter is changed, all conditions using this parameter need to be reconfigured.

[API Reference ↗](#)

[Send feedback](#)

<

Based on parameter

×

Support Ticket > Request Type

is

Customer Request

Select an operator

Only showing compatible operators

is

✓

is not

matches regex

There are multiple operators available depending on the selected parameters. For single value parameters, the following operators are available:

Operator	Example	Data Example	Description
is	name <i>is</i> John Doe	"John Doe" is "John Doe" = TRUE	The left value exactly matches the right value.
is not	Current User <i>is not</i> John Doe	"John Doe" is not "Maria Smith" = TRUE	The left value and right value do not match.
matches	name <i>matches</i> ^[A	E	I
is less than	Aircraft > Engine Count <i>is less than</i> 2	4 is less than 2 = TRUE	The left value is smaller than the right value.
is greater than or equals	Aircraft > Engine Count <i>is greater than or equals</i> 2	4 is greater than or equals 2 = TRUE	The left value is greater than the right value.

For parameters with multiple values, the following operators are available. Object reference list:

API Reference ↗

 list of values (either the object value or the value of the defined property):

Send feedback

Operator	Example	Data Example	Description
includes	Aircrafts > Pilot Name <i>includes</i> "John Doe"	["John Doe", "Maria Smith"] includes "John Doe" = TRUE	At least one of the left values exactly matches the right value.
includes any	List of names <i>includes any</i> Aircrafts > Pilot Name	["King Louis", "John Doe"] is included in ["John Doe", "Maria Smith"] = TRUE	At least one of the left values exactly matches at least one of the right values.
is included in	name <i>is included in</i> ["John Doe", "Maria Smith"]	"John Doe" is included in ["John Doe", "Maria Smith"] = TRUE	The left value exactly matches at least one of the right values.
each is	Aircrafts > Pilot Name <i>each is</i> "John Doe"	["John Doe", "Maria Smith"] each is "John Doe" = FALSE	All left values exactly match the right value.
each is not	Aircrafts > Pilot Name <i>each is not</i> "John Doe"	["John Doe", "Maria Smith"] each is not "King Louis" = TRUE	All left values do not exactly match the right value.

Example

Since a user in our example is a member of many groups but the comparison is to a single group, we need to select the `includes` operator to check for an overlap. However, the operating status needs to exactly match an expected status, so the `is` operator must be set.

Value

The value represents the other side of the comparison. The value can either be based on an existing parameter, a static value, or no value. No value checks whether the first value is empty (or null). Like the operator, the available options depend on the first value type.

[API Reference ↗](#)

[Send feedback](#)

< Based on parameter X

Support Ticket > Request Type is Customer Request

Select a value

Parameter Specific value No value

Customer Request

i Example

We can now finalize the two conditions needed in the aircraft example. For flight controllers, the correct group needs to be selected as a static parameter. This is because the group should not change, but should stay the same every time an Action is submitted, regardless of the context. Hence the **specific value** is used and the desired group is selected via the dropdown. An aircraft is considered operational when the operating status property is **Yes**, which can be set using the specific value option again.

Logical operators

A logical operator can be used to combine different conditions. Logical operators can also be nested to create even more complex logic and can require either all, any, or no conditions underneath it to be met to pass.

Failure message

Failure messages support defining what error should be displayed whenever the Action cannot be submitted. Every condition and logical operator on the root level has its own failure message. If conditions of lower levels are not met, the failure message of the condition or logical operator level (parent) is displayed. The failure message will be displayed to the end user across Foundry (Object Explorer, Workshop, or Quiver) when the action is not submitted.

[API Reference ↗](#)

[Send feedback](#)

met. The failure message informs the user about why they are blocked from submitting an Action.

PREVIOUS
← Parameters / Performance
considerations

NEXT
Actions on interfaces →

© 2026 Palantir Technologies Inc. All rights reserved.

[Cookies Statement ↗](#)

[Privacy Statement ↗](#)

[Terms of Use ↗](#)

— [Cookies Settings](#)

[API Reference ↗](#)

[Send feedback](#)

Ontology building / Action types / Actions on interfaces

Actions on interfaces

You can create generic actions that apply to all objects of a chosen interface. There are two main ways you can use interfaces from within actions:

- **Interface action rules:** To create, modify, delete, and link objects of the configured interface.
- **Interface reference parameters:** To reference objects of the configured interface. This parameter is required by the "Modify" and "Delete" interface action rules, but can also be used by any other action rules.

Interfaces can also define [interface action type constraints](#), which describe expected action capabilities that implementing object types can satisfy with concrete action types.

Action type constraints define an interface-level action contract; action rules define the edits performed when an action type is submitted.

Interface action type constraints are currently an Ontology Manager modeling and mapping feature, not an end-user application or Ontology SDK invocation surface.

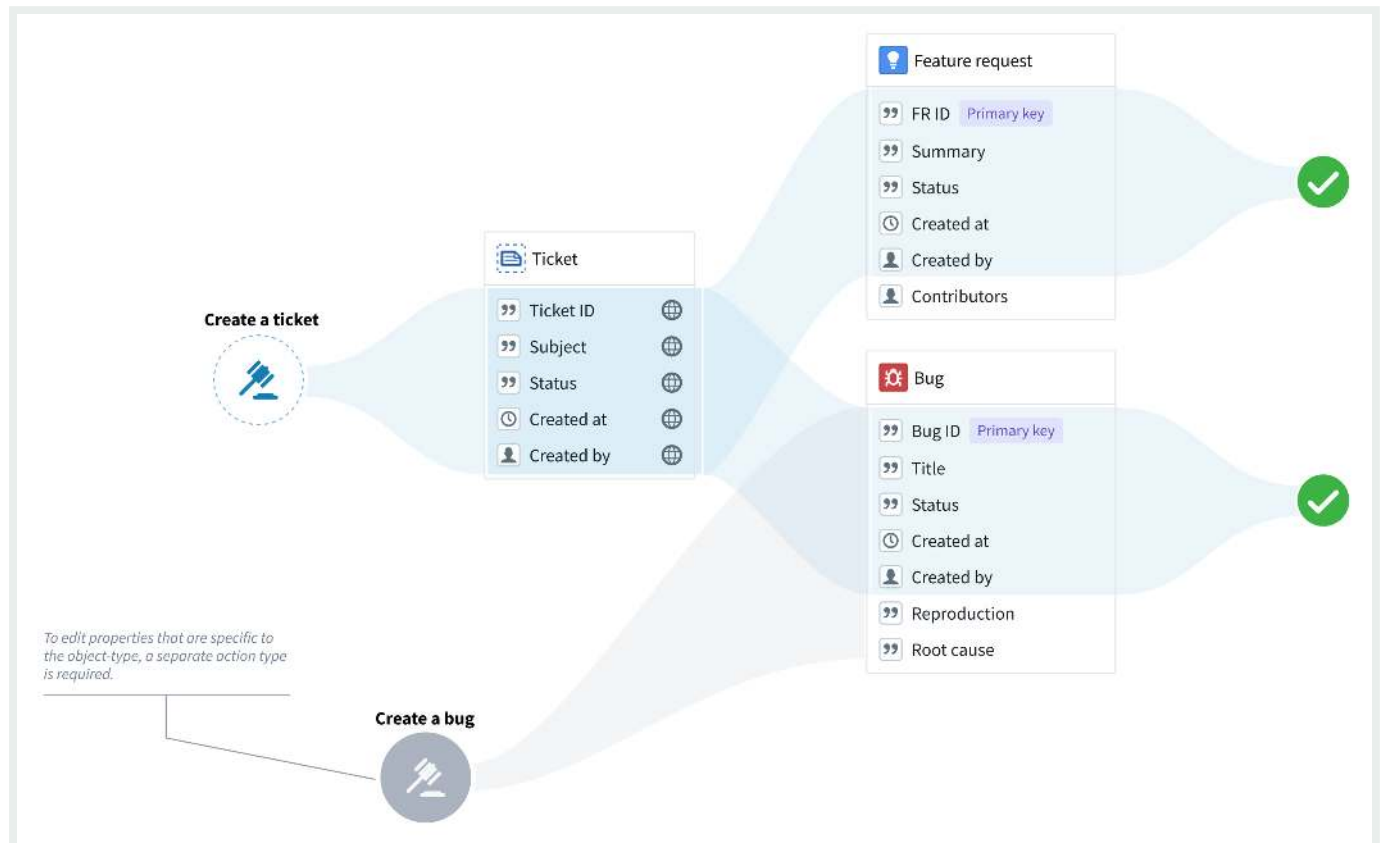
⚠ Interface action submission criteria apply uniformly to all object types that implement the interface. Before you create an interface action, carefully review which users will have permission to create, modify, or delete objects across all object types that implement the interface. [Learn more about establishing control over interface actions.](#)

API Reference ↗

Using action on interface rules

Send feedback

You can use interface action rules whenever the edits can apply to all the object types that implement the interface. In other words, you can use interface action rules only to modify the *interface shared properties* or to delete objects. For example, if “Feature request” and “Bug” are object types of the “Ticket” interface, you can use a “Create a ticket” action type to create bugs and feature requests, but you cannot create any property types that are specific to bugs or feature requests.



Creating a new interface action type

To set up a new interface action type, choose **Action type** from the **New** menu in Ontology Manager.

1. Under **Interfaces**, pick the desired interface and rule type.

[API Reference ↗](#)

[Send feedback](#)

Create a new action type

1 Action type

2 Mapping

3 Metadata

4 Submission criteria

5 Permissions source

6 Roles

STEP 1

Select an action type you want to configure

Enable users to make changes to the ontology by configuring actions they can execute.

Object Link Function Webhook Interface

Interface

Select an interface

Interface actions

☒ Create objects of interface

Configure an action type to create an object instance of the selected interface.

☐ Modify objects of interface

Configure an action type to edit an object instance of the selected interface.

☐ Delete objects of interface

Configure an action type to delete an object instance of the selected interface.

Skip

Next

2. Add the shared properties that you want to include in the action (if applicable).
3. Add metadata to describe your action type. Remember that this metadata should apply to all the object types that implement the interface.
4. Under **Submission criteria**, choose the users that can execute the action (you can apply more complex criteria later on). Remember that these permissions will apply to all object types that implement the interface, as long as the user has permissions to edit them.
5. Select **Create** to finalize the action type.

“Create” actions on interfaces

Because the action type is only associated with an interface, an “Object type” parameter will be automatically generated to indicate the object type that should be created. If using a form or a table, the user will be prompted to pick an object type from a list.

API Reference ↗

Send feedback

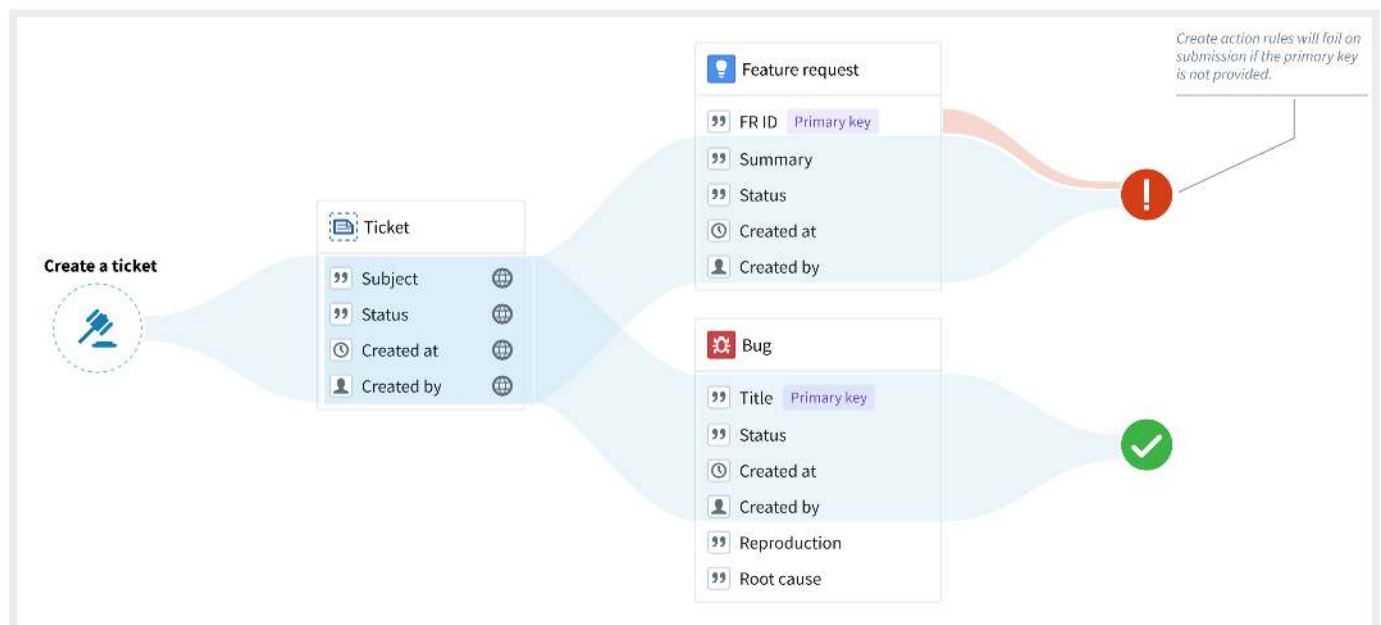
Ticket type

Select an option ▼

 Feature request

 Bug

Note that **objects cannot be created without a primary key**. Therefore, any object type without a primary key assigned in the rule will fail during submission. To avoid failures of this type, make sure that both the interface and the Create rule include an interface property that can be used as the primary key in the object types that implement the interface.



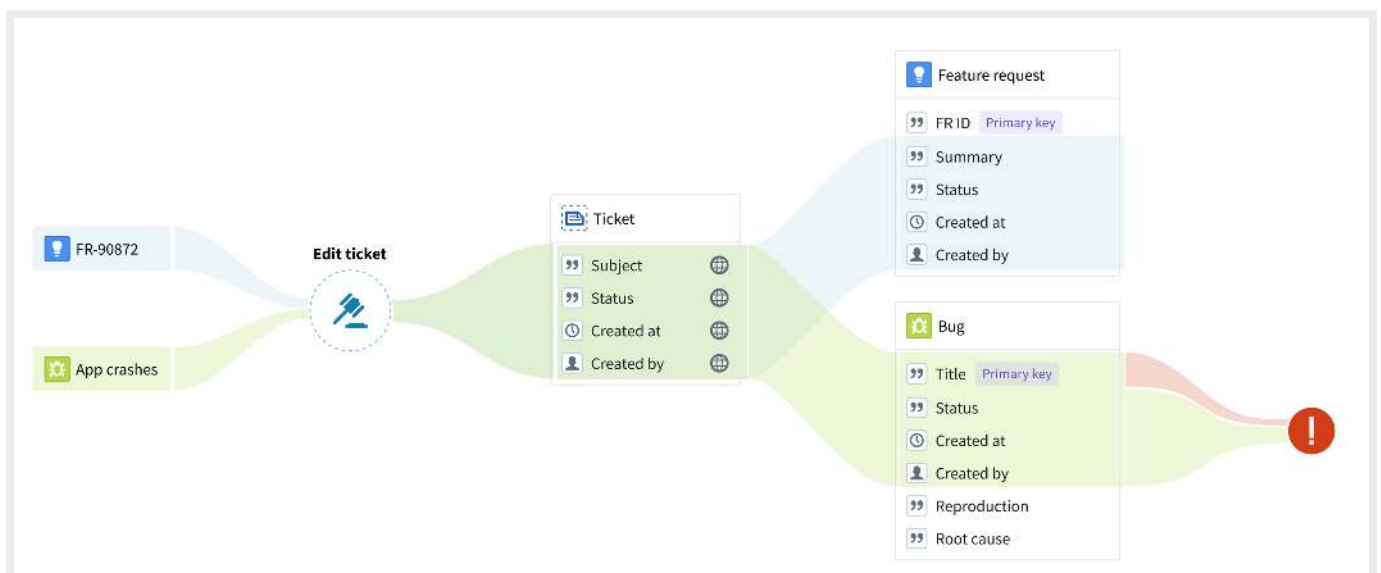
“ API Reference ↗ ons on interfaces

Send feedback

"Modify" rules on an interface can modify any object of the configured interface. An "interface reference" parameter will be generated, constrained to the selected interface. The "interface reference" parameter is similar to the "object reference" parameter, with the exception that the "interface reference" parameter shows objects of any type that implements the interface. If using a form or a table, the user could then pick an object from a list.

Note that primary key values *cannot be modified* by any action type. Therefore, an action will fail on submission if the action tries to modify a primary key property for a selected object type. Always ensure that the action rule does not modify properties that are likely to be used as a primary key by some of the object types that implement the interface.

In the example below, the "Title" property is incorrectly used as the primary key for the "Bug" object type. The "Edit ticket" action will fail on submission because the action attempts to change the primary key of the bug.



"Delete" actions on interfaces

"Delete" action rules can have an "interface reference" parameter assigned to them, instead of an object reference parameter. This interface reference, constrained to a specific interface, will indicate the object to be deleted. If using a form or a table, the user could then pick an object from a list.

[API Reference ↗](#)

"Create link" actions on interfaces

[Send feedback](#)

"Create interface link" rules allow you to create links using an interface link constraint defined on an interface. To configure a "Create interface link" rule:

1. Select the interface you want to create links on.
2. Select the interface link constraint defined on the interface. If the link constraint is between two interfaces, both the source and destination parameters will be automatically generated as interface reference parameters. If the link constraint is between an interface and an object type, the source will be an interface reference parameter and the destination will be an object reference parameter.

Optionally, you can also configure the source and destination objects manually if you do not want to use the parameters autogenerated by actions. These can be:

- An interface reference or object reference parameter referencing an existing object.
- An object created by a "Create object" or "Create object(s) of interface" rule within the same action type.

 If there are multiple concrete link implementations on the object type for the link constraint, the action will fail. Additionally, creating a one-to-many link modifies the foreign key on the many side of the relationship. Ensure there are no conflicts if your action type also modifies the foreign key using a "Create object" or "Modify object(s)" rule.

"Delete link" actions on interfaces

"Delete interface link" rules allow you to delete links using an interface link constraint defined on an interface. To configure a "Delete interface link" rule:

1. Select the interface you want to delete links on.
2. Select the interface link constraint defined on the interface. If the link constraint is between two interfaces, both the source and destination parameters will be automatically generated as interface reference parameters. If the link constraint is between an interface and an object type, the source will be an interface reference parameter and the destination will be an object reference parameter.

API Reference ↗

Send feedback

Optionally, you can also manually configure the source and destination objects instead of using the autogenerated parameters. These must be parameters referencing existing objects — either interface reference or object reference parameters.

 If there are multiple concrete link implementations on the object type for the link constraint, the action will attempt to delete all the concrete link implementations.

Executing actions on interfaces

Actions created with interface action rules can be applied to objects whose object type implements the interface, just like any object-specific action type. For a given object, all object-type-specific and interface-based actions that can be applied to that object will appear in the action dropdown.

Permissions

Interface action rules follow the same permissions as object action types.

See the documentation on [action type permissions](#) for more details.

Level of support

As support for interface action rules and reference parameters expands, availability will vary across the Palantir platform.

Supported applications and services

- **Ontology Manager:** Creation of interface action types and configuration of interface parameters in submission criteria and overrides.

 **API Reference** and **Object Views:** Rendering of actions defined on interfaces.

[Send feedback](#)

Limitations of interface action rules

- Submission criteria apply uniformly across all object types that implement the interface, so you cannot configure different permissions per object type within a single interface action. To restrict access, disable interface actions for specific object types in [Ontology Manager](#) by selecting its **Interfaces** tab and establishing control over actions inherited from an interface in the **Interface action control** section. The ability to apply more granular permission controls to interface actions is under active development.
- Action logs are not yet supported.
- Actions on interfaces cannot be used with functions.

PREVIOUS

← [Submission criteria](#)

NEXT

[Actions on structs](#) →

© 2026 Palantir Technologies Inc. All rights reserved.

[Cookies Statement](#) ↗

[Privacy Statement](#) ↗

[Terms of Use](#) ↗

— [Cookies Settings](#)

[API Reference](#) ↗

[Send feedback](#)

Ontology building / Action types / Actions on structs

Actions on structs

Struct property values can be created and modified with actions, through values supplied in a struct parameter.

Struct parameters

A struct parameter is a parameter of base type `STRUCT`, where the type contains nested parameter fields that have their own individual names and base types. A struct parameter can be only be used to supply values for a struct property. The supported base types for struct parameter fields are `BOOLEAN`, `DATE`, `DOUBLE`, `GEOPOINT`, `INTEGER`, `LONG`, `STRING`, and `TIMESTAMP`.

Below, we have a `Resolution` struct parameter for a `Create Ticket` action. The nested fields of `summary`, `resolutionTime`, and `owner` compile information on how the ticket was resolved into a single parameter.

API Reference ↗



Send feedback

Resolution

No description

Value

Display

Overrides

Details

Type

Struct

General

Visible

Add override

Disabled

Add override

Required

Add override

Constraints

Allow multiple values

Users will be able to select more than one struct

Nested fields

3

summary

resolutionTime

owner

Create new field

Defining actions on struct properties

Using actions with struct parameters, you can create and modify object types with struct properties. The values for the struct property are submitted in a struct parameter mapped to the property; each individual field of the struct property is mapped to a specific field of a struct parameter. In the example below, each field of the `Resolution` struct parameter is mapped to its corresponding field in the `Resolution` struct property of the `Ticket` object type.

[API Reference ↗](#)[Send feedback](#)

Create object

Ticket

Ticket

Experimental

PROPERTY	MAP TO
Id	Unique Identifier 1
Created Time	Created Time
Reporter	Reporter
Resolution	Resolution
Summary	summary
Resolution Time	resolutionTime
Owner	owner

+

Add property

A mapping between a struct property and a struct parameter must be complete, with each field of the struct property mapped to a field in the struct parameter. The base type of the struct parameter field *must* match the base type of a mapped struct property field. If any breaking changes are to be made to the struct property type (for example, if a new field is added, a field is deleted, or a field's base type is changed), then the related action types must also be modified to incorporate those changes.

Struct parameters in an action form

A struct parameter can be populated through action forms similarly to any other parameter type. However, struct parameter fields are rendered as a group in the form instead of individually.

API Reference ↗

Send feedback

Create Ticket

Created Time *

May 30, 2025, 3:00 PM

Edited

EDT

Reporter *

External User

Edited

▼ Resolution *

summary *

Identified and fixed bug in database query

resolutionTime *

May 30, 2025, 5:00 PM

Edited

EDT

owner *

John Doe

Reset

Submit

Default values for struct parameter fields

Default values are defined individually for struct parameter fields. Each struct parameter field is mapped to fields of a specified object type's struct property. A default value must be defined for all fields in the struct parameter and must be mapped to fields of the same object type struct property. Only struct property fields can act as default values for struct parameter fields. The object type whose struct property fields will act as default values is specified in the `ObjectReference` parameter.

[API Reference ↗](#)[Send feedback](#)

Nested fields3

summary

resolutionTime

owner

Create new field

Default value

Set a default value for the input. It can be overridden by users.

Ticket Reference > Resolution

Struct fields

summary

resolutionTime

owner

Set value as

Summary

Resolution Time

Owner

An instance of an object of the type specified in the `ObjectReference` parameter is supplied when submitting the action, and that object's struct property field values will automatically fill in the values of the corresponding struct parameter fields.

[API Reference ↗](#)

[Send feedback](#)

Create Ticket

Ticket Reference*

4561e7cc-a7f4-4967-89d6-1a3a5fa994ab

Edited

Created Time*

May 29, 2025, 1:00 PM

EDT

Edited

Reporter*

Joe Smith

Edited

▼ Resolution*

summary*

Identified and fixed bug in database query

resolutionTime*

May 30, 2025, 5:00 PM

EDT

owner*

John Doe

Reset

Submit

Constraints on struct parameter fields

Constraints can be configured individually for struct parameter fields, as with regular parameters. For example, a string length constraint can be defined on struct parameter fields of string types to only allow string value that are between 10 and 500 characters long. This would mean that the `summary` field of a the `Resolution` struct parameter must be at least 10 characters long, but no longer than 500 characters.

[API Reference ↗](#)[Send feedback](#)

summary

Type

String

Value

General

Visible

Disabled

Required

Constraints

 User input 

 Multiple choice

☒ With minimum length

10

☒ With maximum length

500

☐ Matching regex

A struct parameter value is *only* valid if *all* fields meet the defined constraint. Users can only submit a struct parameter value if each field value satisfies the constraints defined on them. As defined for the `summary` field, a value shorter than 10 characters would be invalid.

[API Reference ↗](#)[Send feedback](#)

Create Ticket

Ticket Reference

Select an option...

Created Time *

May 30, 2025, 3:00 PM

EDT

Edited

Reporter *

External User

Edited

Resolution *

summary *

fixed bug

Must have at least 10 characters.

resolutionTime *

May 30, 2025, 5:00 PM

EDT

owner *

John Doe

1 issue

Reset

Submit

Limitations

Consider the following limitations when creating or modifying struct parameters with actions:

- Struct property values can only be created or modified through *struct parameters*. Other forms of entry, such as static values or references to object properties, are not supported.
- A struct property can only be created or modified through a *single* struct parameter. A struct property mapping in actions cannot more than one parameter.
- A struct parameter can only be used to create or modify *struct* properties. Struct parameter fields cannot be used individually to create or modify non-struct properties.
- Only references to a *single object type struct property values* can act as default values for struct parameter fields. Other forms of entry, such as static values, are not supported.

API Reference ↗

PREVIOUS

← Actions on interfaces

Send feedback

Function-backed actions / Overview →

© 2026 Palantir Technologies Inc. All rights reserved.

[Cookies Statement ↗](#)

[Privacy Statement ↗](#)

[Terms of Use ↗](#)

— [Cookies Settings](#)

[API Reference ↗](#)

[Send feedback](#)

Function-backed actions

In an action type, [rules](#) define the ways objects should change when the action is applied. Many action types can be defined using simple rules which allow you to create, modify, and delete objects, or create and delete links between objects.

In some cases, however, simple rules are not sufficient to describe the changes that you want to make. For example, you may want to:

- Modify multiple objects that are currently linked together. For example, you may want to set the `status` field of an `Incident` object to `Closed`, and also set the `status` of all linked `Alert` objects to `Resolved`.
- Modify an object's properties based on some more complex logic. For example, you may want to compute a value based on some business logic that reads data from several objects, then write that value into an object property.
- Create several different types of objects and set up links between them.

To support use cases like these, action types can be configured to call a [function](#) that defines the logic of how objects should be modified. These action types are often referred to as **function-backed actions**. By using a function, you can create action types of any level of complexity, reading any number of objects and modifying objects as you see fit.

Although function-backed action types are very flexible, you should note that they are subject to both [action type limits](#) and [function execution limits](#).

Get started with function-backed actions by following the [tutorial](#).

API Reference ↗



NEXT

Send feedback →

© 2026 Palantir Technologies Inc. All rights reserved.

[Cookies Statement ↗](#)

[Privacy Statement ↗](#)

[Terms of Use ↗](#)

— [Cookies Settings](#)

[API Reference ↗](#)

[Send feedback](#)

Ontology building / Action types / Function-backed actions / Getting started

Getting started

This tutorial explains how to create an action type that is backed by an [Ontology Edit function](#).

Prerequisites

In this tutorial, we will use the same `Demo Ticket` object type and sample objects as in the [Getting Started with Actions tutorial](#).

Start by writing an Ontology edit function that performs the desired edits for your action. This requires:

- Setting up a repository using the functions on objects TypeScript template,
- Importing the relevant object types into your repository, and
- Publishing the Ontology edit function for actions to read.

Information on these steps can be found in the functions documentation:

- **[Getting started](#)**: Follow this tutorial to create a basic functions repository and publish a function.
- **[Functions on objects](#)**: Follow this tutorial to create a function that uses object data.
- **[Ontology edits](#)**: Use this reference to create an Ontology edit function.

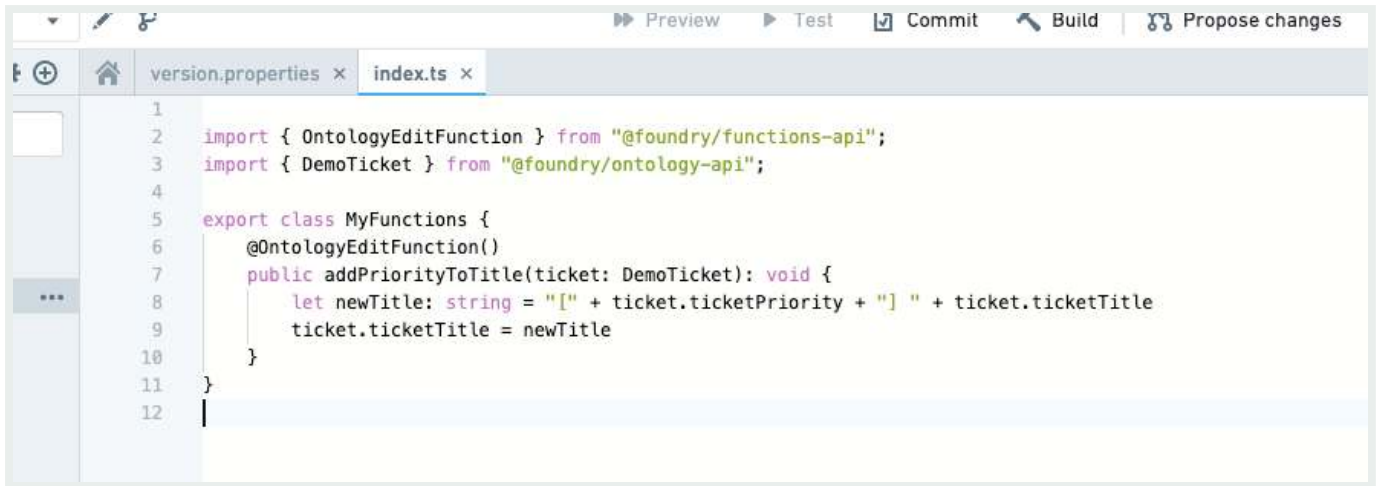
Once you have written and published an Ontology edit function, the steps below will connect the function to an action so that the function can be used to make edits to objects.

For this tutorial, we have written and published the following Ontology edit function from a repository:

API Reference ↗



Send feedback



```
1
2 import { OntologyEditFunction } from "@foundry/functions-api";
3 import { DemoTicket } from "@foundry/ontology-api";
4
5 export class MyFunctions {
6   @OntologyEditFunction()
7   public addPriorityToTitle(ticket: DemoTicket): void {
8     let newTitle: string = "[" + ticket.ticketPriority + "]" + ticket.ticketTitle
9     ticket.ticketTitle = newTitle
10  }
11 }
12
```

For convenience, the code is available here:



```
1 @OntologyEditFunction()
2 public addPriorityToTitle(ticket: DemoTicket): void {
3   let newTitle: string = "[" + ticket.ticketPriority + "]" + ticket.ticketTitle
4   ticket.ticketTitle = newTitle;
5 }
```



Functions for use in action types must be annotated with `@OntologyEditFunction()` instead of `@Function()`. Further details can be found in the documentation for [functions on objects](#).

Creating a function-backed action

In the **Rules** section, add a single rule of type **Function**. Search for the function you published as part of the [prerequisites](#), and pick the latest version. Configure the inputs to match up to the action parameters, as below. Note that a function rule cannot be combined with [other Ontology rules](#).

[API Reference ↗](#)

[Send feedback](#)

Run function

addPriorityToTitle

on Demo Ticket

0.1.2

☐ Auto upgrade to compatible versions

Required inputs

ticket

→

Ticket

Code Preview

[Edit in Code Repositories](#)

```
6  @OntologyEditFunction()
7  public addPriorityToTitle(ticket: DemoTicket): void {
8      let newTitle: string = "[" + ticket.ticketPriority + "] " + ticket.ticketTitle
9      ticket.ticketTitle = newTitle
10 }
```

+ Add new rule

When selecting the function, all inputs of the function will automatically be created as parameters and added to the **Parameters** tab. In the example shown in these screenshots, a `Demo Ticket` parameter of type **Object reference** has been created. The parameter can now be customized further if needed.

Form content

Ticket

Used in Rules

New

+ Add new parameter

Add section

API Reference

Send feedback

Ticket

No description

Value

Display

Overrides

Details

Type

Object reference

Demo Ticket

General

Visible

Add override

Disabled

Add override

Required

Add override

Constraints

Allow multiple values

Users will be able to select more than one object reference

Object dropdown

Default value

Set a default value for the input. It can be overridden by users.

Save your action and configure it across the platform as described in the [guidance for integration with other applications](#).

Changing function version

By default, if the function logic is changed, the action does not automatically update to match it. Instead, you must return to the **Rules** section of the action and upgrade the version of the function that the action is referencing. For example, if we published version 0.1.2 of the function, we would need to update it here:

API Reference ↗

Send feedback

Run function

addPriorityToTitle

on Demo Ticket

0.1.2

✓ 0.1.2

0.1.1

ticket

→

Ticket

↗

Code Preview

Edit in Code Repositories ↗

```
6 @OntologyEditFunction()
7 public addPriorityToTitle(ticket: DemoTicket): void {
8     let newTitle: string = "[" + ticket.ticketPriority + "] " + ticket.ticketTitle
9     ticket.ticketTitle = newTitle
10 }
```

+ Add new rule ▾

Auto upgrades

You can optionally choose to enable auto upgrades for the function that the action is referencing. If enabled, the action will depend on the function at a [version range](#) and [resolve the version](#) at runtime.

To enable auto upgrades for an action, navigate to the **Rules** section of the action and select the **Function** parameter. In the **Function** dropdown, select the minimum version of the function that you want to be run and enable the **Auto upgrade** option. This will correspond to a version range dependency that comprises all backward compatible versions, such as minor or patch upgrades, of the selected minimum version.

[API Reference ↗](#)[Send feedback](#)

addPriorityToTitle
on Demo Ticket

1.0.1

☒ Auto upgrade to compatible versions

 The behavior of this Action Type may be modified by picking up changes to its backing Function, including changes made by users who do not have edit permissions on this Action Type. This can result in breaks at runtime. [Documentation](#)

Required inputs

ticket

→

Ticket

 Auto upgrades are disabled for function versions of the form `0.y.z`. These versions are reserved for initial development where function API and behavior may change frequently and should not be considered stable. Refer to the documentation on [choosing a release version](#).

Security

If auto upgrades are enabled for a function-backed action, users who do not have [edit permissions on the action](#) can modify the action's behavior by making changes to the backing function. This is because edit permissions on the function are not tied to the permissions on the action.

Breaking changes

Auto upgrades can result in action execution failures due to [breaking changes](#) in bad function releases.

Provenance

The provenance of the action is set according to the provenance of the selected minimum function version. If a newer release of the function returns edits outside of this provenance (for example, an additional object type), action execution will fail.

[API Reference ↗](#)

 Currently, the provenance consists only of the object types that

[Send feedback](#)

at runtime.

PREVIOUS

← Overview

NEXT

Batched execution →

© 2026 Palantir Technologies Inc. All rights reserved.

[Cookies Statement ↗](#)

[Privacy Statement ↗](#)

[Terms of Use ↗](#)

— [Cookies Settings](#)

[API Reference ↗](#)

[Send feedback](#)

Batched execution

When an action is triggered in batches, such as in [Workshop inline edits](#) or in [Automate](#), the backing function is usually called once per request in sequence, and all edits are applied atomically at the end of the action call.

Alternatively, to improve performance or resolve edit conflicts, you may wish to configure a function to receive the whole batch of action calls in a single execution.

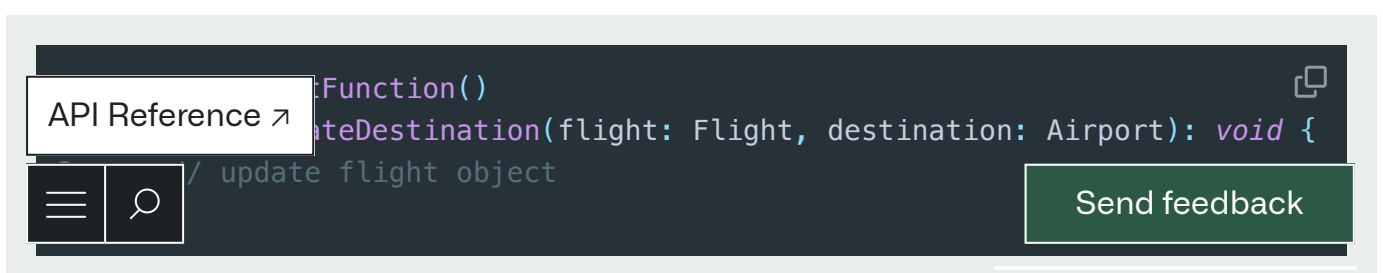
To enable batched execution, the function must receive a *single input parameter* containing a *list of structs* (also known as a "map" or "dictionary"). You will then be able to enable batched execution and pass data into the fields of this struct in the same way you would usually pass data to a function's top-level inputs.

When using batched execution:

- A single action call will invoke a single function execution with a *single entry* in the list input parameter.
- A batched action call will invoke a single function execution with *several entries* in the list input parameter.

Example

Instead of a function-backed action with the following signature:



A function can instead receive a "batch" of requests and process them all in a single execution:

```
1 @OntologyEditFunction()
2 public updateDestinationBatch(batch: {flight: Flight, destination: Airport}[]) {
3     batch.forEach(({flight, destination}) => {
4         // update flight object
5     });
6 }
```

You can then enable batched execution for this function when configuring the action type:

 Run function

updateDestinationBatch

on  Flight  Airport

0.2.12

Enable batched execution

When Action is called several times in a batch, process all requests in a single Function execution

☒

Required inputs

 flight

→

 Flight

↗

 destination

→

 Destination

↗

Code Preview

Edit in Code Repositories ↗

```
90 @Edits(Flight, Airport)
91 @OntologyEditFunction()
92 public updateDestinationBatch(batch: {flight: Flight, destination: Airport}[]): void {
93     batch.forEach(({flight, destination}) => {
94         // update flight object
95     });
96 }
```

+ Add new rule ▾

PREVIOUS

← API Reference ↗

NEXT

Side effects / Overview →

Send feedback

[Cookies Statement ↗](#)

[Privacy Statement ↗](#)

[Terms of Use ↗](#)

— [Cookies Settings](#)

[API Reference ↗](#)

[Send feedback](#)

Side effects

Action types are designed to support the full range of decision-making processes within an organization. When the Ontology serves as the system of record for a decision-making process, using [rules](#) to define object modifications allows you to express business processes with great flexibility. In order to support the full range of organizational processes, action types support a few additional features:

- For real-time processes, you may need to *notify* users about changes that are happening in the system so they can take action in response.
- In cases when a system besides Foundry is the source of truth for your organization, you may need to *integrate* with the other system to support the existing business process. This pattern is sometimes referred to as "decision orchestration."

Side effects in action types enable you to send data out of Foundry to integrate with existing organizational processes. There are two main types of side effects:

- [Notifications](#) allow you to flexibly configure how a user should be notified when an action is applied. This includes the ability to send an email to users on the platform.
- [Webhooks](#) allow you to connect to systems outside Foundry in a highly flexible way, including sending requests to a REST API or an ERP system. This enables you to write to other sources systems in your organization, or more flexibly send notifications to users by integrating with messaging systems.

You can learn more about notifications and webhooks using the links above, or review these guides to get started:

[API Reference ↗](#)[Send feedback](#)

© 2026 Palantir Technologies Inc. All rights reserved.

[Cookies Statement ↗](#)

[Privacy Statement ↗](#)

[Terms of Use ↗](#)

— [Cookies Settings](#)

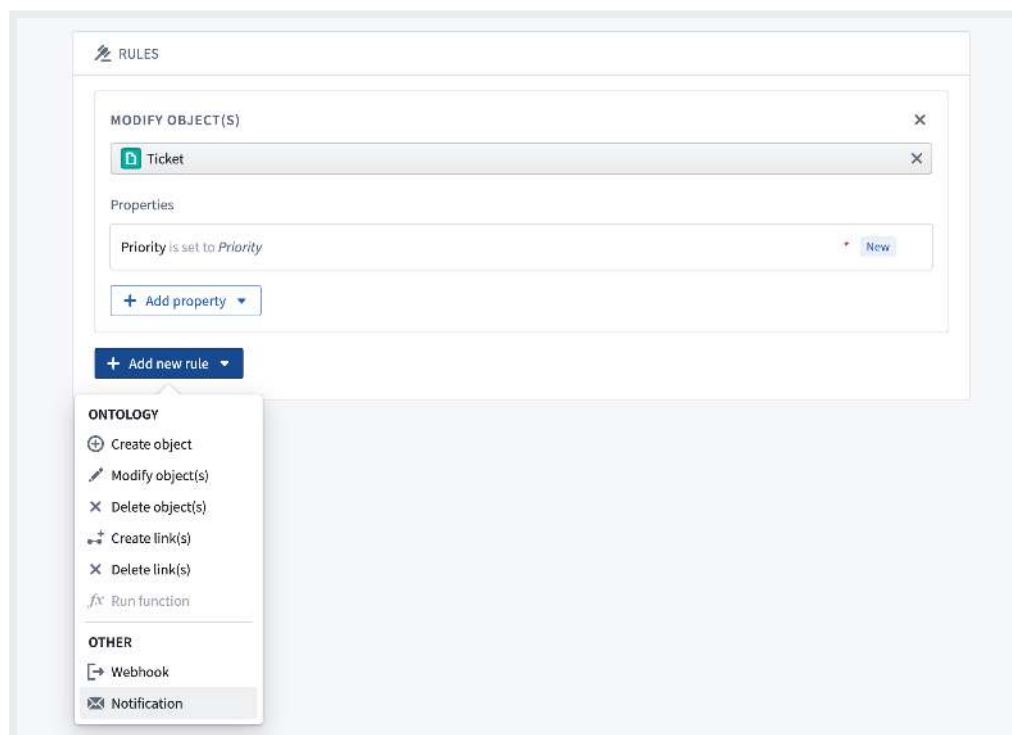
[API Reference ↗](#)

[Send feedback](#)

Ontology building / Action types / Side effects / Notifications

Notifications

Notifications can be added to an action through the **Add new rule** dropdown menu. [Learn more about how to add a notification.](#)



Configuring a notification requires specification of [recipients](#) and [content](#). The following sections provide more detail on these options.

Recipients

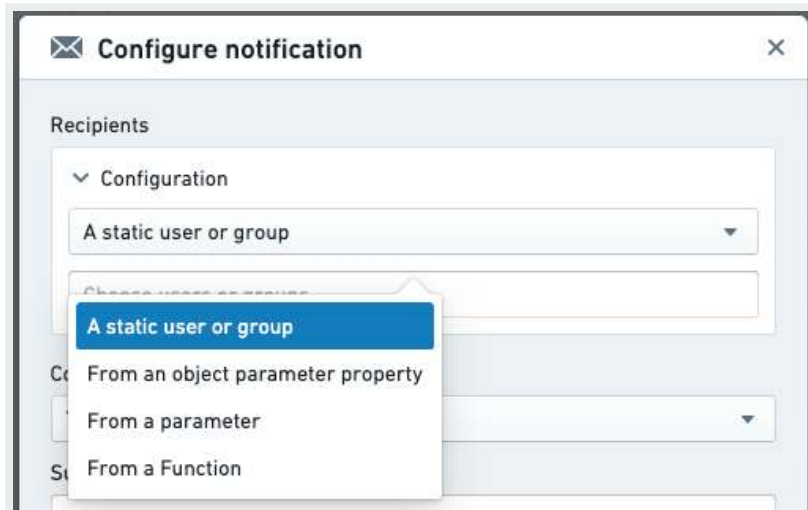
[API Reference ↗](#)

On the **Recipients** option of a notification allows you to specify the users or groups that will receive a notification when the action runs. Notification

[Send feedback](#)

recipient individually. Adding users as CC (carbon copy) recipients to email notifications is not supported.

There are several supported ways of specifying recipients:



- **Static:** In the configuration, you may select a set of users or groups who will always be notified when the action runs.
- **From a parameter:** If you have a parameter to the action that is a Foundry user or group ID, you may specify this as your recipient for a notification.
 - This can be used to allow the sender to select one or more recipients in the user interface of an application or module that uses this action, or to automatically detect and send a notification to the user running the action.
- **From an attribute of an object parameter:** If you have an object parameter to the action, and one of the properties of that object contains a Foundry user or group ID, you may specify that property of a parameter as the recipient. This is also possible for lists of Foundry user and group IDs.
- **From a function:** If your use case is not covered by the above options, you may write a custom function which takes in action parameters and will return the list of users or groups who should be notified. [Learn more about how to write a function that returns a list of users or groups.](#)

Examples of use cases for recipients based on a function include:

- Combining the other options for recipients; for example, notifying the assignee

API Reference ↗

an attribute of an object parameter and also always notifying a static set of additional recipients.

- Recipient selection based on other parameters or property values; for example, whenever there is a new task in EMEA, notify one set of users; whenever

Send feedback

there is a new task in North America, notify a different set of users.

- Any other custom logic which does not fit into the structured options.

i Recipients may change their preferences for how notifications are delivered to them. For example, one user may choose to only have notifications delivered in their web browser, while another user may choose to receive both in-platform toasts and emails. If a user has action notifications turned off in their personal preferences, they will not be notified. However, they may still view their notifications when logged into Foundry by going to "Notifications" and then "See All" in the Workspace.

Content

There are a number of options for customizing the content of notifications. Content may be configured via *Template* or provided via a custom *function*. Selecting template content will allow you to configure the full content directly in the configuration dialog. Function content will require you to have a published function, which returns the appropriate notification type.

The screenshot shows a configuration dialog with two main sections: 'Content' and 'Subject'. The 'Content' section has a dropdown menu currently set to 'Template'. The 'Subject' section has a dropdown menu with two options: 'Template' (highlighted in blue) and 'From a Function'. The 'From a Function' option is currently selected, and a text input field is visible next to it.

Content components

1. **Subject:** Usually, content will include a subject line. By default, this will be the same for all delivery mechanisms.
2. **Body:** The body of the notification. For in-platform notifications, this will display inside the notification toast. For email, this will be rendered inside the body of the email.

API Reference ↗

Specify a link. This will appear as a button just below the body content of the text of the button can be customized.

- The following options are available for configuring a link:
 - Link to an existing object parameter

Send feedback

- Link to a Workshop app
- Link to a Carbon workspace
- Link to a newly created object

4. Advanced Email Configuration: When configuring a notification, you may specify a custom content body to use when delivering a notification via email. This option allows you use HTML for more advanced formatting which is not supported for in-platform notifications. The preview will show you how your notification will look, excluding any parameter references. Recipients will only receive this content if they set their preference to receive notifications via email.

i Triple handlebars may be used to reference parameters and user attributes in the Subject, Body, and Link mentioned above. When editing a section, clicking on one of the available parameters will auto-generate the correct handlebar reference for that parameter or user attribute.

5. From a Function: When selecting "From a Function", you do not configure the sections listed above. Instead, you must provide a Function that returns a **Notification** object with the appropriate properties specifying each section of your custom content. You may need to use a Function if any of the following applies:

- The notification content is completely different depending on the recipient or input parameters to the Action.
- You want to have a different subject line for email and in-platform notifications.
- You want to use full link URLs, including links to external systems or applications that live outside of Foundry.
- You want to perform Search Arounds, aggregations, or query data beyond what is provided via parameters when rendering the content.
- You have any other custom requirements that are not possible via the template content options.

More information on the Notification return type can be found in the [Functions documentation](#).

API Reference ↗

data used for generating notification content will reflect the state of the Energy before edits of the current Action are applied. To give notification recipients access to the latest state of specific objects, it is possible to objects referenced via object parameters, or links to newly created objects (if

Send feedback

those objects are created via a "create object" rule and not via a function) in the notification.

Example configuration

This is an example configuration for a notification.

The screenshot shows a 'Configure notification' dialog box with the following sections and callouts:

- 1. Recipients:** A field labeled 'To:' with the value 'fx getUser'.
- 2. Content:** A dropdown menu currently showing 'Template'.
- 3. Subject:** A text field containing 'New notification about {{{parameter}}}'.
- 4. Parameters:** A row of three buttons: 'Parameter' (with a quote icon), 'Recipient' (with a person icon), and 'Current user' (with a person icon).
- 5. Body:** A large text area containing the text: 'Hi {{{recipient_first_name}}},
You have a new custom notification.'
- 6. Link (optional):** A section containing a 'Reset' link, a dropdown menu with 'Workshop module', another dropdown menu with 'Please select a workshop module', and a text field with 'Go to Application'.
- 7. Advanced email configuration (optional):** A collapsed section indicated by a right-pointing arrow.

At the bottom of the dialog are 'Cancel' and 'Save' buttons.

[API Reference ↗](#)

1. **Recipients** configuration
2. **Content** configuration

[Send feedback](#)

- Choose from template (configure directly in the Ontology app dialog) or Function (specify a Function that returns a fully-formed `Notification` object).

3. **Subject** line for template notification.

4. Available **parameters** based on the available parameters to the Action. Click on a parameter to generate the `{{{}}}` syntax to reference that parameter.

5. **Body** content for template notification.

6. **Link** configuration for template notification (optional).

7. **Custom HTML content for email** with template notification (optional).

Other key information

Maximum recipient limits

- There is a maximum of 50 recipients when using the "From a Function" option to render the notification content. There will be a warning in the configuration panel when selecting "From a Function" under the content configuration options, and the number of recipients will be checked each time the Action is run. If the number of recipients is over the limit, a red error toast will be displayed and the Action will fail to run.
- There is a maximum of 500 recipients for a single Action notification when the content is configured directly in the configuration dialog using the "Template" option.

The screenshot shows a configuration dialog with two main sections: 'Content' and 'Email'. In the 'Content' section, there are two options: 'Template' (with a list icon) and 'Function' (with a blue 'fx' icon and a checkmark). The 'Function' option is selected. Below these options, a text label reads 'Configure notification content using a Function which returns a Notification object.' Below this is a dropdown menu labeled 'Select a Function...'. In the 'Email' section, a 'Subject' field is partially visible with the text 'Subject: Fur'. A warning toast is displayed over the 'Function' section, stating: 'Notifications with content from a Function have a limit of 50 recipients. Any attempt to send to more recipients will result in an error.' At the bottom left, there is a link 'API Reference' with an external link icon. At the bottom right, there is a green button labeled 'Send feedback'.

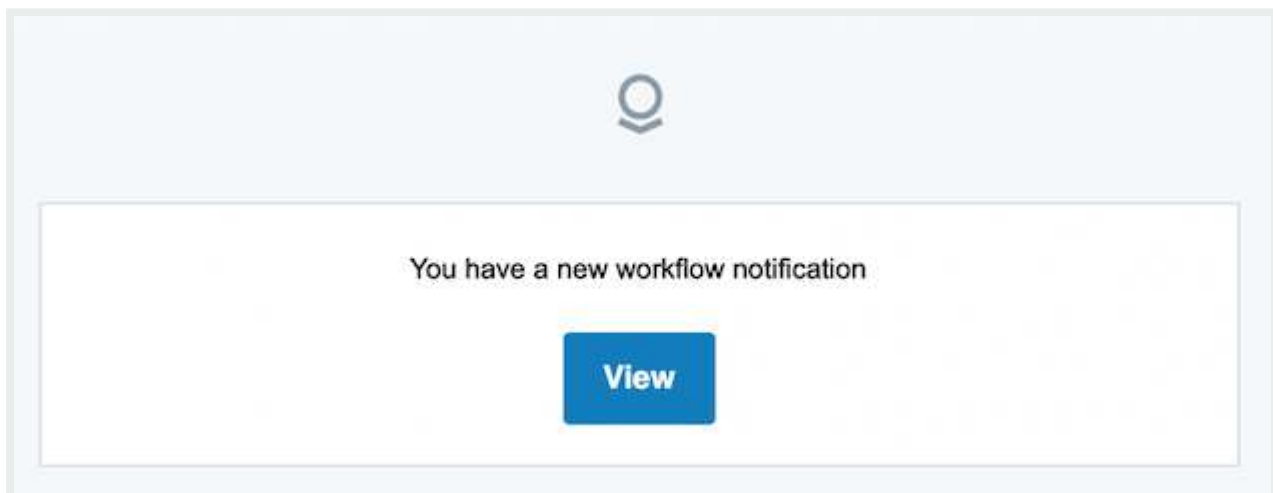
Content length limits

- The maximum subject length is 250 characters.
- The maximum body length is 1,000 characters. When rendering custom HTML content for email, the maximum length is 51,200 characters.

Keep in mind that these maximum content lengths are validated and truncated when notifications are rendered. This means that if the rendered content is dynamic (for example, if the notification content includes object data), any content longer than the allowed maximum lengths will be truncated and indicated by trailing `...`.

Strict redaction

If "Strict Redaction" or "Group Redaction" on outbound email notifications is enabled for your Foundry instance, custom notification content will not be rendered. Instead, users will receive the generic message shown below. Selecting "View" will direct them into Foundry where they can view the full notification content. [Learn more about email content redaction in Foundry.](#)



Recipient user accounts

- Groups will be resolved to individual users in order to check permissions on the data
- Foundry user and group IDs can be found via Settings under Account. The configuration interface for notifications provides selectors for users and groups

API Reference ↗

Send feedback

set of recipients. This will only display users and groups for which the person configuring the Action has adequate permissions.

- If recipient(s) are configured via reference to an object property, make sure the property stores the Foundry user or group ID as a string. You can use conditional formatting to display the associated user or group display name (for more detail, see the [value formatting documentation](#)).
- Sending directly to email addresses is not supported.

Links to newly-created objects

You must reference the primary key of a new object when linking it, since an object RID is not generated by time the notification is rendered.

Example: You have an Action that creates a new `task` object, and will be generating a unique ID when creating the task. Inside your Action notification you render a link to the newly created object using the [parameter options provided by Object Explorer](#).

- There are two supported ways of specifying URL links when using Function generated content:
 - Full link example: `https://<your-foundry-instance>.com/workspace/module/view/latest/<module-rid>`
 - Relative link example: `/module/view/latest/<module-rid>`

Required data access for recipients

- Users may only receive notifications containing data which they are allowed to view.
- In cases where there are multiple recipients, all recipients must have access to the object data rendered in the notification content.
- When configuring your Action, two methods of handling notification failures are available at the bottom of the **Security & Submission Criteria** tab in the sidebar:
 - **Require all users to have permissions (default):** If any recipients do not have the required access, an error will be shown when attempting to apply the Action. If this happens, no data will be edited and no notifications will be sent.

API Reference ↗

Require at least one user to have permissions: If at least one user can see the object, the Action will succeed. Only users with permissions will receive notifications.

Send feedback

Override and disable email content redaction

If the [organization settings allow](#), you can bypass other strict redaction settings set at the organization level for particular action types and have an action type that sends non-redacted content.

To override redaction for email notifications, navigate to the **Security & Submission** tab, then **Notification settings > Disable notification redaction**.

 Notification settings

Action failure

Choose how to handle notifications when notified users do not have permission to view an object edited by this action.

☒ Require all users to have permissions Default

If any users cannot see the object the action will fail and no users will receive notifications

☐ Require any user to have permissions

If at least one user can see the object the action will succeed. Users with permissions will receive notifications

Disable notification redaction

Everyone in the organization will be able to see unredacted notifications despite settings in Control Panel.

☐

To learn how to enable this for the organization, refer to [the email redaction documentation page](#).

Ontology building / Action types / Side effects / Set up a notification

Set up a notification

This tutorial demonstrates how to set up an action with a notification.

We will be using an action which updates the `Priority` property of an `Alert` object, and also notifies the `Assignee` (a Foundry user) which is stored as a property on that Alert object. If you want to follow along, you'll need to have the following already set up:

- An object with the correct properties and configured to be editable via actions
- An action which takes in one of your objects as well as a parameter containing the new priority and updates the priority property on the specified object. If you previously followed [the tutorial on getting started with actions](#), you should already have this set up.

If you are new to managing objects, you can read about [how to set up an object type](#).

Prerequisites

Complete the Getting Started tutorial

This tutorial assumes you already completed the [Getting Started](#) tutorial for actions.

Add the assignee property to your object type

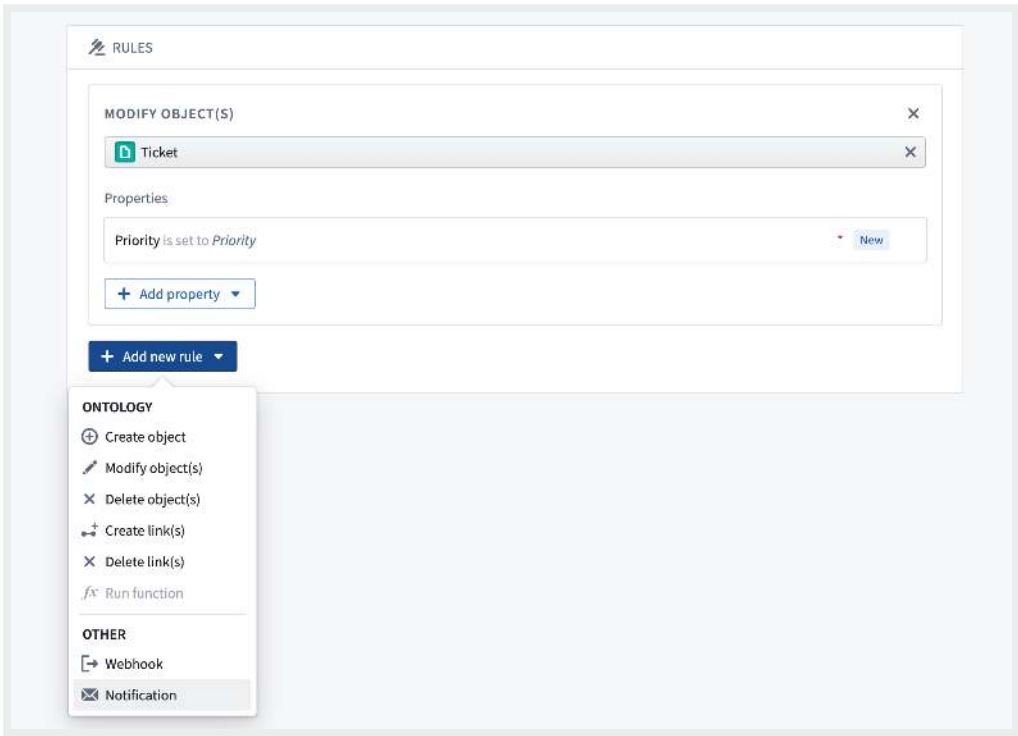
For this tutorial, you will need to have a property on the `Alert` object that is called `Case`. This property should be of type `FoundryUser` and represents the Foundry user ID for the currently assigned user. Typically, if you are using actions to construct your workflow, you will be able to capture and store user IDs with `FoundryUser` selector components in your application. These will show up in Foundry wherever they are displayed in Foundry.

API Reference ↗

Send feedback

Add a notification

First, navigate to your action that updates the ticket priority. Under the **Rules** section select **Add new rule**, followed by **Notification**. This will open the configuration dialog for adding a notification.



Configure recipients

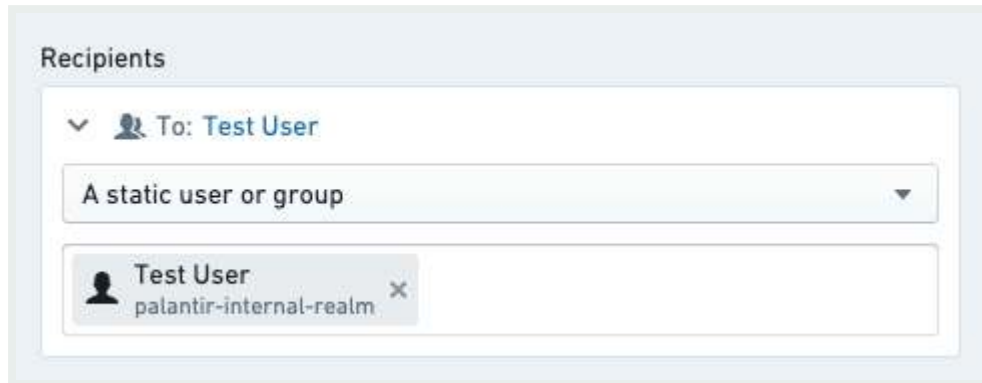
For this example, you will send the notification to the assignee, which is stored as a property of the `Alert` object being edited. To do this, use the option "Recipient(s) from property of object parameter" in the **Recipients** dropdown. Select the `Alert` object that is available as a parameter to the action, then select the `Case managers` property when prompted.

You should see the selected object parameter and property displayed in the **Recipients** section of the configuration. Keep in mind that the recipient must always be a Foundry user ID. If this property contains something else such as string email addresses, no notifications

w [API Reference](#) ↗

Send feedback

i For testing, you may initially want to configure the action with hardcoded recipient(s) that can be used to validate the logic and notification content is configured as expected.



[Learn more about other recipient configuration options.](#)

Configure notification content

Next, you will configure the content of the notification by customizing the notification to address the recipient by name and including the old and new priority of the `Alert` object in the content. An example notification configuration is available below.

First, select "Template" from the content options. This is the most straightforward way to configure the content and does not require writing any code.

For the subject line, enter your desired message. To add a parameter reference, add a forward slash `/` and select the desired parameter from the dropdown list. If your selection is an object parameter, you will be asked to select which property you want to reference.

For the body, enter text that addresses the recipient by name, identifies the user who made a change, and reports the previous and updated status.

As with the object reference in the subject, you can select the "Recipient", "Current User", and any parameter options from the dropdown list in order to generate the correct reference to API Reference ↗ es.

[Learn how to generate notification content with more complex requirements.](#)

Send feedback

Configure a link

Finally, you will add a link to the Object View of the specified **Alert** in Object Explorer. Select "Object View" and then select your ticket object parameter from the dropdown. Then, add a label for the link button that reads **View Ticket**.

Now you are ready to save your entire notification configuration:

[API Reference ↗](#)

[Send feedback](#)

Recipients

▼

👤 To:

🗉 Case managers

From a parameter
▼

🗉 Case managers

Content

📄 Template
✓

fx Function

Configure notification content using the template provided below. By default, the same content will be used for in-platform push notifications and emails.

Subject

Priority changed for alert
🚩 Alert -
🗉 Alert Id

Use ` ` to add parameters.

Body

Hi
👤 Recipient First Name
,

We wanted to let you know that
👤 Current user First Name
changed the priority of
🚩 Alert -
🗉 Alert Id
to
123
Updated ticket priority .

Have a great day,
Ticket Manager

Use ` ` to add parameters.

▼ Link (optional)
Reset

Object Parameter
▼

🚩 Alert

☐ Open in Carbon
Will use the user's default carbon workspace

View ticket

[Learn more about other types of links that can be configured.](#)

API Reference
↗

Send a test notification

Send feedback

To verify, create a test alert with yourself as the assignee. In order to run the action, you will then need to expose the action in Object Explorer or via a button in a Workshop module as described in the [actions documentation](#).

Once you've made a test change, you should receive both an in-platform push notification and an email notification to the email account specified on your Foundry user profile. Previews for both in-platform and email notifications are displayed within the notification configuration view.

If you did not receive an email, it may be because you have email and/or in-platform notifications disabled. You can verify this in **Notifications** under **User Settings**.

Next steps

- Explore other optional features, such as [custom content](#) for when the recipient chooses to receive notifications via email.
- Configure complex logic for recipients or content using [functions](#).

PREVIOUS
← [Notifications](#)

NEXT
[Webhooks](#) →

© 2026 Palantir Technologies Inc. All rights reserved.

[Cookies Statement](#) ↗

[Privacy Statement](#) ↗

[Terms of Use](#) ↗

— [Cookies Settings](#)

[API Reference](#) ↗

[Send feedback](#)

Ontology building / Action types / Side effects / Webhooks

Webhooks

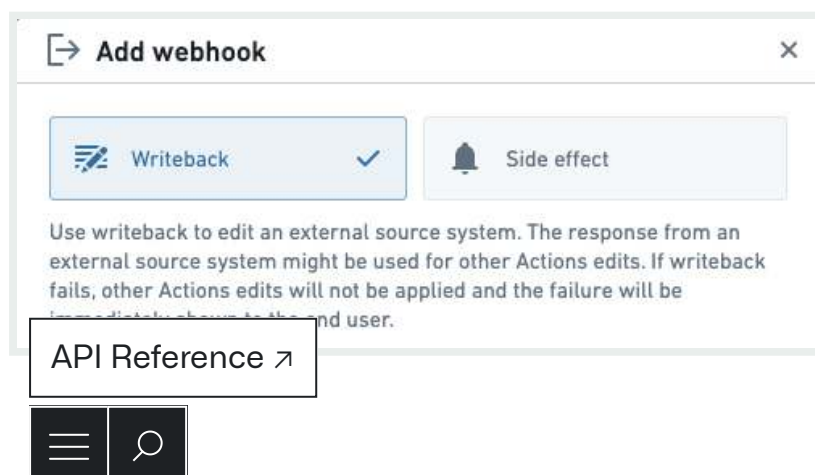
A [webhook](#) is a concept in Data Connection that enables sending a request to an external system, such as Salesforce, SAP, or any configured HTTP server, typically to modify data in that external system.

By setting up a webhook and then configuring it for use in an action, you can send data to an external system when end users apply an action in Foundry. This enables workflows in Foundry to connect directly with source systems and write back data and decisions into those systems.

This section details the various options available for configuring webhooks in an action. For a step-by-step tutorial, see the documentation on [how to add a webhook in an action](#).

Webhooks: Writeback vs. side effect

There are two ways that webhooks can be configured for use in an action: as a **writeback** or as a **side effect**.



For convenience, below is a table comparing the behavior of writeback and side effect webhooks.

Type	When applied	Failure shown to end user?	Timing
Writeback	Before object changes	Yes	Before user sees success or failure
Side effect	After object changes	No	May be after user sees success message

The following sections describe writeback webhooks and side effect webhooks in more detail.

Writeback webhooks

When configured as a **writeback**, the webhook will be executed *before* any other rules are evaluated; if the webhook execution fails, no other changes will be made. If you want to ensure that changes are not made in Foundry before the external system, you should set up your webhook as a writeback.

This behavior enables some degree of transactionality between Foundry and the external system. Using a writeback webhook guarantees that if the request to the external system fails, no changes will be applied to the Foundry Ontology. However, it is still possible that the external request may succeed but Ontology changes could fail.

Because the action stops being applied when a writeback webhook fails, you can only configure a single webhook as a writeback. If this webhook fails when the action is applied, an error will be shown to the end user describing the failure.

When a webhook is configured as a writeback, its output parameters can be used in subsequent rules. See the [output parameters](#) section below for more details.

Side effect webhooks

API Reference ↗

When configured as a **side effect**, a webhook will be executed *after* changes are evaluated. This means that modifications to Foundry objects will occur before side effects

Send feedback

are applied. You can configure multiple side effect webhooks in a single action, and they will be executed in no particular order. In an action with side effect webhooks, the end user will see a success message after Foundry objects are modified; executing the side effects may happen after the success message is shown.

If you need to call a webhook multiple times from a single action, this can be achieved with a side effect webhook by providing a list of payloads as an input. This will trigger the webhook as many times as there are payloads in the list provided and will be processed in no guaranteed order. An example of this can be found below in the [input parameters](#) section.

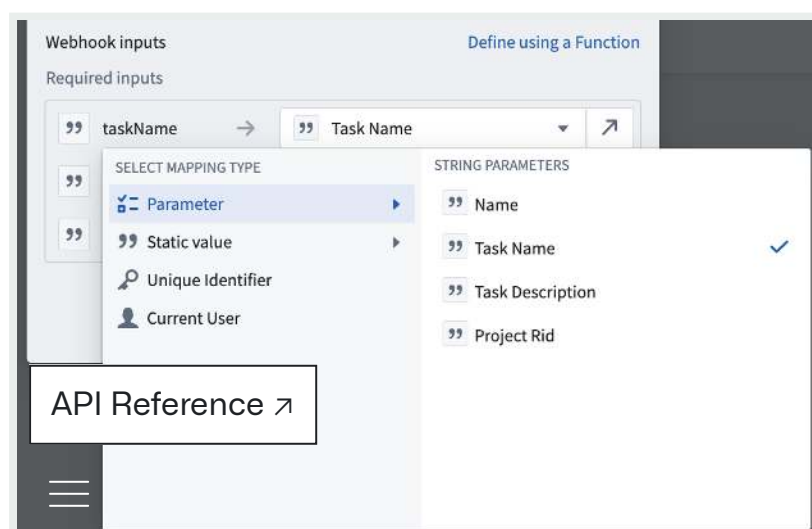
You should use side effect webhooks when you want to send best-effort notifications or write back to multiple external systems.

Input parameters

In order to configure a Webhook in an Action, you must populate all of its required input parameters. General reference material about Webhook input parameters is available in the [Data Connection documentation](#).

There are two ways to configure Webhook input parameters: by mapping to Action parameters, or by using a Function.

When mapping to **Action parameters**, each required Webhook input must be set to either an Action parameter of the same type, a static value, or a property of an object parameter.



Send feedback

When using a [Function](#), you must select a Function that returns a custom type that includes all of the required Webhook input parameters and strongly matches the Webhook type, otherwise you will receive an

`OntologyMetadata:ActionWebhookInputsDoNotHaveExpectedType` error. Using a Function to populate Webhook input parameters can be useful when you want to use logic to populate inputs, especially if this logic is based on Ontology objects. For example, you can retrieve linked objects and pull property values from those objects to prepopulate Webhook inputs.

As an example, suppose you have a Webhook which takes three input parameters with IDs `name`, `industry`, and `country`:

Input Parameters (3)

i Input parameters represent the inputs that can be passed in when a user executes the Webhook. Typically, a Webhook is configured for use in Action types, where you can specify a mapping between Action parameters and Webhook input parameters.

PARAMETER ID ? Add description... x

name

A String

☐ Only allow specific values

PARAMETER ID ? Add description... x

industry

A String

☐ Only allow specific values

PARAMETER ID ? Add description... x

country

A String

☐ Only allow specific values

+ Add parameter

You can write a Function that returns a custom interface of the same structure:

```
1 export interface MyWebhookInput {
2   name: string;
3   industry: string;
4   country: string;
5 }
```

API Reference ↗

Send feedback

Then, you can select this Function when configuring Webhook inputs in an Action, mapping Action parameters to the parameters required by the Function:

Webhook inputs Define using Action parameters

Select a Function returning expected inputs of the Webhook

returnWebhookInput
on Company

0.22.0

Required inputs

company is set to Company * New

Below is a full code example of a Function that loads data from an Ontology object and uses it to populate Webhook inputs.

```
1 import { Function, UserFacingError } from "@foundry/functions-api";
2 import { Company } from "@foundry/ontology-api";
3
4 export interface MyWebhookInput {
5     name: string;
6     industry: string;
7     country: string;
8 }
9
10 export class MyWebhookFunctions {
11     @Function()
12     public returnWebhookInput(company: Company): MyWebhookInput {
13         if (!company.name || !company.industry || !company.country) {
14             throw new UserFacingError("Some required fields are not set.");
15         }
16         return {
17             name: company.name,
18             industry: company.industry,
19             country: company.country,
20         };
21     }
22 }
```

API Reference ↗

Send feedback

A side effect Webhook may be called multiple times by returning a list of payloads from a Function. Below is an example Function which takes two companies as inputs, and returns a list containing two payloads matching the input parameters expected by a Webhook. If this Function is used from Actions to return the inputs for a side effect Webhook, it will result in two separate Webhook executions.

```
1 import { Function } from "@foundry/functions-api";
2 import { Company } from "@foundry/ontology-api";
3
4 export interface MyWebhookInput {
5     arg1: string;
6     arg2: string;
7 }
8
9 export class MyFunctions {
10     @Function()
11     public createWebhookRequest(company1: Company, company2: Company): My
12         return [
13             {
14                 arg1: company1.someProperty,
15                 arg2: company1.someOtherProperty,
16             },
17             {
18                 arg1: company2.someProperty,
19                 arg2: company2.someOtherProperty,
20             }
21         ];
22     }
23 }
```

Output parameters

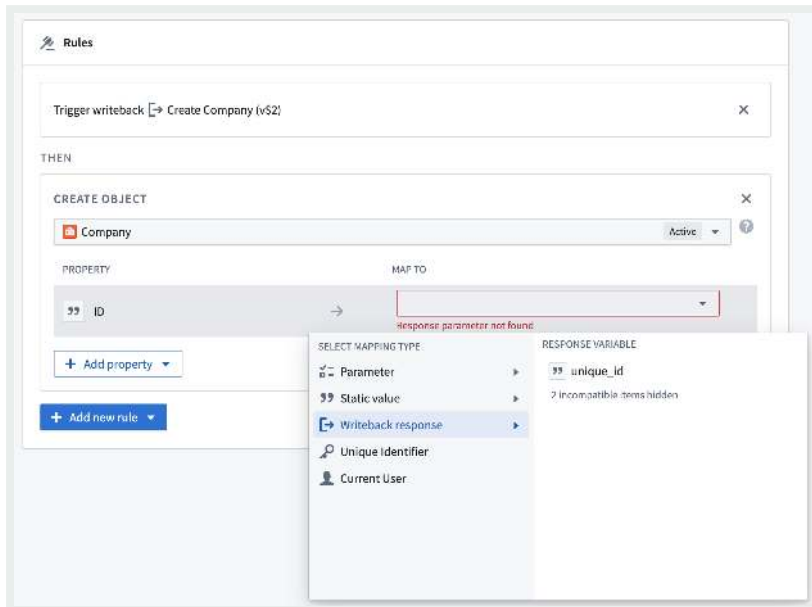
When a Webhook is configured as a [writeback Webhook](#), you can use its output parameters in subsequent rules. This is useful when the external system returns data that you want to immediately write into a Foundry object or use in a subsequent [notification](#) or [side effect Webhook](#).

API Reference ↗

General reference material about Webhook output parameters is available in the [Data Connection documentation](#).

Send feedback

To use an output parameter in a subsequent logic rule, select **Writeback response** when populating the value for a logic rule, then select the specific output you wish to use:



OAuth 2.0 authentication

When a webhook is configured on a REST API source that uses an [outbound application](#) for authentication, Foundry manages the OAuth 2.0 authorization flow on behalf of the user. Developers do not need to handle token acquisition or refresh. Foundry passes the correct access token with every webhook call.

For a full overview of OAuth 2.0 outbound application support across Foundry workflows, see the [OAuth 2.0 outbound applications](#) documentation.

PREVIOUS
← Set up a notification

NEXT
Set up a webhook →

© 2026 Palantir Technologies Inc. All rights reserved.

[API Reference](#)

[Privacy Statement](#)

[Terms of Use](#)

[Send feedback](#)

Ontology building / Action types / Side effects / Set up a webhook

Set up a webhook

Prerequisites

1. Complete the [getting started](#) tutorial to learn about how to set up a basic action.
2. Set up a [webhook](#) to create a connection to an external system. You may need to work with a system administrator who has permissions to connect to source systems before creating a webhook.

Steps

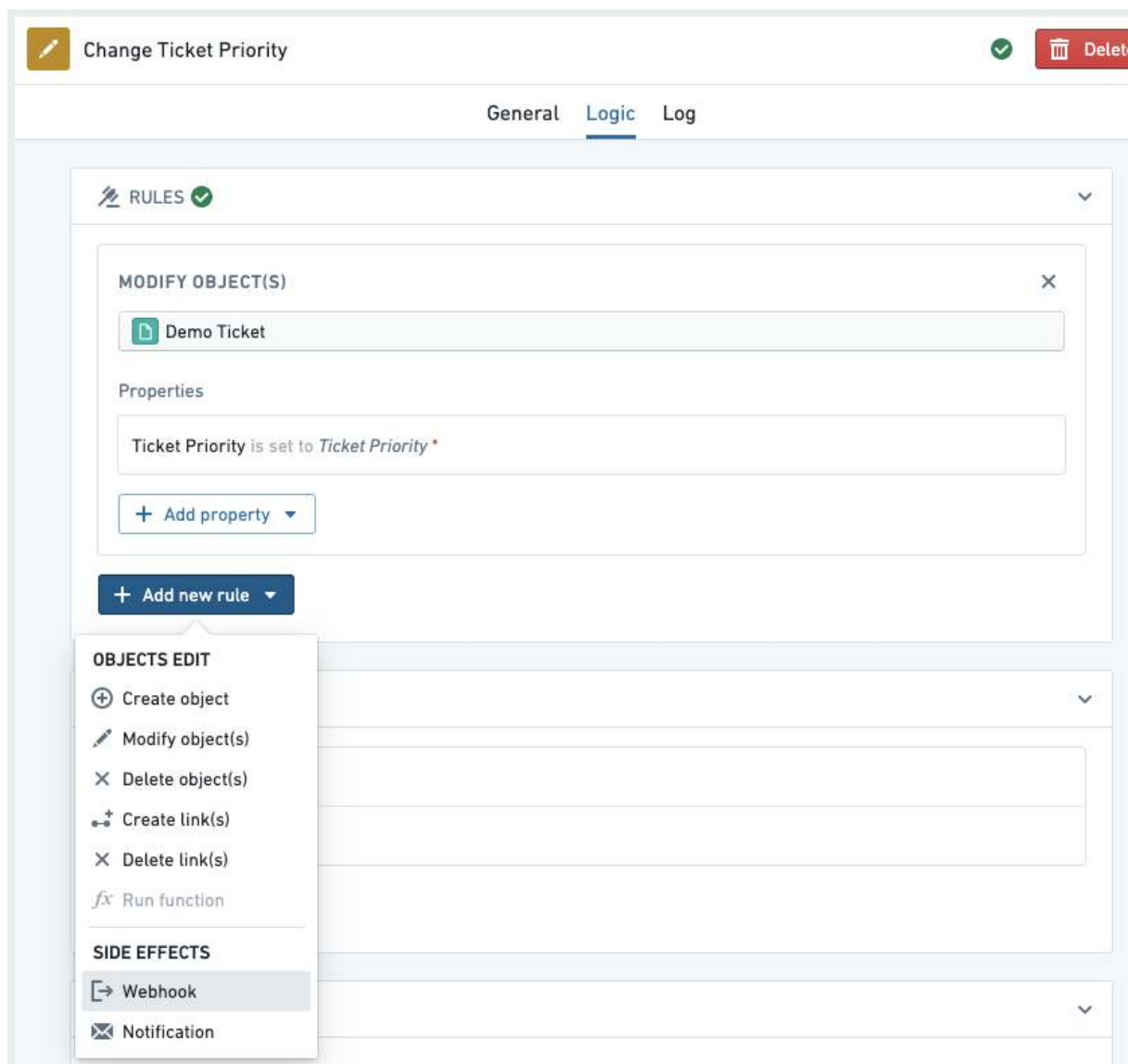
In the below example, suppose that we have a webhook called **Modify ticket priority** which accepts a `priority` string value and sends it to an external system.

Navigate to your action, select the **Logic** tab, select **Add new rule**, then select **Webhook**.

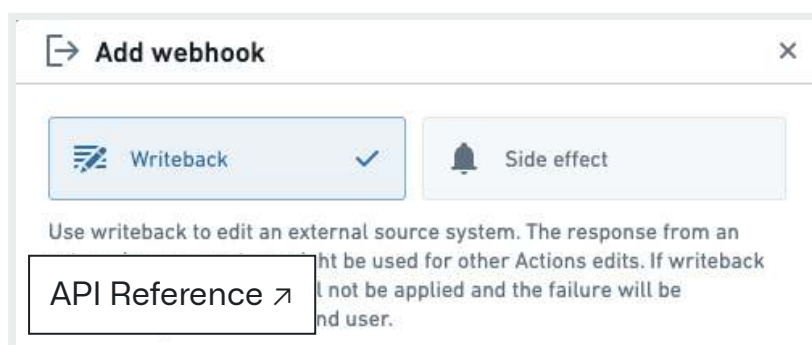
API Reference ↗



Send feedback



By default, the newly added webhook is configured as a **side effect**; this makes the webhook run *after* objects are edited in Foundry. Alternatively, select **Writeback** to have the webhook run *before* object edits.



Send feedback

In the menu below, select the webhook you want to execute. In this example, we select the **Modify ticket priority** webhook. If you wish, you can select a different version of the webhook.



Select webhook 

 Modify ticket priority 

 1 

Next, configure webhook input parameters based on action parameters. By default, a new action parameter is generated for each webhook input unless an existing action parameter with the same name already exists, in which case the existing parameter is automatically configured. In this example, we map the webhook input called `priority` to an existing action parameter called `Ticket Priority`.



Webhook inputs [Define using a Function](#)

Required inputs

priority is set to Ticket Priority *

Select **Add webhook** to complete adding the webhook rule to your action. Note that after the webhook is added, the Rules section shows that the writeback will occur before objects are modified.

[API Reference ↗](#)

[Send feedback](#)

RULES

Trigger writeback Modify ticket priority (v1)

THEN

MODIFY OBJECT(S)

Demo Ticket

Properties

Ticket Priority is set to *Ticket Priority* *

+ Add property

+ Add new rule

You may want to remove any autogenerated parameters created when you added the webhook which are now unused.

PARAMETERS

Demo Ticket

Ticket Priority

Priority New Unused

Remove

+ Add new parameter

Select **Save** in the top-right to save your changes to the action. Now, when the action is applied, the request to the external system will be made before any changes are applied to objects in Foundry.

API Reference ↗

Send feedback

This tutorial demonstrated how to add a webhook to an action. To learn more, try these resources:

- View the [webhooks](#) section to learn about all the options available for webhooks in actions.
- Learn how to configure webhook input parameters using [functions](#).
- Learn more about the [webhooks concept](#) in Data Connection.

PREVIOUS
← [Webhooks](#)

NEXT
[Trigger schedule build](#) →

© 2026 Palantir Technologies Inc. All rights reserved.

[Cookies Statement](#) ↗

[Privacy Statement](#) ↗

[Terms of Use](#) ↗

— [Cookies Settings](#)

[API Reference](#) ↗

[Send feedback](#)

Ontology building / Action types / Trigger schedule build

Trigger schedule build

A [schedule](#) defines a set of resources Foundry recomputes as part of a [build](#). By configuring a **schedule rule** on an action type, you can trigger a build of that schedule whenever the action is applied. This enables end-user workflows in the Ontology to recompute datasets without users having to navigate to [Data Lineage](#) or the [Builds application](#).

When an action type contains a schedule rule, the action's Ontology edits are applied *after* the build begins. Edits do not wait for the build to finish. Instead, the action triggers the build, captures the schedule run RID, and immediately applies the rest of the rules, including the Ontology edits.

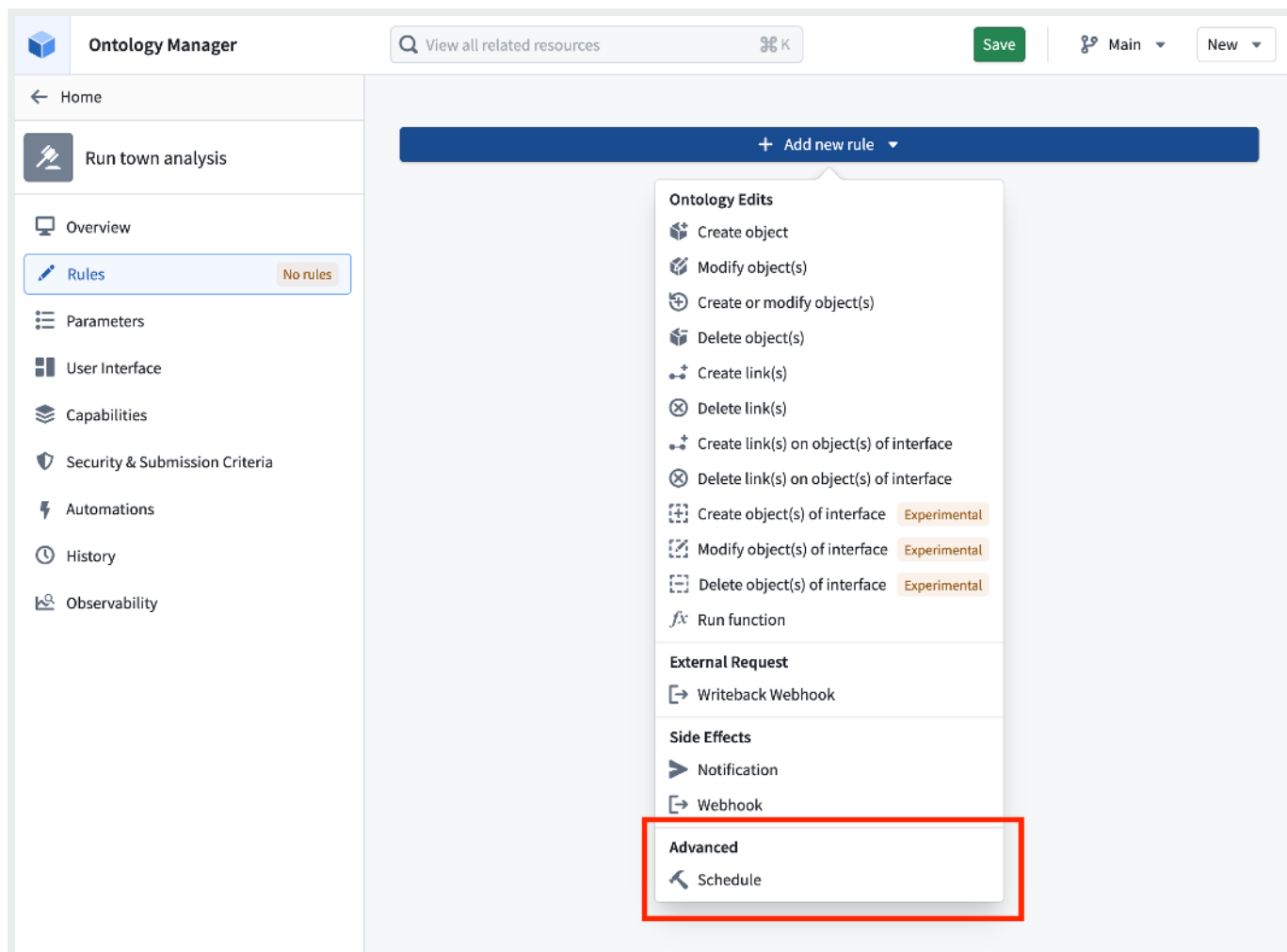
Configure a schedule rule

Add a schedule rule to an action type and select a schedule. The schedule must be in [project-scoped mode](#).

API Reference ↗



Send feedback



If the selected schedule is [parameterized](#), you must provide a value for each schedule parameter. When the action is applied, the resolved parameter values are passed to the schedule and forwarded to the underlying parameterized transforms inside the build.

Schedule rules are particularly useful when paired with [parallelized parameterized schedules](#). Review the [parameterization documentation](#) to learn more about using actions in the Ontology for parallelized schedules.

Permissions

The action's [submission criteria](#) manage the permissions needed to trigger a schedule through the action. If users satisfy the action submission criteria, they can run the schedule without any direct permissions on the schedule.

API Reference ↗



Foundry checks whether a user has permission to run the schedule if the schedule is referenced and whenever the schedule rule is edited. Referencing a schedule

Send feedback

from an action type delegates control over running it from the schedule to the action type. Anyone who can manage actions on the action type then controls who can trigger the schedule.

Track build progress

When a schedule rule is triggered, the resulting schedule run is identified by a **schedule run RID**. This RID is exposed as a value that can be referenced from the action type's Ontology edit rules, allowing you to write it into a string property of an edited object. This is useful when you want to keep a record on the object of the build that was triggered by the action.

To capture the schedule run RID, configure a **Modify object** or **Create object** rule on the same action type and map a string property of the target object to the schedule run RID value provided by the schedule rule.

The screenshot displays the 'Ontology Manager' interface. On the left, a sidebar contains navigation links: Home, Run town analysis, Overview, Rules (highlighted), Parameters, User Interface, Capabilities, Security & Submission Criteria, Automations, History, and Observability. The main area is titled 'Schedule' and shows a 'Town analysis schedule' with a 'Change' link. Below this, the 'Required inputs' section lists 'town_name' and 'risk_factor', which are mapped to 'Town Name' and 'Risk Factor' respectively. A 'THEN' section follows, showing a rule configuration for '1. Create object' for the 'Town Analysis Object'. The 'PROPERTY' column lists 'Town Name' and 'Schedule Run RID'. The 'MAP TO' column shows 'Town Name' and 'Schedule run RID'. A dropdown menu for 'Schedule run RID' is open, showing options: Parameter, Static value, Unique Identifier, Current User, and 'Schedule run RID' (selected). A tooltip message states: 'Store the schedule run's RID to track the build progress'. At the bottom left, there is a button labeled 'API Reference ↗'. At the bottom right, there is a green button labeled 'Send feedback'.

API Reference ↗

Send feedback

To render the stored RID as a live build status indicator, apply [resource RID formatting](#) to the property. With formatting enabled, Foundry displays the RID value as a link with an icon and text that reflects the current status of the build: `Running`, `Ignored`, `Failed`, or `Succeeded`.

© 2026 Palantir Technologies Inc. All rights reserved.

[Cookies Statement ↗](#)

[Privacy Statement ↗](#)

[Terms of Use ↗](#)

—[Cookies Settings](#)

Ontology building / Action types / Configure sections

Configure sections

The action form can be customized with **sections**. These sections provide a logical grouping of parameters to organize an action form. Sections also support columns, descriptions, and conditional overrides.

The screenshot displays a form titled "New Flight Route". It is organized into several sections:

- Aircraft Types ***: A search input field with the placeholder text "Search..."
- Departure Airport *** and **Arrival Airport ***: Two dropdown menus, each with the placeholder text "Select an option..."
- Start Date *** and **End Date (optional)**: Two date selection input fields, each with the placeholder text "Select a date"
- Business Targets**: A section with a right-pointing chevron icon (>)

At the bottom right of the form are two buttons: "Cancel" and "Submit".

API Reference ↗

Adding a section to an action form



Send feedback

In the Form tab, click **Add section**. This will open a detailed section configuration modal where you can add a title, choose a column layout, and optionally write a user-facing description. The description is not stylized and, unlike parameter descriptions, will always be shown in the section itself, not in a tooltip.

You can organize parameters in columns to make better use of the space within a form or to group related parameters closer together. A section can be divided into one or two columns. Separate columns are especially useful when you use parameters that don't require a lot of space within the form.

The screenshot shows a configuration modal for a section titled 'Airport Details'. At the top, there is a 'Back' button and a trash icon. The modal is divided into several sections: 'Title' with a text input containing 'Airport Details'; 'Description (Optional)' with a text area containing 'Add description...'; 'Layout' with two options: '1 Column' and '2 Columns' (which is selected with a checkmark); 'Display' with a toggle for 'Show title bar' (currently off) and a note 'Only sections with a title can be collapsed.'; 'Visibility' with two options: 'Visible' (selected with a checkmark) and 'Hidden'; 'ID' with the value 'section-1' and a 'RID' field; 'SECTION CONTENT' which lists two parameters: 'Departure Airport' and 'Arrival Airport', both marked as 'Used in Rules and Validations'. Below the list are two buttons: '+ Create new parameter' and '▼ Add existing parameter'. At the bottom, there is a button '+ Add a conditional override'.

Sections are also collapsible, can be hidden entirely, and can make use of conditional overrides, giving you more ways to customize the form behavior. All of the features will also allow for smarter forms which present required parameters under the appropriate circumstances. A section can be hidden at first and only shown based on a prior parameter value.

[API Reference ↗](#)

[Send feedback](#)

Adding parameters to a section

There are two ways to add a parameter to a section: from within the section configuration view, or from within the **Form** tab.

In the section configuration view, click **Add new parameter**. From here, configure the newly added parameter inside the section. Alternatively, click **Add existing parameter** to move an existing parameter into the section.

The **Form** tab lists the sections with their parameters in a single overview. Click the eight dots on the left hand side of the parameter and drag it into an existing section. Parameters and sections display in the form based on their order in this **Form Content** section.

Ontology building / Action types / Upload media

Upload media

Actions support uploading media files using an action form or table. For most use cases in Foundry, uploading to media reference properties is the recommended method.

Media reference properties (backed by [media sets](#)) offer several advantages over attachment properties:

- **Scalability:** Support for billions of files with efficient storage and retrieval.
- **Built-in transformations:** Many media transformations and LLM capabilities are supported and easy to use out of the box.
- **Advanced previews:** Built-in rendering and rich preview functionality for supported formats.
- **Format support:** Support for tailored workflows on both standard formats and specialized formats, such as NITF, GeoTIFF, and DICOM.

Users can upload media files via a file-picker interface, with files persisted to the media set upon successful action submission.

⚠ [Format conversions](#) only happen after the action completes and the media file has been uploaded to the media set.

Configuration

For more information on configuring media reference properties and setting up media uploads, see [Configure media reference properties](#) and [Upload media](#).



Send feedback

Permissions

Permissions for uploading media via an action are managed by the [action submission criteria](#). If users satisfy the action submission criteria, they do not need any permissions on the backing media set to upload media.

i Edit permission on a media set will be checked when it is added to an object type or referenced by an action type for the first time. Adding a media set to your ontology delegates access control from the media set to the ontology. This means that anyone who can manage actions on the object type, can control who is able to upload media to the media set.

PREVIOUS
← [Configure sections](#)

NEXT
[Upload attachments](#) →

© 2026 Palantir Technologies Inc. All rights reserved.

[Cookies Statement ↗](#)

[Privacy Statement ↗](#)

[Terms of Use ↗](#)

—[Cookies Settings](#)

[API Reference ↗](#)

[Send feedback](#)

Ontology building / Action types / Upload attachments

Upload attachments

Actions support uploading attachments from Workshop, Object Explorer, Object Views, Quiver, and Slate. Permissions to view, edit, and delete an attachment are consistent with the object to which they are uploaded. For example, if a user has view permissions to an object, they will be able to view and download attachments stored on this object. Replacing an existing attachment would require edit permissions on the object.

You can upload a single attachment or a list of attachments. To upload an attachment using actions, follow the configuration steps for both action types and object types.

Configuring action types

In the parameter configuration view, select **Attachment** as the parameter type. Attachments can only be uploaded using attachment parameter types. The corresponding column in the object-backing dataset must be a **String** and the edited object property must be of type **Attachment**.

To upload several media files at once, select **Allow multiple values**. Note that support for several media files in one action requires an additional toggle during [object type configuration](#) to **Allow multiple**, as described below.

Configuring object types

In the [object detail view](#), select **Attachment** as the property type. Attachments can only be uploaded to **Attachment** property types.

To upload several media files to one property, toggle **Allow multiple**. The property in the object-backing dataset must be an **Array**.

API Reference ↗

Send feedback

Architecture and limits

Attachments are immediately uploaded to Foundry once they are added to an action form. Upon form submission, permissions to view, edit, and delete an attachment are inferred from the user's permissions on the underlying object type. If form submission fails or is cancelled, the outstanding attachment is no longer directly reachable and will automatically be permanently deleted after some time. Similarly, attachments that belong to deleted objects or are no longer mapped to an object (which occurs when the corresponding property is deleted) are no longer directly reachable and will eventually automatically be permanently deleted.

Attachments are supported for both logic-backed and function-backed actions. There is a global, fixed file size limit of 200MB.

- Each attachment can be linked to a maximum of ten objects in its lifetime. If an attachment has been linked to ten objects, it cannot be linked to any other objects even if one or more of the original linked objects has been deleted. After reaching this limit of ten linked objects, you can upload the file again as a new attachment to link it to more objects.

PREVIOUS
← [Upload media](#)

NEXT
[Scale and property limits](#) →

© 2026 Palantir Technologies Inc. All rights reserved.

[Cookies Statement](#) ↗

[Privacy Statement](#) ↗

[Terms of Use](#) ↗

— [Cookies Settings](#)

[API Reference](#) ↗

[Send feedback](#)

[API Reference ↗](#)

[Send feedback](#)

Scale and property limits

Several limits are in place to ensure edited object types can quickly process edits and update user-facing data without slowing down live applications. Actions submitted that exceed these limits will not succeed and will display an error message to the user.

Configuration limits

The `Allow multiple values` toggle allows users to pass in a list of values to a parameter.

Limit	Maximum
Number of elements in a primitive list parameter	10,000
Number of elements in an object reference list parameter	1,000
Number of elements in a list parameter when used in submission criteria	1,000

Edit limits

Limit	Maximum
Number of object types you can edit in a single action submission	50
Number of objects you can edit in a single action submission	10,000
API Reference ↗ Number of an object in an action submission	32KB (OSv1), 3MB (OSv2)

Batch call limits

An action can be called a maximum of 10,000 times in a batch. This limit is reduced to 20 when the action is function-backed and the function is not configured to use [batched execution](#).

The edits applied in a batched action call are treated as a single group when enforcing [edit limits](#), regardless of which request in the batch was the cause of the edits.

Additional limits may apply, depending on the calling application.

Supported property types

Below are specifications for supported single and array property types. Note that some property types are only supported by Object Storage v2 (OSv2).

Single property types

Property type	Parameter type	Supported
Attachment	Attachment	Yes
Boolean	Boolean	Yes
Byte	Integer	Yes
Cipher text	String	Yes
Date	Date	Yes
Decimal	Decimal	Yes
Double	Double	Yes
Float	Double	Yes
Geopoint	Geopoint	Yes
Geoshape	Geoshape	Yes
API Reference ↗ reference	Geotemporal series reference	Yes (OSv2 only)
Integer	Integer	Yes

Send feedback

Property type	Parameter type	Supported
Long	Long	Yes
Mandatory control	-	Not supported as a property or in actions
Media reference	Media reference	Yes (OSv2 only)
String	String	Yes
Short	Integer	Yes
Struct	Struct	Yes (OSv2 only)
Timestamp	Timestamp	Yes
Time series reference	Time series reference	Yes (OSv2 only)
Vector	Double list	Yes (OSv2 only)

Array property types

Array property type	List parameter type	Supported
Attachment	Attachment	Yes
Boolean	Boolean	Yes
Byte	Integer	Yes
Cipher text	String	Yes
Date	Date	Yes
Decimal	Decimal	Yes
Double	Double	Yes
Float	Double	Yes
Geopoint	Geopoint	Yes
Geoshape	Geoshape	Yes
Geotemporal series reference	Geotemporal series reference	No
API Reference ↗	Integer	Yes
Long	Long	Yes
Mandatory control	Mandatory control	Yes

Send feedback

Array property type	List parameter type	Supported
Media reference	-	Not supported in actions
String	String	Yes
Short	Integer	Yes
Struct	Struct	Yes (OSv2 only)
Timestamp	Timestamp	Yes
Time series reference	-	Not supported as a property or in actions
Vector	-	Not supported as a property or in actions

Supported properties

Currently, actions cannot be used to edit the **primary key** of an object. Modifying the primary key is equivalent to deleting an object and then adding a new object; instead of editing the primary key with an action, you can create or delete an object directly using [rules](#).

Notification recipients

When using [side effect notifications](#), a maximum of 500 recipients can notified in a single action. This limit is reduced to fifty recipients when notifications content is rendered "From a function". For further information about limits to account for when generating notifications, see the documentation on [maximum recipient limits for notifications](#).

Inline edits

Action-backed inline edits are validated and submitted differently than standard [actions](#). For standard actions, multiple parameters need to be set in order for the action to be valid. However, for action-backed inline edits, every parameter is optional and defaults to the existing value of the object, so a user can make individual changes to properties one at a time.

This documentation discusses how to avoid unexpected results when using inline edits. Inline edits are available in both Workshop and Object Explorer. The configuration of the inline edit action depends on where the action is used.

Object Explorer inline edits

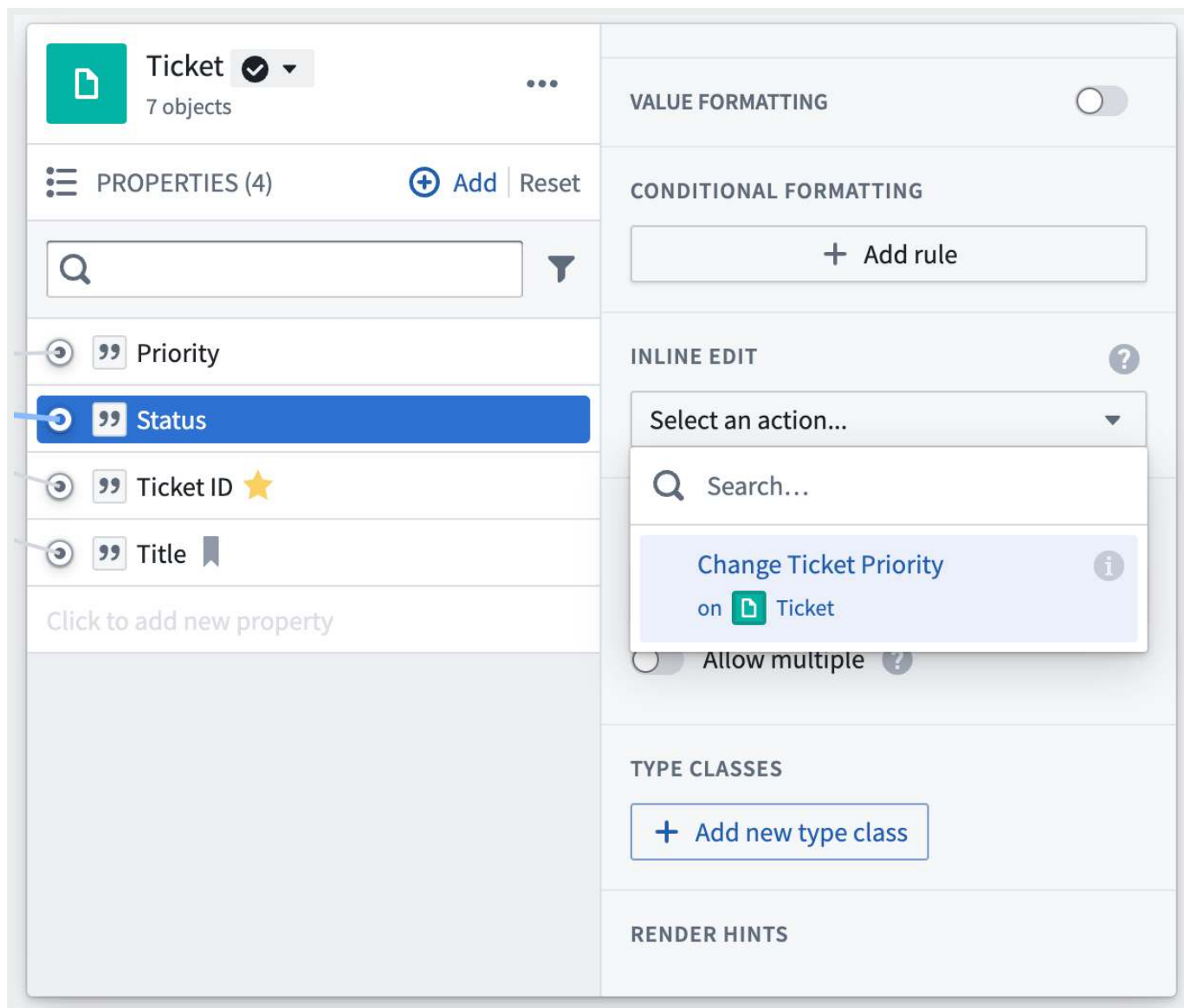
Inline edits allow users to quickly edit values of an object in the [Object Explorer results view](#) or native Object View widgets, like the property or metric cards widget.

Configuration

API Reference ↗



Send feedback



To set up an inline edit action, navigate to the **Properties** tab of your object type and then to the **Interaction** tab in Ontology Manager. Select a property and navigate to **Inline edit** in the sidebar. In the dropdown menu, select one of the available action types or create a new one. Creating a new one will trigger the action type creation workflow. Each property can have only one inline edit action type.

You can use the same action type as an inline edit for multiple properties, or you can have separate action types for different properties.

Action type requirements for inline edits

None API Reference can be used as inline edit action types. To be accepted, the action type must meet the following requirements:

- May only modify a single object of a single object type.

[Send feedback](#)

- Default values must be enabled.
- Default values must come from the object reference parameter on which the inline action is defined. As a result, properties that are being changed in the action cannot be mapped to static values or special values like "Current User" or "Current Time".
- Visibility status and overrides can be set; however, they will be ignored if the inline edit is used in Object Explorer and Object Views.
- [Side effect webhooks](#) or [side effect notifications](#) cannot be enabled.

Workshop inline edits

No additional configuration is required to use an action type as an inline edit in Workshop, but not all actions are suitable for cell-level editing. For information on how to configure inline edits, see the [Workshop documentation](#).

Background

When running a single action, edits are validated and submitted one at a time (sequentially). Inline edits differ in that they are validated and submitted in bulk. Because of this, not all actions are suitable for inline edits. Actions that may fail or have unexpected results due to inline edits include:

- Any action that attempts to read data to which another action could have written, or
- Two actions that try to write to the same object.

i When inline edits are applied to a [Scenario](#), the submitted actions are applied sequentially (in a non-deterministic order) rather than simultaneously (as is normally the case with inline edits). As a result, inline edit actions that ordinarily fail due to multiple actions trying to write to the same object may succeed when applied to a scenario, though we do not recommend building applications that depend upon this difference in behavior.

API Reference ↗

Validating Actions

Send feedback

Actions must submit non-conflicting edits to be effective as Action-backed inline edits. In practice, this means multiple Actions configured in the same table edit widget must not:

- Write to the same object,
- Create the same link, or
- Attempt to keep aggregate values consistent.

Invalid inline Actions

Actions will return an error if an inline edit attempts to edit the same object twice. Also, adding or deleting join table links is not supported by inline edits and will result in a user-facing error message.

As users apply inline edits, [submission criteria](#) will be applied to each edit, but the edits will be submitted in bulk. Both parameter and global submission criteria will be evaluated for each edited object, but submission criteria that reference shared or linked objects are not compatible with inline edits. This is because when applying inline edits, cumulative submission criteria compare the edited value to the unedited values for the column. At final submission, the edits will be submitted all at once and will succeed if they all pass parameter and global submission criteria for the corresponding object.

Submission criteria on objects shared between multiple Action types or linked objects are therefore evaluated once per edit, before any edits are made.

 Submission criteria that reference shared Action types or linked objects are not compatible with inline edits, and bulk updating objects could violate submission criteria rules that work as expected when applied sequentially (one at a time).

Example: Invalid inline Actions

Imagine a **Delay Flight** Action that can delay a single flight by a maximum of 20 minutes at an airport that can delay all flights by a maximum of 50 minutes.

API Reference ↗

criteria – the 20 minute requirement and the 50 minute total – will be evaluated each time a cell is updated.

Send feedback

- Because no edits are yet submitted, the 50 minute total will compare the new delays to the sum of unedited delays in the column (the delays from before inline editing began).
- The second submission criteria (that all the delays at the airport sum to less than 50 minutes) relies on an aggregated value and is shared by all the objects in the column.
 - Since inline edits are submitted in bulk, this second submission criteria will not be effective in limiting the total duration of flight delays at a given airport; the resulting edits could sum to greater than the 50 allowed by the second submission criteria.
- This Action would not be suitable for table editing as it would cause inconsistent results compared to running the Action individually for each cell.

PREVIOUS

← [Scale and property limits](#)

NEXT

[Permissions](#) →

© 2026 Palantir Technologies Inc. All rights reserved.

[Cookies Statement](#) ↗

[Privacy Statement](#) ↗

[Terms of Use](#) ↗

— [Cookies Settings](#)

[API Reference](#) ↗

[Send feedback](#)

Permissions

Permissions apply to action types in the following ways:

- Who can view a given action type?
- Who can edit a given action type?
- Who can apply an action type with a given set of parameters?

Apply action

The ability to apply an action type depends on the configuration of the object types and link types it is editing. In all cases, the user submitting the action must be able to view the edited object types and link types and their datasources, and pass the [submission criteria](#). If the object type only allows edits via actions, users can make edits for all the objects they can view. For object types and link types allowing edits beyond actions, the user also needs edit permissions on the writeback dataset if the object type or link type is backed by a dataset. If the object type or link type is backed by a [Restricted View](#), the user needs to pass the edit policy.

i Use the **Check access** panel in the sidebar to check a user's access to a Workshop module, including access to dependent action types and their submission criteria. For more information, review the [check access panel documentation](#).

Submission criteria

API Reference ↗

Submission criteria allow for fine-grained control over who can apply an action type. Submission criteria can require a specific user ID or group ID and can be combined with

Send feedback

information from parameters. For more information see the [submission criteria documentation](#).

Read and write authorizations

Read and write authorizations limit which marked data an action can read directly or receive as input from other logic and enforce minimum security for data it creates or modifies. For more information see the [read and write authorizations documentation](#).

Object edits permissions

Object edits can either be locked down so that edits are only allowed via actions, or reopened so that edits are allowed via actions, Foundry Forms, direct Object Explorer edits, and API calls. To enforce a consistent security paradigm across many workflows, by default, new object types only allow edits via actions. Other forms of edits are not recommended for new usage.

For object types that only allow edits via actions, the user submitting the action will only need **Read** access on the objects that are being edited. This means that it is possible for users to create objects that they cannot view.

By contrast, when an object type backed by a dataset can be edited by actions, Foundry Forms, direct Object Explorer edits, and API calls, the user submitting the action must have **Edit** permissions on the writeback datasets of all objects being edited. A user with **Edit** permissions will be able to view all data in a writeback dataset.

Therefore, setting an object type to be edited by actions, Foundry Forms, direct Object Explorer edits, and API calls is discouraged since granting **Edit** permissions simply for object editing may expose more data to a user than is required to complete the Ontology editing workflow.

With either writeback setting, an action type's configuration does not display permission settings on affected underlying object types; the person configuring the action type must ensure permissions are correct.

[API Reference ↗](#)

Updating edit permissions on an object type to "Only allow edits via actions" prevents historical, non-action edits, but they will prevent further edits from Foundry Forms, direct

[Send feedback](#)

Object Explorer edits, and API calls.

Permission breakdown

Object type

Object instances

View object type

View the object type schema and metadata

Users must meet **all** of the following requirements to view the definition of this resource.

B [] Test Object

Viewer permissions ⓘ

AND

Organizations · Any of ⓘTitanPalantirPalantir 2

Markings ⓘNone

Edit object type

Edit the object type schema definition and metadata

Users must meet **all** of the following requirements to edit the definition of this resource.

B [] Test Object

Editor permissions ⓘ

AND

Organizations · Any of ⓘTitanPalantirPalantir 2

Markings ⓘNone

[Learn more about writeback permissions.](#)

Side effect permissions

Any user who can set up an action may configure side effects.

- Webhook side effects are not enabled by default. Additional permissions are required to

API Reference ↗

API Reference ↗ Look plugin in the Data Connection app before it can be used in the Foundry setup page. Contact your Palantir representative with any questions about using webhooks on your Foundry instance.

Send feedback

Send feedback

Submission criteria must pass as normal; if the action submission criteria fail, then side effects will not be triggered.

Recipients must have access to any object data included in the notifications.

- If a user does not have access to all data included in the notification content, the notification will not be sent to them.
- If there are multiple recipients and some are missing the correct permissions data included in the notification, only the users with sufficient permissions will be notified.
- If notifications fail to send for whatever reason, edits may still succeed.

The user executing the Action must be able to view the users and/or groups that will be receiving a notification.

PREVIOUS
← [Inline edits](#)

NEXT
[Read and write authorizations](#) →

© 2026 Palantir Technologies Inc. All rights reserved.

[Cookies Statement ↗](#)

[Privacy Statement ↗](#)

[Terms of Use ↗](#)

—[Cookies Settings](#)

[API Reference ↗](#)

[Send feedback](#)

Read and write authorizations

Beta

Read and write authorizations are in the [beta](#) phase of development and may not be available on your enrollment. Functionality and platform support may change during active development.

Read and write authorizations add security boundaries to an action type. [Read authorization](#) limits the marked data that an action can read directly or receive as input when invoked by another piece of logic. [Write authorization](#) sets minimum security requirements for data created or modified by an action. These authorizations supplement the user's existing permissions and the action type's [submission criteria](#). They do not grant access or replace existing permissions or submission criteria.

Read authorization

Read authorization provides an additional upper bound on the data an action can read during execution or receive as input. Read authorization applies when Foundry loads values into parameters, including when another piece of logic invokes the action, and when function-backed actions perform reads. If data exceeds the configured read authorization, the action cannot load that data even if the user could otherwise access it.

The user applying the action must still have permission to access the data. If no markings are configured for read authorization, the action has no additional read boundary beyond the user's access.

API Reference ↗



Send feedback

Write authorization

Write authorization defines the minimum security that data created or modified by an action must meet. Foundry validates the resulting data security and blocks the action if the result is below the configured minimum.

Write authorization does not grant a user access to markings, automatically add any configured markings, or make an otherwise invalid edit permissible. The user must have the required permissions, provide valid marking values where applicable, and pass the action type's submission criteria. If no markings are configured for write authorization, this feature does not enforce additional minimum output security.

A configured write authorization is not enforced for writes managed by the platform, including [action logs](#) and [edit history](#) objects. This means that security applied to action logs and edit history may be below the write authorization.

Mandatory marking guarantees

[Mandatory markings](#) are intended to propagate along data dependencies so derived data retains the protections of its inputs. For actions, the action's logic sets the security applied to output data. An action can therefore produce output at the same, higher, or lower security than the data it receives or reads. Read and write authorizations set upper and lower bounds on what an action can do. Together, they guarantee the permitted security differences between the action's inputs and outputs.

Configuring write authorization to be less restrictive than read authorization allows the action to write data below the security boundary of data that the action can read. This intentionally severs mandatory marking propagation and may result in data declassification. The action type editor displays a warning whenever the read and write settings differ.

When read and write authorizations differ, Foundry checks the permissions of the user saving or publishing the action type. The user must have permission to declassify every mandatory marking severed between the read and write boundaries. If the check succeeds, the action type may perform the approved declassification; Foundry does not repeat this permission check at runtime.

API Reference ↗

Read and write authorizations are currently optional. If read authorization is not configured, Foundry does not perform declassification permission checks. If the action type is saved or published. The action can therefore read data above its write

Send feedback

boundary and write less restrictive data, which can cause a data spill. Configure read authorization whenever an action may read marked data.

Authorization rules by marking category

Foundry combines configured markings according to their category:

Marking category	Read authorization	Write authorization
Classification markings	Follows the environment's configured hierarchy and conjunctive or disjunctive category rules.	Follows the environment's configured hierarchy and conjunctive or disjunctive category rules.
Other mandatory markings	All configured markings are required.	All configured markings are required.
Organizations	All configured Organizations are required.	At least one configured Organization is required.

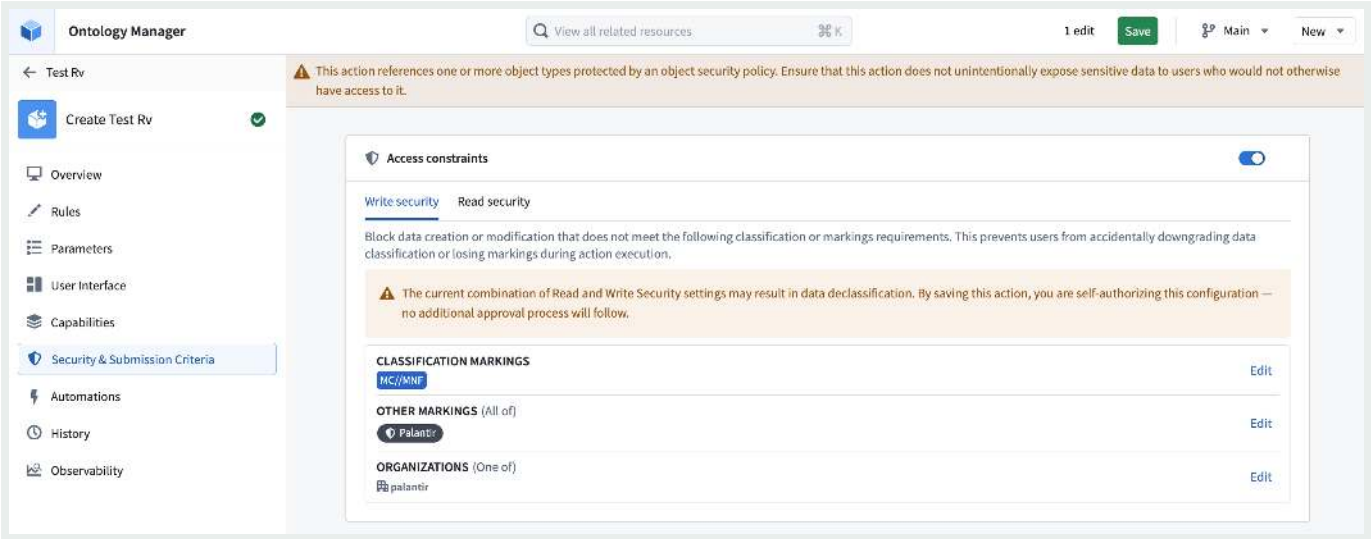
Classification behavior can vary between environments. Review [Classification-based Access Controls](#) for details about classification hierarchies and conjunctive or disjunctive categories.

Configure read and write authorizations

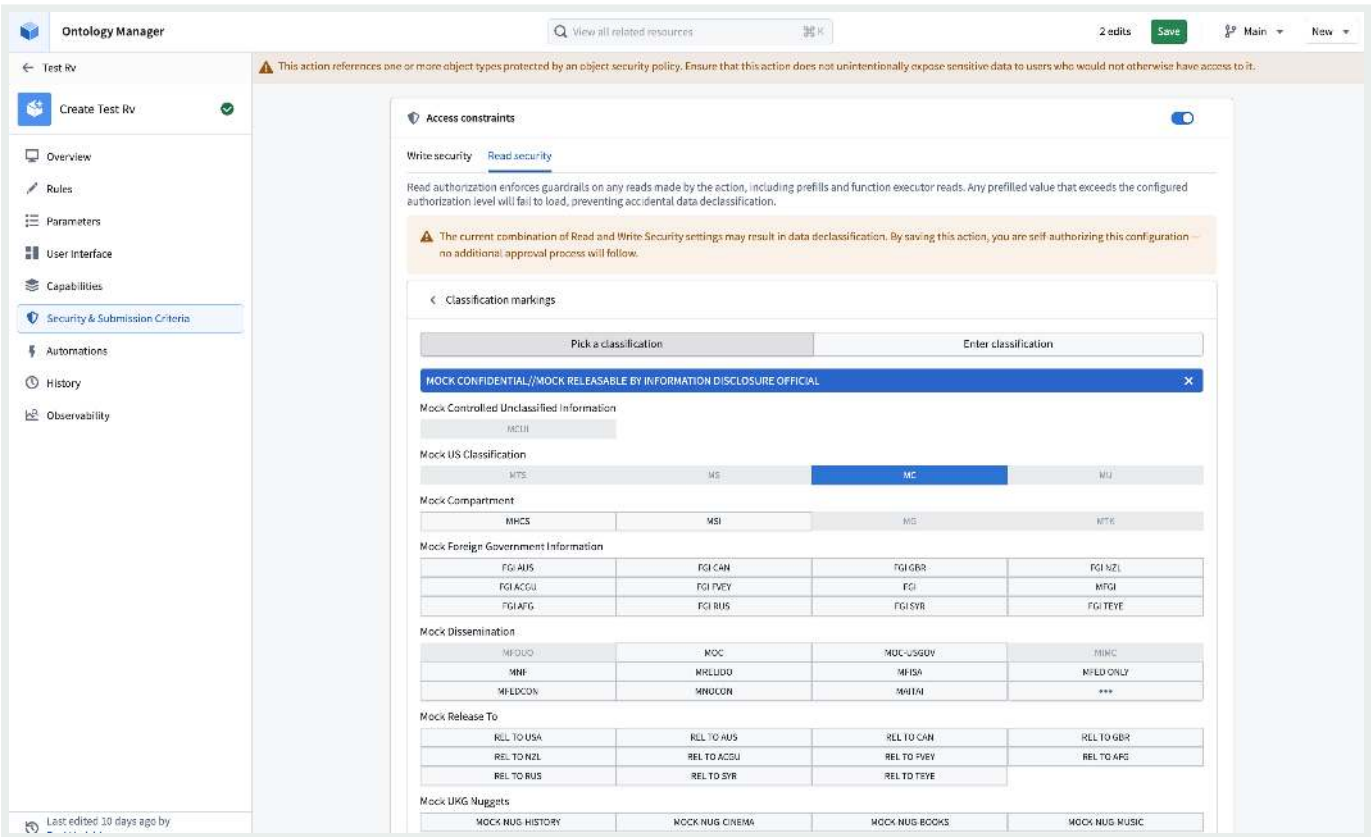
In the action type editor, enable **Access constraints** to configure authorizations. Enabling access constraints initializes both authorizations with no markings. Disabling access constraints clears both authorizations.

[API Reference ↗](#)

[Send feedback](#)



Use the **Write security** and **Read security** tabs to configure each authorization separately. The editor displays a warning when the settings differ. If an authorization contains values that you cannot view, those values appear as redacted and cannot be edited.



API Reference ↗

Before submitting an action, the action form can display **Minimum read** on write authorization. The status can be:

Send feedback

- **Security passed:** The destination security is expected to meet the configured minimum.
- **Security not passed:** The destination security is below the configured minimum.
- **Can't be determined:** Foundry cannot determine the destination security before submission.
- **Not set:** No write authorization markings are configured.
- **Redacted:** The user cannot view the configured authorization.

Create Test Rv

Name *

Date *

Select a date (MM/dd/yyyy)

Location *

DEG ▼

Minimum required security

Can't be determined

This requirement prevents the action from creating data below the minimum specified security. Security will be validated automatically.

MS//REL TO USA, AUS

1

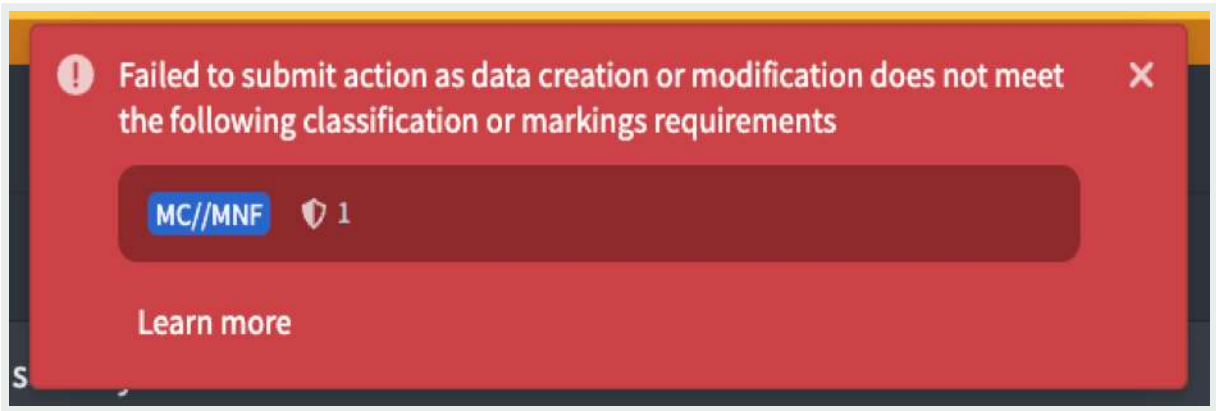
Cancel

Submit

The form status provides early feedback. Foundry performs the authoritative validation when the action is submitted and fails the action if the resulting data does not meet write authorization.

[API Reference ↗](#)

[Send feedback](#)



PREVIOUS
← Permissions

NEXT
Monitoring →

© 2026 Palantir Technologies Inc. All rights reserved.

[Cookies Statement ↗](#)

[Privacy Statement ↗](#)

[Terms of Use ↗](#)

— [Cookies Settings](#)

[API Reference ↗](#)

[Send feedback](#)

Ontology building / Action types / Monitoring

Action monitoring

Actions in Foundry can be monitored to track performance and reliability. This page explains the available monitoring capabilities for actions.

Available monitoring rules

Action monitoring in Foundry supports two key rule types:

1. **Action duration p95:** Alerts when the 95th percentile execution time exceeds thresholds.
2. **Number of action failures in window:** Alerts when failure count exceeds thresholds within a timeframe.

For detailed configuration options and parameters, review our [monitoring rules reference documentation](#).

Set up action monitoring

To set up monitoring for your actions, follow the standard process for creating monitoring views and rules:

1. Create a monitoring view as described in the [monitoring views overview documentation](#).
2. Add a monitoring rule for an action or action type as described in the section on [adding a monitoring rule](#).

3. [API Reference](#) ▸ [Configure thresholds and severity levels](#).
4. [Alerts](#) ▸ [Configure alert notifications](#) following the [alert subscription guide](#).

Send feedback

Foundry Notifications

This will trigger an in-platform notification and might trigger an email, based on your Foundry Notification settings.

Low

Medium

High

Resolution

User name

userId

☐

☐

☒

☒

+

Add users or groups

PagerDuty Notifications

This will route a page to PagerDuty. Who gets paged is based on your on-call settings configured in PagerDuty.

Severity ⓘ

Severity type of the PagerDuty integration

My Pagerduty

Medium

-

High

▶

Test

+

Create new PagerDuty integration

Slack Notifications

This will route an alert to the specified channels in your Slack source.

+

Create new Slack integration

Webhook Notifications

This will trigger an alert by calling your custom webhooks.

+

Create new Webhook integration

Dynamic scopes

Action monitors support **Workflow Lineage**, **Workshop**, and **OSDK application** as dynamic scopes. When you select one of these scopes, the monitor automatically tracks all actions the scoped resource uses and adjusts as actions are added or removed without further intervention.

Select scope

Select a resource type and monitoring scope in order to begin applying monitors

Action types

Selected action types

+ Add



Add action types

Add action types in order to apply monitoring rules to the

-  **Project** Dynamic
Monitor all action types within a given project in Foundry
-  **Workshop module** Dynamic
Monitor all action types used within a given Workshop module
-  **Workflow Lineage** Dynamic
Monitor all action types used within a given Workflow Lineage graph
-  **Developer Console** Dynamic
application
Monitor all action types used within a

Related documentation

- [Monitoring rules reference](#)
- [Monitoring views overview](#)

PREVIOUS

← [Read and write authorizations](#)

NEXT

[Undo or revert Actions](#) →

© 2026 Palantir Technologies Inc. All rights reserved.

[Cookies Statement](#)

[Privacy Statement](#)

[API Reference](#)

[Send feedback](#)

Ontology building / Action types / Undo or revert Actions

Revert or undo actions

Action reverts in [Ontology Manager](#) allow an action to be reverted (that is, undone) immediately after the action has been applied. You can revert an action by selecting **Undo** in the success message after any successful action application.

New actions are revertible by default.

i Action reverts are only available for Object Storage v2; meaning that only actions that modify or create an object type in [OSv2](#) can be reverted. If your object types are not currently stored in Object Storage v2, you can migrate by following this [guide](#).

Configure a revertible action

Currently, actions can only be reverted by the user who applied the action.

In the **Form** tab of an action, toggle on the **Allow revert after action submission** button. Once this toggle is correctly configured and saved to the Ontology, your action can be reverted.

API Reference ↗



Send feedback

The **Allow revert after action submission** toggle in the **Form** tab will be enabled by default for actions created after May 2024 that only modify OSv2 object types. If an action existed before May 2024 and modifies an object type in OSv2, action reverts will not be toggled on by default but can be manually enabled.

You will not be able to revert an action if it only modifies OSv1 object types.

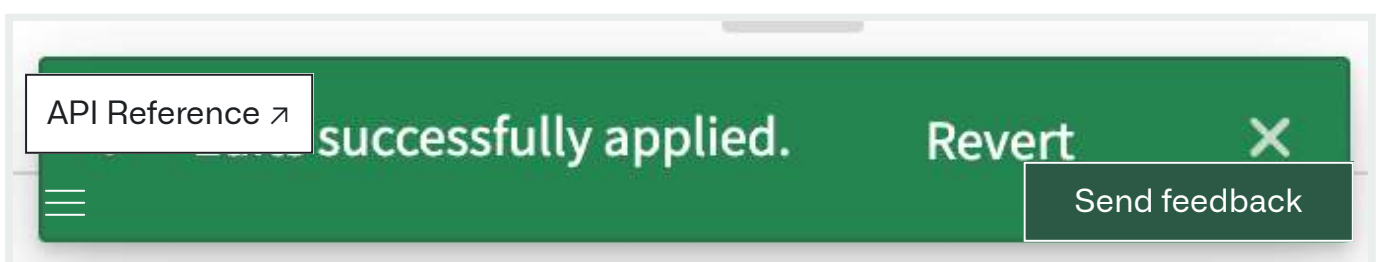
Revert an action

Revert action

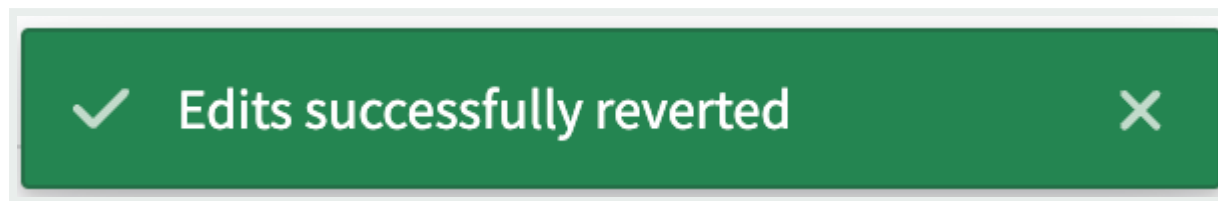
The toast below is your only opportunity to revert the action. This is especially important to note when performing delete actions.

Once reverted successfully, users will see a similar toast to the original action success as shown below.

Edits applied:



Edits reverted:



Caveats

An action revert may fail in some cases:

- An action on an object cannot be reverted once any subsequent edit has been made to the object, even if the edit is on a different property. In other words, an action on an object can only be reverted if the action is the most recent edit to an object.
- An action cannot be reverted if action reverts has been toggled off after action submission, even if action reverts have been toggled on again.

An action revert only reverts the edits to the object instance, but it will not revert side effects, such as notifications or webhooks, nor will it call them in the same way that the applied action would have.

Undoing a delete action without the revert action toast

If a delete action is performed and you wish to undo the deletion, but the revert action toast is no longer available, the only remediation options available are to:

- Migrate to a new object type and copy over the desired edits using functions; or
- Drop all edits on the object type.

← PREVIOUS
Monitoring

NEXT →
Branching action types

API Reference ↗

© 2026 Palantir Technologies Inc. All rights reserved.

[Cookies Statement](#) ↗

Send feedback

[Privacy Statement ↗](#)

[Terms of Use ↗](#)

— [Cookies Settings](#)

[API Reference ↗](#)

[Send feedback](#)

Ontology building / Action types / Branching action types

Branching action types

Ontology actions integrate with Global Branching, enabling you to test action types on a branch without affecting your production environment. You can run actions, validate their configurations, and observe edits in an isolated branch context before merging changes to `main`.

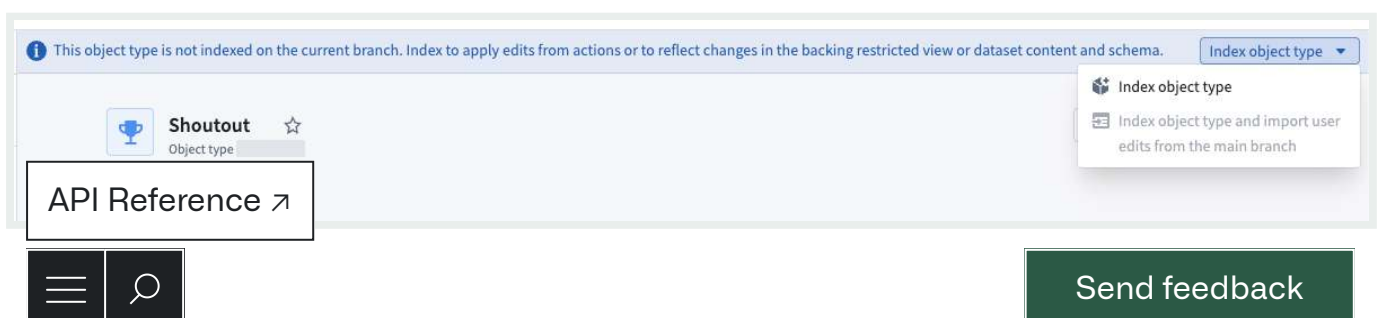
For general information on Global Branching concepts and workflows, refer to the [Global Branching documentation](#).

Running actions on a branch

You can test actions in a Workshop module on a branch to validate that they have been configured correctly. When all relevant object types are indexed on a branch, you can run the action type and see the resulting edits on the branch.

Prerequisites

To run an action on a branch, all object types that are edited by the action type must be indexed on that branch. You can index object types through the individual object type page or through the action type page in Ontology Manager.



i Running actions on a branch is intended as a testing mechanism. Action edits on a branch will not be merged back into `main`.

Working with function-backed actions on a branch

Function-backed actions on a branch behave differently depending on whether they are branch-aware. Branch-aware functions can be modified on the branch and read schemas from it, while non-branch-aware functions only read schemas from `main`. Regardless of branch-awareness, all function-backed actions execute only on the branch and do not write changes back to `main`. See supported function types in the [Global Branching documentation](#).

Managing side effects on branches

i The following information applies specifically to webhooks, functions with external calls, and notifications that are applied **via actions** on branches. Side effects applied through other means will behave as defined by those consumers.

Webhooks

By default, webhooks do not execute when an action is applied on a branch. This behavior is to prevent accidentally writing to external systems while in a testing environment.

In such cases, you will see a toast notification indicating this behavior.

✓ **Action applied but webhook not triggered:** External system calls are disabled for this action type on branches

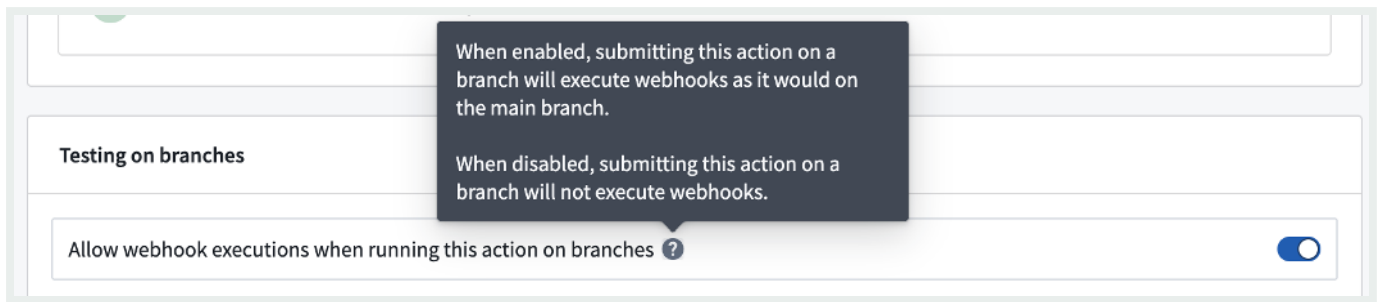
Configure ↗



API Reference ↗ In some cases where testing a webhook on a branch is desirable, for instance, testing a READ endpoint.

Send feedback

To override the default behavior, configure the action type's **Security and submission criteria** tab in Ontology Manager to enable webhook executions on branches.

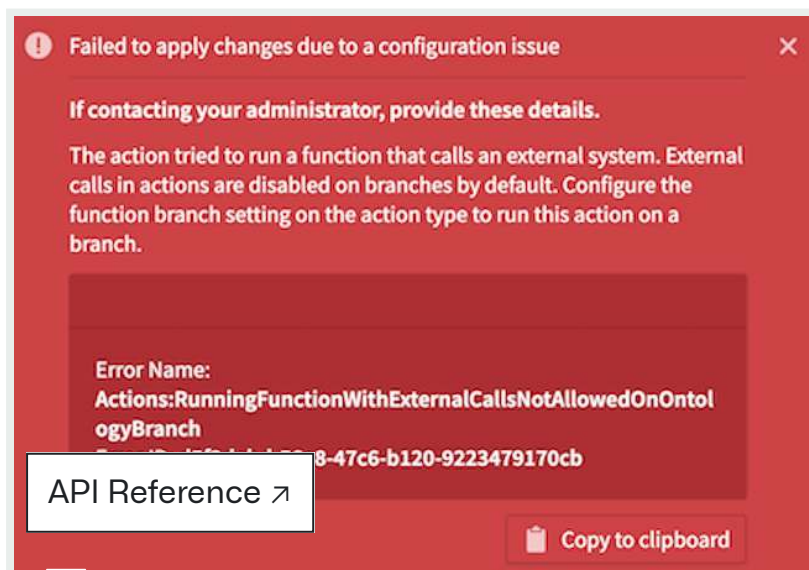


- i** If webhooks on branches are enabled, the webhook will run exactly as it would on `main`. Consequently, if the webhook is configured to hit an external production environment, it will continue to do that even if the action is executed on a branch.

Functions with external calls

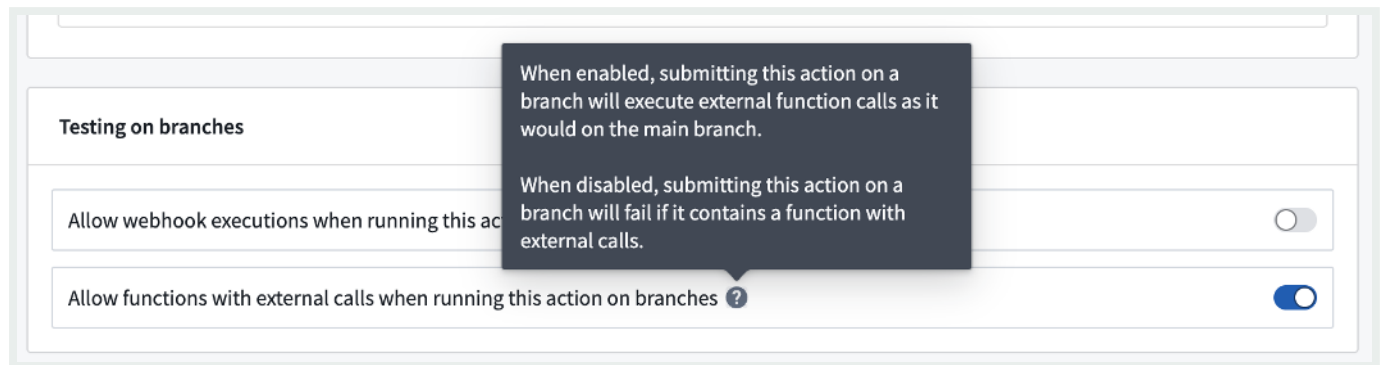
By default, function-backed action logic with external calls does not execute when an action is applied on a branch; the action fails entirely. This behavior is to prevent accidentally writing to external systems while in a testing environment.

In such cases, you will see a toast notification indicating failure, with an explanation of the behavior.



Send feedback

However, testing on branches is sometimes necessary, for example, when calling READ endpoints. You can override this restriction by enabling functions with external calls on branches in the action type's **Security and submission criteria** tab in Ontology Manager.

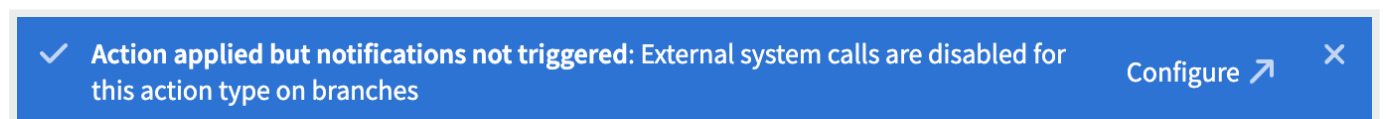


- i** If functions with external calls on branches are enabled, the function will make the same external calls as it would on `main`. Consequently, if the function is configured to hit an external production environment, it will continue to do that even if the action is executed on a branch.

Notifications

By default, notifications do not execute when an action is applied on a branch. This behavior is to prevent accidentally notifying recipients while in a testing environment.

In such cases, you will see a toast notification indicating this behavior.



However, there are some cases where testing notifications on a branch is desirable.

To override the default behavior, you can enable notifications on branches in the action type's **Security and submission criteria** tab in Ontology Manager.

At the [API Reference](#), specify the notification recipients when the action runs on a branch:

- **Branch owner:** Send all notifications to the branch owner.
- **Default recipients:** Notify the recipients configured on the original notifications.

[Send feedback](#)

Testing on branches

Allow webhook executions when running this action on branches ?

Allow functions with external calls when running this action on branches ?

Allow notifications when running this action on branches ?

Notification recipients on a branch

Choose who receives notifications when running this action on branches

Branch owner ▾

Branch owner

Default recipients

Known limitations

- Action edits on a branch are for testing only and will not be merged back into `main`.
- Webhooks and email notifications are not executed by default when an action is run on a branch.
- Functions that make calls to external systems will fail by default when an action is run on a branch.

PREVIOUS
← Undo or revert Actions

NEXT
Action metrics →

© 2026 Palantir Technologies Inc. All rights reserved.

[Cookies Statement ↗](#)

[Privacy Statement ↗](#)

[Terms of Use ↗](#)

[API Reference ↗](#)

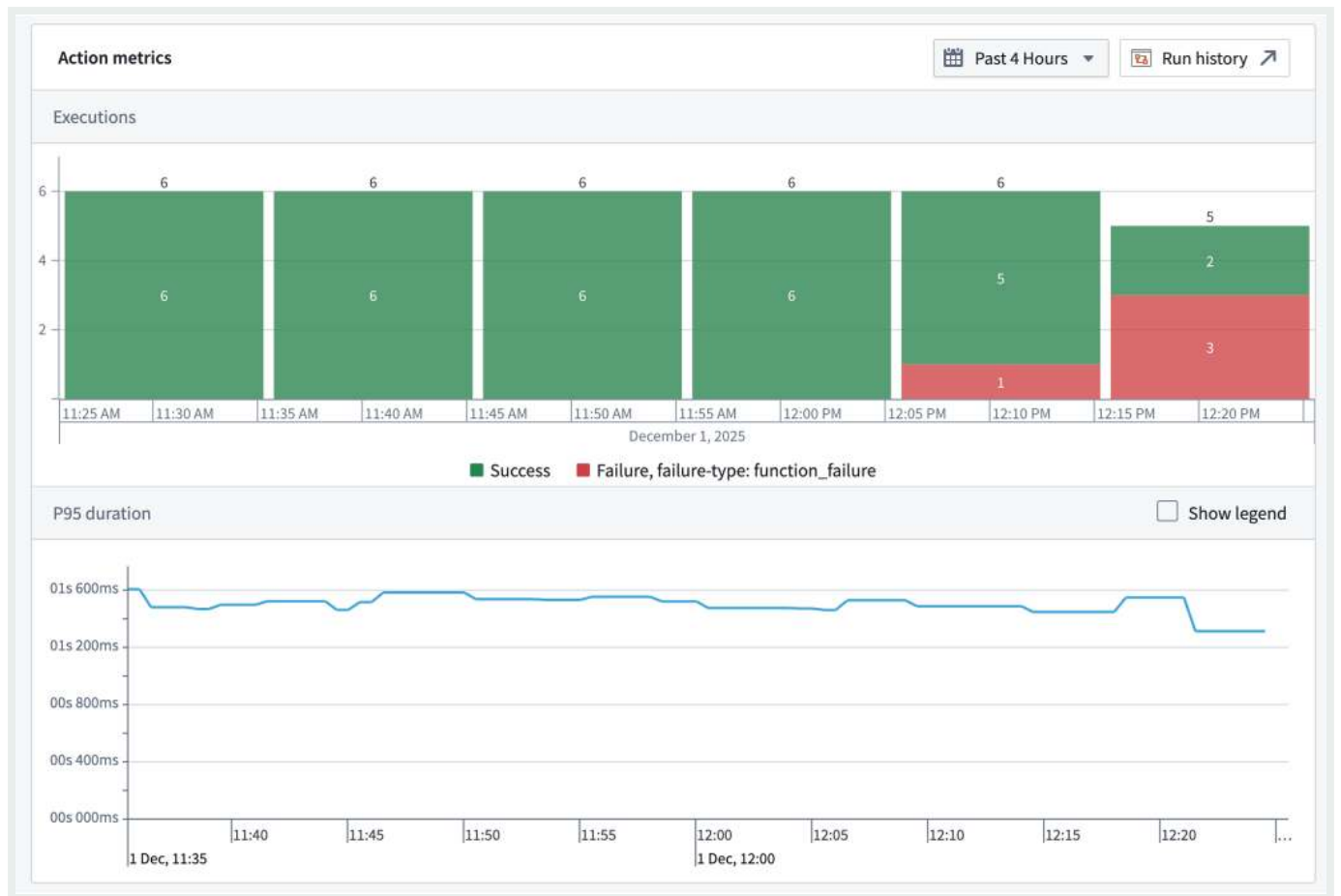
Send feedback

Action metrics

Action metrics display the near real-time usage of an action type over the last 30 days. You can access these metrics from the action type's overview page in [Ontology Manager](#), or in [Workflow Lineage](#) by selecting the action node for a given execution. The following metrics are available:

- **Success/failure metrics:** Monitor the current status of your actions with success and failure counts. This enables rapid identification of issues and supports proactive troubleshooting, allowing you to address failures as soon as they occur.
- **P95 duration metric:** Track the 95th percentile (P95) execution duration for each action type. This metric highlights the upper range of execution times, helping you detect performance bottlenecks and optimize workflows for consistent and efficient operation.

You are also able to access [run history](#), which provides a complete view of a given action's executions over the past seven days. Learn more about [AIP observability capabilities](#).



All metrics are updated in near real-time using the latest data from the Foundry Telemetry Service (FTS). This ensures you have access to the most current information for monitoring, debugging, and maintaining the health of your actions.

You can also embed action metrics directly inside an operational application using the [Observability Chart widget](#) in [Workshop](#) to monitor resource health alongside the rest of a workflow.

Action failure types

Action metrics do not require action logs to be displayed. Unlike action logs, action metrics track failures.

Action metrics have a variety of categories of failures that may be displayed. These categories are:

[API Reference ↗](#)

- **Invalid parameter failure:** The action was submitted with a parameter or parameters that are not valid within the context of the action.

[Send feedback](#)

- **Scale limit failure:** The action affected more than the permitted limit of object types (by default, usually 10,000).
- **Authentication failure:** The user did not pass the security submission criteria for the action.
- **Side effect failure:** The action failed due to a webhook or an incorrectly configured side effect.
- **Function failure:** The action failed because the underlying function failed. This failure mode is only possible for function-backed actions.
- **User-facing function failure:** The function backing the action threw an error intended to be displayed to the user. This failure mode is only possible for function-backed actions.
- **Conflict failure:** The action failed due to a conflict, such as a concurrent modification.
- **Unclassified failure:** The action failure did not fall into any of the above categories.

Permissions

To view action metrics, you must be a `viewer` on the action.

PREVIOUS
← [Branching action types](#)

NEXT
[Action log](#) →

© 2026 Palantir Technologies Inc. All rights reserved.

[Cookies Statement ↗](#)

[Privacy Statement ↗](#)

[Terms of Use ↗](#)

—[Cookies Settings](#)

[API Reference ↗](#)

[Send feedback](#)

Ontology building / Action types / Action log

Action log

The action log models all action submissions as object types to be analyzed and displayed in object-aware Foundry tooling. Use an action log object type as an input to decision-making workflows and to monitor changes to your Ontology.

i The action log is designed to capture decisions made by submitting actions and make these decisions available as data in the Ontology. For use cases where logging all edits to an object is desired, [edit history](#) can be enabled for an object type.

Background

Actions are the primary way to modify the Ontology and trigger related side effects. Often, these Ontology modifications are the result of a specific decision or are accompanied by data audit requirements. The action log simplifies generation and maintenance of object types that represent these decisions and data edits. For easy identification, all action log object types are prefaced with `[LOG]`.

Action log Ontology

Action log object types map one-to-one with action types. Submitting an action generates a single new object of the corresponding action log object type. This newly-created object is a [API Reference ↗](#) to all objects edited by the submitted action. By modeling log object types one-to-one with action types, the action log supports capturing context beyond simple object edits, such as which other objects were concurrently [Send feedback](#) the world (as represented by the Ontology) at the time of action submission.

For example, imagine a `Close Alerts` action type that modifies the "Status" property of many selected `Alert` objects to "Closed". When configured with an action log, closing 10 `Alert` objects at once will yield a single `action log` object with foreign key links to all 10 `Alert` objects.

i To apply an action log-backed action type, users need the appropriate permissions for the action log object type, just as they do for any other object types that the action type might create or modify through rules and functions.

Action log schema

By default, action log object types store:

- **Action RID:** Unique identifier for a single action submission
- **Action type RID:** Unique identifier for a single action type
- **Action type version:** Version number that auto-increments each time an action type is updated
- **Timestamp:** UTC timestamp of action submission
- **UserId:** Multipass user ID for action submitting user
- **Edited objects:** Primary key values of all objects edited by the action. Note that storing properties of edited objects other than the primary key is not supported.
- [Optional] **Summary:** A customizable string to describe the action
- [Optional] **Parameter values**
- [Optional] **Property values of object reference parameters** (this is not supported for object reference parameters if `allow multiple values` is enabled)

Action log object types can be configured to store object properties that are not edited by the action. This allows you to store data edits as well as relevant information about the context of or motivation for the Ontology edits.

Returning to the example of a `Close Alerts` action type, imagine the `Alert` objects also have a "Priority" property containing values "High Priority" and "Low Priority" as well as a "Source" machine. The action log supports storing these properties, even if they are not edited by `Close Alerts`. By aggregating on "Priority" without editing the column we can answer questions such as "where is the source of all High Priority alerts?" or "how long does it take to close "High Priority" alerts?".

Send feedback

Action log on function-backed action types

To configure the action log for a function-backed action type, the backing Ontology edit function must have `Edits` provenance configured. See the [functions documentation](#) for more information on `Edits` provenance.

Action log timeline

You can view action log object types in a timeline using a custom Workshop widget. With this widget, the timeline can be configured to support data audits in order to help answer the questions "what changed, by whom, and when?"

Within Workshop, action log object types can be unioned together for a holistic view of edits within a use case or across an Ontology.

Configure the action log timeline by selecting the edited object type. Then choose which action log object types to display, along with the desired action log object type properties.

[API Reference ↗](#)

[Send feedback](#)

Ontology building / Action types / Add action types to a Marketplace product

Add action types to Marketplace product

Use [Foundry DevOps](#) to include your action types in [Marketplace products](#) for other users to install and reuse. [Learn how to create your first product.](#)

Supported features

Most action type features are supported, except actions that reference [object types with unsupported features](#). When preparing your action type for packaging, ensure your action type **Security & Submission criteria** does not reference a user; update any user references to refer to groups instead.

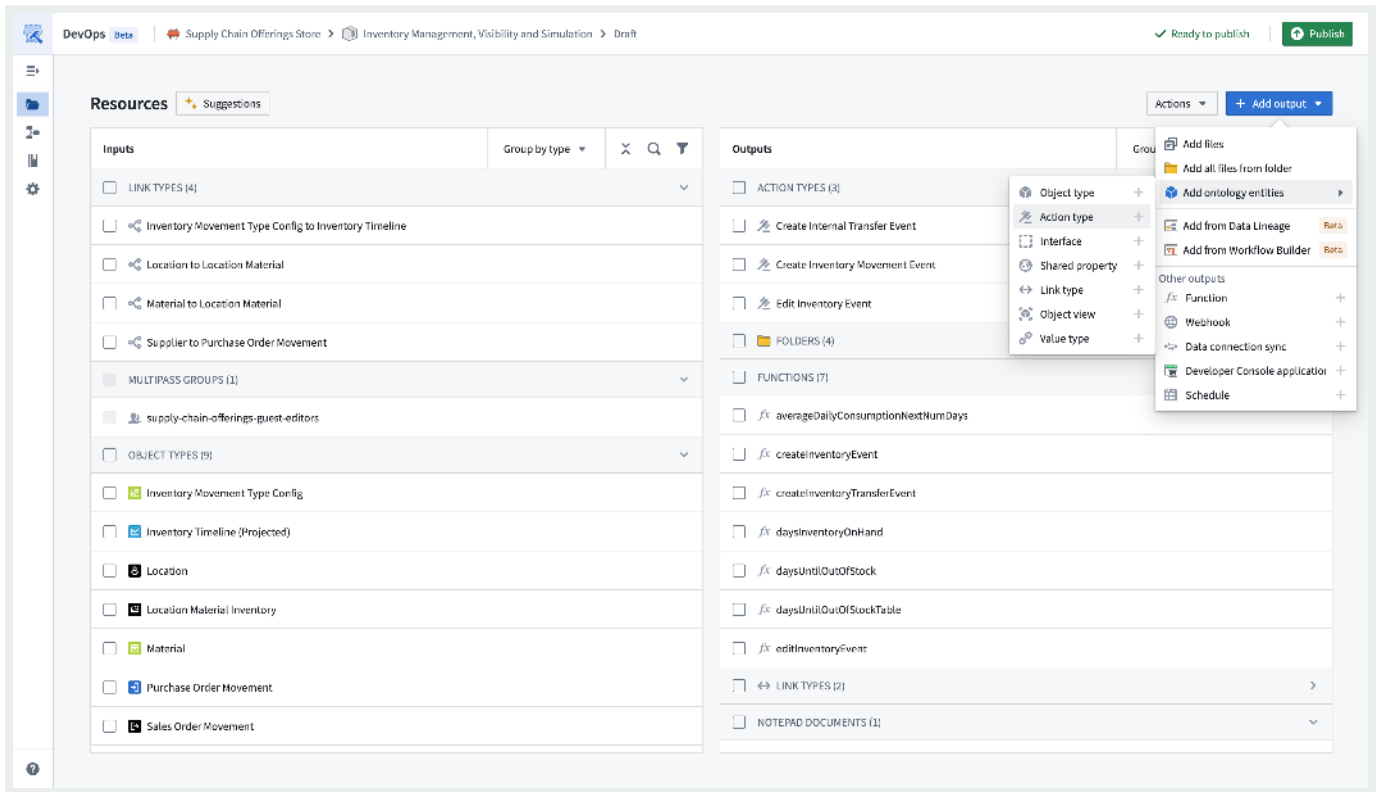
Adding action types to products

To add an action type to a product, first [create a product](#) and then select the **Action type** content type as below.

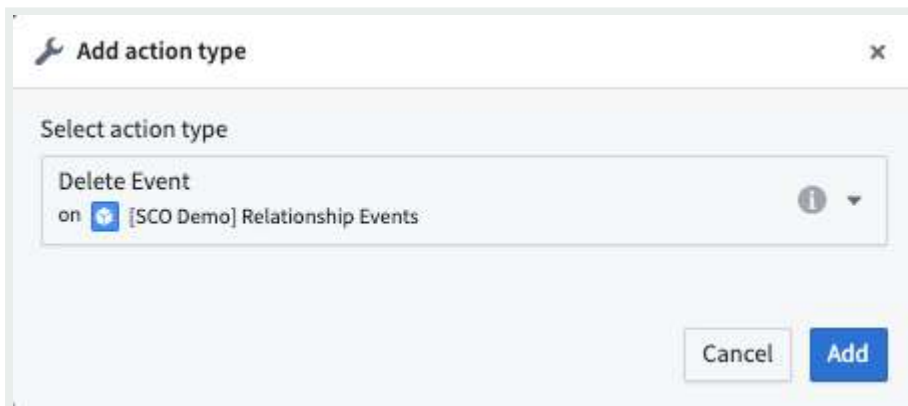
API Reference ↗



Send feedback



You will then be prompted to choose an action type.



While you can select action types directly, we recommend first adding content like [Workshop applications](#) and then selecting relevant actions via the dependencies panel as shown below.

[API Reference](#) ↗

[Send feedback](#)

← All products

My product

Ontology product

Creating new product Draft

General information

Content

Inputs

Documentation

Installation mode

Folder structure Beta

Review and publish

Workshop Applications (1)

+ Add workshop application

NAME	DEPENDENCIES
Inventory Management Application	14 dependencies

Inventory Management Application Included

Remove

General

Dependencies

Inputs that are required for the generation of this workshop application Edit multiple

Ontology

Object types 4

Material

Input

↔ 1

Add

Location Material

Input

≡ 7 and ↔ 3

Add

Inventory Timeline

Input

≡ 10 and ↔ 1

Add

Location

Input

≡ 1 and ↔ 1

Add

Link types 3

Material to Location Material

Input

Add

Location to Location Material

Input

Add

[Notional] Location Material to Inventory Timeline

Input

Add

Action types 2

Create Inventory Movement Event

Input

Add

Edit Inventory Event

Input

Add

Functions

>

Workshop Applications 1

Inventory Chart

Input

Add

PREVIOUS

← Action log

NEXT

Functions / Overview →

© 2026 Palantir Technologies Inc. All rights reserved.

[Cookies Statement ↗](#)

[Privacy Statement ↗](#)

[Terms of Use ↗](#)

API Reference ↗

Send feedback

